



Incentivized Mechanism Design for Collaborative Proofs and Programs through Blockchain and Theorem Provers

Submitted by

Jin Xing LIM

Thesis Advisor

Associate Professor Georgios PILIOURAS

Engineering Systems and Design (ESD)

A thesis submitted to the Singapore University of Technology and Design in
fulfillment of the requirement for the degree of Doctor of Philosophy

2022

PhD Thesis Examination Committee

TEC Chair:	Prof. Karthik Natarajan
Main Advisor:	Assc. Prof. Georgios Piliouras
Co-advisor(s):	Dr. Shaowei Lin
	Dr. Barnabé Monnot
Internal TEC member 1:	Asst. Prof. Karthyek Murthy
Internal TEC member 2:	Asst. Prof. Ernest Chong

Abstract

Engineering Systems and Design (ESD)

Doctor of Philosophy

Incentivized Mechanism Design for Collaborative Proofs and Programs through Blockchain and Theorem Provers

by Jin Xing LIM

Scientific research, and particularly research in mathematics, is arguably one of the crowning achievements of our collective human intellect. Its creation increasingly requires collaboration between multiple researchers with different and sometimes complementary backgrounds. On the other hand, its verification requires a careful matching between the expertise of reviewers and authors. Unfortunately, errors do occur and sometimes are only corrected many years after they appear in print, if at all. Nevertheless, at least when it comes to mathematical research, computer-verified formalized proofs are possible as a final arbiter of mathematical truth. Formalization of mathematics is the process of digitizing mathematical knowledge, which allows for formal proof verification of not only mathematical results, but computer programs as well. However, they are not easy to produce even for relatively simple statements. Hence, such approaches typically lack far behind the current research frontier. Aimed at addressing the above-mentioned issues, the three main contributions of this thesis are as follows:

1. To promote collaboration and expert reviewing within mathematical research, we propose a novel blockchain-based system that allows mathematicians to collaborate by sharing their partial pen-and-paper proofs (proofs written in natural language), and for reviewers to review thereafter.
2. To close the gap between the set of formalized and unformalized (i.e., pen-and-paper) mathematical knowledge, we propose a blockchain-based system that allows mathematicians, computer scientists and even automated artificial intelligence (AI) tools to collaborate by sharing their partial formalized proofs (proofs in program code).
3. To ensure the quality and correctness of each partial proof, we propose a proof of concept of incentive mechanisms that incentivize mathematicians to share their partial results immediately and expert reviewers to review.

Publications

Jin Xing Lim, Barnabé Monnot and Georgios Piliouras (2022). “Blockchain-based Mechanism Design for Collaborative Mathematical Research”. In: *2022 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, pp. 1-9

Jin Xing Lim, Barnabé Monnot, Shaowei Lin and Georgios Piliouras (2021). “A Blockchain-Based Approach for Collaborative Formalization of Mathematics and Programs”. In: *2021 IEEE International Conference on Blockchain (Blockchain)*. IEEE, pp. 321-326.

Jin Xing Lim, Barnabé Monnot, Georgios Piliouras and Shaowei Lin (2021). “(Auto)Complete this Proof: Decentralized Proof Generation via Smart Contracts”. In: *Online proceedings of the 6th Conference on Artificial Intelligence and Theorem Proving (AITP 2021)*. pp. 74–76.

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my main advisor, Georgios Piliouras, for inspiring me to pursue this research topic. His patient guidance along the way allows me to discover and achieve results in an area I never thought I could. He led by example and showed me what it means to be a successful researcher, and how not to settle for less in order to bring the best out of myself.

Second, I am truly grateful to my two co-advisors, Shaowei Lin and Barnabé Monnot. Prior to meeting Shaowei, I have always had deep appreciation towards pure mathematics, particularly in the area of mathematical logic, but thought that it has little to none practicality. He was the one who convinced me that pure mathematics can be applied in the real world through fields such as formal methods and automated programming. It was his enthusiastic encouragement and extraordinary points of view that persuaded me to pursue my Ph.D. at SUTD in the first place. While Shaowei was the one who started my Ph.D. journey, Barnabé was the one who helped me in finishing it. He equipped me with the necessary knowledge on blockchains by directing me to the right materials and answering all my queries. As a research scientist at Ethereum himself, he always keeps me up to date with the latest advances in the field of blockchain.

Third, I am thankful for the people I have met over the years at SUTD. Thank you Zhangsheng Lai, Rachel Koh, Trung Dung Vu, Tenzin Chan, Lin Geng Foo and Stefanos Leonardos for your valuable feedback on my presentations and papers. It is because of you allguys that my Ph.D. journey so enjoyable. Special thanks to Sin Chee Chen, and the staff at ESD, for making sure that my journey at SUTD was a smooth one. I would also like to thank the rest of my thesis committee members: Karthik Natarajan, Karthyek Murthy and Ernest Chong for organizing memorable classes, attending my qualifying exam, and providing feedback on my thesis. Finally, my Ph.D. would not be possible without the funding through SUTD Ph.D. Fellowship.

Last but not least, I would like to give my greatest appreciation to my wife-to-be Winnie Sim, my brother Jin Wang Lim and my loving parents for supporting and believing in me. I am especially thankful for my mother. Despite not having any formal education, she provides me with unconditional love and support to make sure that I achieve the highest level that I can reach. Thank you!

Contents

PhD Thesis Examination Committee	i
Abstract	ii
Publications	iii
Acknowledgements	iv
1 Introduction	1
1.1 Motivation	1
1.1.1 Decentralized conference for collaborative mathematics	2
1.1.2 Unified proof and program synthesizer	3
1.2 Related Works and Gaps	4
1.2.1 Decentralized science (DeSci) movement	4
1.2.2 Blockchain-based projects for formalized mathematics	5
1.2.3 Incentive mechanism design	6
1.2.4 Gaps in the related works	7
1.3 Thesis contribution and outline	9
2 Preliminaries	11
2.1 Mathematical proofs	11
2.1.1 Formalized proofs	11
Why formalized proofs?	13
2.1.2 Proof representation	15
Proofs as directed-acyclic graphs (DAGs)	15
Proofs as string diagrams	16
2.2 Blockchains	20
2.2.1 Blockchain basics	20
Consensus mechanisms	21
Execution model	24
Why blockchains?	26
2.2.2 Blockchain applications and advancement	27
Decentralized applications (dApps)	27
Token-Curated Registries (TCRs)	28
Non-fungible tokens (NFTs)	29
Zero-knowledge proofs (ZKPs)	30
Off-chain executions of transactions and smart contracts	32

3	Blockchain architecture	33
3.1	Data layer	33
3.2	Client layer	36
3.3	Incentive layer	39
3.3.1	Licensing sublayer (for formalized version)	40
3.3.2	NFT sublayer (for pen-and-paper version)	41
4	Incentive mechanism	43
4.1	Game theory concepts and definitions	43
4.1.1	Nash and subgame Nash equilibrium	43
4.1.2	Perfect Bayesian equilibrium (PBE)	48
4.2	Model of the incentive mechanism	54
4.2.1	Proof completion game model	55
4.2.2	Incentivized reviewing process	56
4.2.3	Dynamics and strategies of the model	58
4.3	Analysis of the incentive mechanism	59
4.3.1	Analysis on string diagram of single subproof	59
4.3.2	Analysis on string diagram of multiple sequential subproofs	61
4.3.3	Analysis on string diagram of multiple parallel subproofs	66
5	Illustrative example	74
6	Conclusion and discussion	86
6.1	Discussion on assumptions and limitations	87
6.2	Future directions	91
A	Formalization of divide-and-conquer algorithm in Coq	92
A.1	Well-founded induction	95
A.2	Different algorithm paradigms for lists	96
A.3	Sorting algorithms based on different inductive principles	100
	Bibliography	103

List of Figures

2.1	Directed acyclic graph (DAG) representation: solid edges - completed subproofs, dotted edges - incompleted subproofs. Goal - all paths from node S to T to be connected by just solid edges.	17
2.2	Examples of how lemmas of a conjecture C can be represented in sequential or parallel manner in a DAG depending on the initial subproof	18
2.3	String diagram representation: strings - theorem statements, solid nodes - completed subproofs, dotted nodes - incompleted subproofs. Goal - all paths from edge <code>sort_prog</code> to <code>True</code> to be connected by just solid nodes.	20
2.4	String diagram representations of partial proofs	21
2.5	String diagrams corresponding to the DAG representation shown in Figure 2.2	22
2.6	String diagram representing proof of a binary search specification, which requires the proof of a sorting program and a search through splitting program	23
2.7	Simplified diagram of a blockchain	23
2.8	Diagrams to show the difference between chains with and without fork	24
2.9	Simple illustration to show the differences between the two main blockchain execution models	25
2.10	The Ali Baba cave	31
3.1	Flowchart of how a prover's formalized proof contribution gets recorded and eventually rewarded in our blockchain-based system across the three layers of our system architecture. Images' sources: IPFS , TCR , right-to-use	34
3.2	Flowchart of how a prover's pen-and-paper proof contribution gets recorded and eventually rewarded across the three layers of our system architecture.	34
4.1	An extensive game in which player 1 moves both before and after player 2.	45
4.2	Payoff tree of Example 4.1.14	52
4.3	Reviewing model with $L = 2$. ($U_{Prover}, U_{Reviewer}$) represents the payoffs of Prover and Reviewer respectively at each outcome of the game.	60
4.4	String diagram G comprises of all subproofs $0, 1, \dots, m_0$ while G' comprises of only the parallel subproofs $1, 2, \dots, m_0$	67
5.1	Example of how the user interface of our system can look like	75
5.2	String diagram representing linkages between files (containing conjecture, partial proof or completed object). The 4 input edges of the node "ct00" are the 4 imported files of <code>ct00.v</code> . The output edge of node "ct00" represents the conjecture "sort_prog", which leads to a dotted node representing an empty proof yet to be completed.	78

5.3	Update in the string diagram shown in Fig. 5.2 after P_1 uploads the entry of <code>ct00.v</code>	79
5.4	Update in the string diagram shown in Fig. 5.3 after A uploads the entry of <code>ct02.v</code> . We use red nodes to represent entries from AI provers.	80
5.5	Update in the string diagram shown in Fig. 5.4 after P_2 uploads the entry of <code>ct03.v</code>	81
5.6	Collaboration of contributions from human and AI provers that yields a proof of insertion sort algorithm.	82
5.7	String diagram the incentive mechanism is implemented on in deciding the values of R^{ct01} , R^{base} and R^{IH}	83
5.8	New string diagram representation of the alternative proof presented in <code>ct05.v</code> . Note that <code>sort_prog_base</code> is solved as P_3 imported and used the result from <code>ct02.v</code>	84
5.9	Collaboration of contributions from human and AI provers that yields a proof of merge sort algorithm. Note that labelling of the edges are removed for convenience.	85
A.1	Summary of how well-found induction eventually leads to different sorting algorithms	100

List of Tables

4.1	Normal form of the game in Example 4.1.2. Row player = player 1. Column player = player 2.	46
-----	--	----

*To my mother who sacrifices everything for me to be who I am
today*

Chapter 1

Introduction

1.1 Motivation

Publication of scientific research, particularly in mathematics, requires a thorough, unambiguous assessment of its correctness and novelty. This process usually takes long periods of time and unfortunately can sometimes be unreliable, especially in specialized or recently developed fields where it is hard to identify reviewers that are both available and have sufficient expertise in the topic. Furthermore, research collaboration is itself, typically as well as unnecessarily, constrained by the social connections of the authors. There are little to no effective mechanism to enable large groups of mathematicians to work together dynamically without some strong coordinating center, while being guaranteed proper recognition and reward for their contributions. Mathematical research is thus an exceptional hard task, with relatively antiquated and ossified supporting mechanisms.

There is one way of ensuring fully reliable and correct mathematical results without the need of reviewers and that is through the use of formalized mathematics. Formalization of mathematics is the process of digitizing mathematical knowledge into machine code so that proofs can be formally and automatically checked and recorded within a computer system. This idea was initially proposed in the *QED manifesto* (Boyer, 1994) during the 1994 CADE conference and it is currently done through the means of theorem provers such as Isabelle (Wenzel, Lawrence C Paulson, and Nipkow, 2008), Coq (Bertot, 2008) and Lean (Moura et al., 2015). Other than verifying mathematical proofs, theorem provers can also be used to verify programs and protocols (Filliâtre, 2011; Sergey, Wilcox, and Tatlock, 2017). With the rise of artificial intelligence (AI), machine learning tools can be implemented to aid us in automating, and hence, formalizing proofs partially (Jiang, Papapanagiotou, and Fleuriot, 2018). However, as work in this area is still at its early stage, it requires a lot of man-hours and collaboration to formalize the ever-growing mathematical knowledge and programming world.

One such collaboration project using formalized mathematics is the verification of a difficult theorem in a cutting edge mathematical field called condensed mathematics (Hartnett, 2021). The project started with Peter Scholze, one of the originators of condensed mathematics, wrote a blog post, titled "Liquid Tension Experiment" (Scholze, 2020), explaining the technical aspects of a result he wished could be verified through the means of the theorem prover, Lean. Surprisingly, through the collaboration of mathematicians and computer scientists working on what they are good at, the proof of the result was formally verified in less than seven months, which was initially thought would take around two years to complete.

There is another notable effort in this direction but instead of machine code, it is done through the use of the traditional mathematical proofs written in natural language, which we call them *pen-and-paper proofs*. The *Polymath project* proposed by Gowers, 2009 aimed to expedite collaboration between mathematicians in solving popular open problems through posting partial ideas/progress. The key idea behind it was that this practice would speed up the process of completing a proof as having more participants increases the odds that one of them will discover a great insight at different stages of the proof. The first project, called the *Polymath1* project, was a success as a group of mathematicians managed to produce a new combinatorial proof for the density version of the Hales–Jewett theorem (Polymath, 2012) within a matter of seven weeks. Despite this promising start, there has been no further output from the project since their work in 2014 on prime numbers (Polymath, 2014b; Polymath, 2014a). Although there are several ground rules set up by Tim Gowers, mainly to discourage collaborators to work solely on their own, the difficulty of appropriately incentivizing participation seems to have stalled further progress and more widespread adoption despite its revolutionary potential (Cranshaw and Kittur, 2011; Friesike and Fecher, 2016).

These projects show the importance of having dynamic collaborations especially in difficult or new mathematical research areas, however, can be hindered by difficult obstacles such as incentivizing participation. Thus, this raises our driving challenge: *to design a system architecture that decentralizes this process and allows for effective collaboration between multiple intelligent agents in mathematical research*. To this end, the thesis provides roadmaps of two blockchain-based approaches to speed up and improve the quality and collaboration of mathematical research through pen-and-paper proofs and formalized proofs respectively (more details in Section 1.3). The two main goals we hope to achieve by introducing these approaches are to have a *decentralized conference for collaborative mathematics* (for the pen-and-paper proofs version) and a *unified program synthesizer* (for the formalized proofs version).

1.1.1 Decentralized conference for collaborative mathematics

Due to the struggles of scientists making a living during the early to mid-1800s, scientific conferences was organized to make science more accessible to the public (Gewin, 2019) since then. Improvements and changes, such as having appropriate code of conduct, going green with reduced paper use and having gender equality for participants (women was initially banned due to blatant sexism), have been implemented over years of conferences. During the pandemic outbreak of COVID-19 in 2019 (Wu, Chen, and Chan, 2020), most conferences have been cancelled, postponed (especially in the early stage of the outbreak) and eventually held virtually through video-calling platforms such as Zoom¹. However, the structure of every conference remains generally the same. Authors or groups of authors submit their works before certain deadline which are subjected to reviews by reviewers assigned by a centralized group of people called the program committee. After one to two rounds of reviews and rebuttals, the program committee eventually decides if a paper would be accepted in a conference. Although centralization provides benefits such as standardization of work, it may be

¹<https://zoom.us/>

prone to bias and as a result, assessment of each paper's correctness and novelty can sometimes be unreliable.

Our goal is to create a new generation of conferences making use of this fast-growing technology. With the blockchain-based approaches discussed in the rest of the thesis, there are several benefits over the pre-existing structure of current conferences. First, decentralization of blockchain allows anyone to submit and verify works. Reviews and acceptance of each work is controlled by a diversified group of people instead of a centralized committee. This can improve the assessment of each work if the right mechanism is in place. Second, every details of each author's work or reviewer's review can be recorded and their identities can be stayed as anonymous. These records are all time-stamped as well keeping track of the authors' contributions in chronological order as well. Anonymity and chronological records can reduce bias in evaluation of work and settle dispute of the order of who writes what first. Lastly, with the introduction of smart contracts (more details in Chapter 2), incentive mechanisms can be implemented through them to distribute rewards, if any, appropriately. The rewards can be correctly distributed due to the time-stamped records of authors' contributions, which is hard to track in our current conference structure.

The thesis focuses specifically on creating a decentralized conference for mathematical research as every proof of a result can be broken down and illustrated through connections of partial progresses (more details in Chapter 2). Through the incentive mechanisms implemented via smart contracts, we can incentivize mathematicians and/or computer scientists to not only submit completed work but also partial results. With the sharing of partial works and continuation of where others have left off, this allows collaboration, and hence, speeds up the progress of mathematical research. Examples of such successes can be seen in platforms such as [stackoverflow](https://stackoverflow.com/)² and [mathoverflow](https://mathoverflow.net/)³. Users are able to progress in your own work due to the help of contributors they do not know in person. We want to build a system where all contributors can get rewarded for their contributions either through some form of real currencies (e.g., USD) or virtual currencies (e.g., cryptocurrencies).

Just like how many technologies such as healthcare, logistics and others are currently being revolutionized through the use of blockchains (Al-Jaroodi and Mohamed, 2019), we aim to create a new form of conferences where submissions and evaluations of works can be done in a decentralized manner and every partial contribution and even review from reviewer can be rewarded.

1.1.2 Unified proof and program synthesizer

Program synthesis is the generation of computer programs automatically just from high-level specifications (Gulwani, 2010) and there are many approaches in achieving it. In recent years, progress in this area seems to deviate towards inductive specifications such as input-output examples (Shaw, Swartout, and C. C. Green, 1975; Summers, 1977) such as the works of SKETCH (Solar-Lezama and Bodik, 2008) and the FlashFill (Gulwani, Harris, and Singh, 2012). Another approach is through deductive synthesis which started since the late 1960s (C. Green, 1969; Manna and R. J. Waldinger, 1971). A

²<https://stackoverflow.com/>

³<https://mathoverflow.net/>

framework of how it could be done was presented in Manna and R. Waldinger, 1980 which stated that any specification of a program can be written as a theorem as the form $\forall a, P(a) \implies \exists z, R(a, z)$, where a represents the input, P is the property that a satisfies, z is the output and R represents the relation that holds for a and z (we will see example/s later). Through constructive logical reasoning, the proof generated is in fact a program satisfying the stated specification. As one can formalize theorem statements and proofs through theorem provers, other than producing mathematical proofs, we can use them to synthesize programs as well.

With the boom in AI, machine learning tools can be implemented to aid us in automating the proofs in these theorem provers. There has been continuous efforts to improve the state-of-the-art efficiency of automated proof search such as Sledgehammer (Meng, Quigley, and Lawrence C Paulson, 2006) and CoqHammer (Czajka and Kaliszyk, 2018), in hope that one can produce a tool that can automate as many proofs (and hence, programs) as possible. Being one of the most crucial and difficult problems in both computer science and AI (Pnueli and Rosner, 1989), we are still unable to produce a fully automated program synthesizer for all programs through this approach.

However, automated tools, such as TacticToe (Gauthier, Kaliszyk, and Urban, 2018) and Tactician (Blaauwbroek, Urban, and Geuvers, 2020), show that there is a drift towards implementing incremental proof construction (Lawrence Charles Paulson, 2020), i.e., suggesting the next best proof step instead of automating the whole proof of a given theorem statement. These tools could help human provers by suggesting the "next best move" in their proofs, facilitating collaboration between human and machines. The goal of our approach is to have blockchain as the backbone to record partial progresses and serve as an "encyclopedia" for proofs and programs, which consists of contributions from both humans and machines. In the future, once we have abundance of automated tools for theorem provers with a certain efficiency level, different tools can make their own partial progresses and come together through this blockchain environment to generate proofs and programs, with little to no human intervention, as an unified proof and program synthesizer.

Before we provide an detailed overall summary of the thesis contribution and outline in Section 1.3, we will provide some related works and the gaps in them that we are addressing in this work.

1.2 Related Works and Gaps

The related works can be split into three main categories - decentralized science movement, blockchain-based projects for formalized mathematics and incentive mechanism design for participation and collaboration. Before this section ends, we describe the gaps in these related works and how we are going to address them.

1.2.1 Decentralized science (DeSci) movement

Decentralized science movement, in short DeSci, is one of the latest Web3 movement where scientists and researchers are making use of blockchain tools, such as smart contracts and tokens, to decentralize the process of generating scientific research work

(Hamburg, 2022). The movement aims to (a) improve scientific funding, (b) allow easy access and sharing of knowledge and (c) shift ownership and value away from profit-hungry intermediaries.

First, blockchain-based funding models, such as Quadratic Funding (Weiss, 2021) and retroactive funding (Optimism and Buterin, 2021), can set up protocols to allow users to seek for grants and fund interesting projects. Such diversification of funding allows different projects to get adequate funding. As a result, this incentivizes scientists to work on those they are interested in, instead of working on projects that are heavily funded by industry intermediaries.

Second, decentralization of blockchain allows scientists to publish their work without a middleman. This allows knowledge to be shared without censorship and also promotes collaboration. Ants Review (Trovò and Massari, 2021) showed that smart contracts can be used to mediate and incentivize between authors and reviewers to share and critique works respectively. This will not only improve the quantity of research being done, but also its quality through decentralized reviewing.

Last, blockchain technology such as non-fungible tokens (NFTs, more on this in Chapter 2) allows authors and funders to own and be associated to their research work directly. This reduces the need of profit-hungry intermediaries, which may influence the direction of research.

1.2.2 Blockchain-based projects for formalized mathematics

We briefly discuss some related works that try to tie blockchains and mathematical proofs together. The paragraphs are arranged in chronological order of their publication dates.

Qeditas (White, 2016) Qeditas was proposed to be a Bitcoin spin-off (Peter, 2014), which serves a lightweight ledger to record publication of formal mathematical definitions and theorems with proofs, ownership rights of these mathematical objects, transactions of these property rights and bounties on unproven conjectures. Any user can participate by either creating definitions, proving new theorems or forming conjectures with bounties for others to prove. Any of these mathematical objects (definitions, theorems with proofs and conjectures) must be formally verified to be well-formed by some preexisting proof checker such as Egal (Brown, 2014) before it can be recorded in Qeditas. Every mathematical object will be issued with an address and will be tied to its author's address as well. The author can choose to either restrict the usage of her mathematical objects or allow others to use them with some fee. This allows users to import and build on existing mathematical objects in the blockchain. Users can also form a conjecture and put a bounty on it. As the benefit of using blockchain is its ability to timestamp every entry, the bounty will go to whoever that prove the conjecture first. Their aim is to encourage and reward people for spending time formalizing mathematics.

MathCoin (Su, 2018) MathCoin was proposed as a blockchain to record formal mathematical objects and have an incentive system on conjectures based on some betting system, instead of bounties. The paper's author did not specify which proof checker

should be used and believes that before a MathCoin blockchain is rolled out, there should be a set of preexisting math-domain addresses, called native addresses. These native addresses include predefined axioms, definitions and simple theorems that users can build on. However, the paper did not mention any incentive system for users to create new definitions or theorems with proofs like the purchasing of objects' rights mentioned in Qeditas. This may discourage users to even participate in this ecosystem as every entry, regardless if it is new definition or theorem, will involve a fee to post and an optional fee to the miner. Any reward will only come from solving conjectures. Users will need to buy some "betting" tokens to bet if a conjecture or its negation will be eventually proven. The price of these tokens will be determined by some function with properties stated in Su, 2018. Once the conjecture is proven, the users who bet on its negation will lose their bets and users who bet on the conjecture itself will get rewarded from those losing bets and public funds, which are accumulated from some ratio of every new currency created by the miners. The paper's author mentioned that as more and more definitions and theorems are created, it will be difficult to store all documentation on chain. Thus, venturing out to use decentralized storage systems such as IPFS (Benet, 2014), as mentioned in Chapter 3, should be considered.

Proofgold (Brown, 2020) The ideas presented in Qeditas are similar to what was implemented in Proofgold⁴ - a decentralized library to contain formal mathematical definitions, theorems and conjectures (with bounties). However, they created their own proof checker, that ensures all uploaded mathematical objects are well-formed, which is based on intuitionistic higher-order logic (Brown, 2020). As compare to using existing theorem or proof checker, users, who would like to participate to creating mathematical objects in this blockchain, need to learn the syntax and rules implemented by them from scratch.

1.2.3 Incentive mechanism design

We discuss the two main papers that the incentive mechanism in the thesis drew ideas from.

PPS (Banerjee, Goel, and Kollagunta Krishnaswamy, 2014) The paper discusses the incentive mechanisms to ensure partial-progress sharing (PPS) in collaborative research platforms. It states that every problem can be broken up into subtasks and the dependencies between them are illustrated through a subtask-network - a directed acyclic graph. Each subtask comes with a reward which is fixed exogenously and will be given to the first agent who publicly shares its solution. It is assumed that every agent work independently and solely to complete all subtasks of a given problem. Suppose there are multiple subtasks available for solving, each agent can choose which subtask to work on and when to reveal the solutions that he/she has completed. Through a game theoretical approach, the paper provides conditions for how the rewards of the subtasks should be fixed in order to ensure PPS to be incentive-compatible for every agent. The paper also provides analysis on the efficiency of the mechanisms as compared to the optimal makespan (i.e., the least amount of time to complete a problem).

⁴<https://github.com/ai4reason/Proofgold>

SPRIG (Carré et al., 2021) Smart Proofs via Recursive Information Gathering (SPRIG) is a protocol which allows two types of agents, *Claimer* and *Skeptic*, to debate over the correctness of mathematical proofs in a decentralized manner. Unlike the other related works mentioned previously, SPRIG is not aimed to create an massive database of all mathematical definitions and theorems. Instead, the protocol aims to incentivize people to debate over relevant and important mathematical results in the community. Furthermore, the authors believe that mathematical proofs should stay at high-level and informal instead of making all details formal for machine to check (unless necessary). In the protocol, Claimer (pronoun: she) provides a high-level proof sketch of some proposition she claims to prove. Skeptic (pronoun: he) is to read the proof and ask questions to her if there is any parts unclear. He can also invalidate her proof if he can find a mistake in the machine-level of her proof details. Claimer and Skeptic have limited time to respond to the questions and ask questions respectively. For any part of the proof, after several rounds of debate (questioning from Skeptic and answering from Claimer), say 9 rounds, she must provide machine-level proof details in order for the part to be valid. To incentivize this kind of debate session to happen and spamming of questions from Skeptic for all posted proofs, Claimer must put a stake with the claim and Skeptic need to put a bounty on each question he asks. If she cannot answer his question within the time limit, he will get her stake. On the other hand, if Claimer or any other user can answer his question, she will get the question's bounty. All documentation of the debate sessions will be recorded on a blockchain so that it can happen in a decentralized manner and timestamp of every entry will ensure that the transactions are validated.

1.2.4 Gaps in the related works

There are several gaps that these related work do not address adequately and which we aim to improve upon.

Mathematical proofs oriented We are very much in line with the goals that DeSci is trying to achieve, which are to decentralize funding, knowledge and ownership. However, the thesis focuses mainly on mathematical proofs (both pen-and-paper and formalized). Mathematical proofs are easier to be dissected into smaller tasks, in terms of lemmas, than other sciences and connections between these tasks can be linked together through diagram (such as directed acyclic graph and string diagram as described in Chapter 2). This can provide distinction in scientist-to-task allocation (i.e., which scientist worked and owned which part of the overall work), and hence, the ease of collaboration. The thesis specifically describes (a) system architectures, (b) how collaboration can be done to solve a common mathematical problem and (c) how such collaboration can be rewarded through incentive mechanism, which are inadequately addressed in the DeSci movement.

Filtering submissions Documentation of formalized mathematical objects, i.e., definitions, theorem statements and proofs, are organized in terms of libraries, such as the Mathematical Components library of Coq `ssreflect` (Gonthier and Mahboubi, 2010) and the Lean mathematical library, `mathlib` (Community, 2020). To be documented in

any of these libraries, experts from the respective community will determine whether each submitted document fulfills certain criteria. Yet, in a decentralized system, there will be no “expert” to filter out submissions. Qeditas suggested strategies such as inclusion of “salted” documents and having fees proportionate to the size of the submitted documents to deter duplicate submissions. Having said that, to filter out novel, non-duplicate and relevant work from the rest, we believe a much stronger mechanism needs to be in place so that the decentralized library will be streamlined. Our approach featuring a Token-curated registry (TCR) (Goldin, 2017; Asgaonkar and Krishnamachari, 2018) allowing users to accept or reject a given submission would be particularly useful, but has not been discussed in any of the prior works.

Collaborative work Although there are several useful ideas suggested by these prior projects to incentivize users to formalize proofs, such as rewarding ownership as intellectual property, bounties and token betting, there has been little to no mention of the use of smart contracts to implement complex incentive mechanisms to split rewards appropriately to collaborators. With the right incentive mechanism in place and the ability to leave proofs with gaps as mentioned in Section 2.1.1, we aim to reward contributors for completed as well as partial results. Contributors need not worry about the authorship of their partial work due to the time-stamped verification provided by blockchain, serving as a motivation for them to submit partial, incomplete results and formalize proofs collaboratively.

Automated AI tools’ participation None of prior projects mention about the involvement of automated AI tools as mentioned previously. However, with the continuous efforts to improve the state-of-the-art efficiency of automated proof search, we see a possibility that these automated tools could take part by looking for open conjectures and automating the proofs. Some of these automated tools require high computing power to produce better automated proof searches. Developers could deploy these tools as nodes in the blockchain environment to look for conjectures to solve instead of requiring users to have the necessary computing capabilities in order to use them.

System for pen-and-paper proofs All of the related works in Section 1.2.2 are projects that want to use blockchain to record and tie contributions of formalized mathematics together. However, as writing mathematical proofs in terms of code using theorem provers is still a relatively new concept to many mathematicians, most would still prefer to write proofs through the traditional pen-and-paper proofs. Knowing this limitation, other than proposing a formalized approach like what the other three projects in Section 1.2.2 did, we also proposed one that accepts pen-and-paper proofs as well. As a result, it would allow a larger audience of mathematicians to participate once the pen-and-paper version is implemented.

Mechanism for rewarding both collaboration and reviewing The SPRIG protocol by Carré et al., 2021 provides a mechanism for provers and reviewers to debate over the correctness of a mathematical proof but does not take into account of collaborative setting like what Banerjee, Goel, and Kollagunta Krishnaswamy, 2014 does. On the other hand, the PPS mechanisms by Banerjee, Goel, and Kollagunta Krishnaswamy,

2014 provides conditions to allocate rewards appropriately in collaborative setting but assumes that reward is given to whoever that submitted a proof first. This assumption may be impractical as this is assuming that the first proof submitted is correct by default. However, we know that every proof needs to be verified either formally through theorem provers or manually by reviewers. The PPS mechanisms do not take into considerations of these verifications like what Carré et al., 2021 does. The incentive mechanisms discussed in this thesis cover the shortcomings of these two papers where they not only consider the debate process between the provers and reviewers, but also show how rewards (and other parameters such as stakes and bounties which we discusses in Chapter 4) can be allocated appropriately to contributions from different provers. We shall see that the mechanisms in both papers are just instances of our mechanisms if certain cases are considered.

1.3 Thesis contribution and outline

The three main contributions of this thesis are as follows:

1. To promote collaboration and expert reviewing within mathematical research, we propose a novel blockchain-based system that allows mathematicians to collaborate by sharing their partial pen-and-paper proofs, and for reviewers to review thereafter.
2. To close the gap between the set of formalized and unformalized (i.e., pen-and-paper) mathematical knowledge, we propose a blockchain-based system that allows mathematicians, computer scientists and even automated artificial intelligence (AI) tools to collaborate by sharing their partial formalized proofs.
3. To ensure the quality and correctness of each partial proof, we propose a proof of concept of incentive mechanisms that incentivize mathematicians to share their partial results immediately and expert reviewers to review.

The outline for the rest of the thesis is as follows:

- Chapter 2 We present some preliminary knowledge of mathematical proofs, where we will expand on formalized proofs and proof representations, and blockchain with its necessary applications that we are using for our approaches.
- Chapter 3 We describe the architectures of our two proposed blockchain-based approaches, where we will first provide details for the formalized proof version before addressing the similarities and differences in the pen-and-paper proof version.
- Chapter 4 Together with some background knowledge of the game theory definitions and concepts required, we provide proof of concept of mechanisms to incentivize provers to share partial proofs immediately and reviewers to review.
- Chapter 5 We illustrate how our formalized proof approach could work through an example to see how provers, including automated AI tools, can collaborate to generate proofs of a sorting program.

Chapter 6 We conclude the thesis with some discussion on the assumptions, limitations and future directions of this work.

Appendix A We explain the formalization of the divide-and-conquer algorithm paradigm and the motivating example, sorting program, through the use of Coq the proof assistant in details.

Chapter 2

Preliminaries

We will provide some preliminaries on mathematical proofs and blockchains in this chapter. For the section on mathematical proofs, we will first take a look at what we mean by formalized proof and the current tools available for us to write and/or automate such proofs. Moreover, we will go through how proofs can be represented by different forms of diagrams as it is necessary in understanding the incentive mechanism described in Chapter 4. After which, the following section will go through the basics of blockchains and their applications, such as decentralized applications (dApps), non-fungible tokens (NFTs) and zero-knowledge proofs (ZKPs), which are essential in our architecture (Chapter 3).

2.1 Mathematical proofs

Traditionally, a mathematical proof is an argumentative piece of work to prove or disprove a stated claim, called *proposition* or *theorem*. This argumentative piece of work usually comes in the form of handwritten or typed text, which can span from a few lines to hundreds of pages long. We shall call this kind of work a *pen-and-paper proof* from now on. However, not all pen-and-paper proofs are valid. For example, to prove the theorem that the addition of even numbers will always result in even number, providing an instance of such fact, e.g., $2 + 2 = 4$, cannot be considered a valid as the argument is not exhaustive. There are rules for proof construction, called *rules of formation*, and predetermined truth statements, called *axioms*, which serve as a premise for inferring future theorems. The abstract structure comprising of these rules and axioms for deducing theorems is called a *formal system*. Depending on the rules of formation and the axioms used, there are many different kinds of formal systems and the most well-known system on which most of the modern mathematics is built is *Zermelo–Fraenkel set theory with the axiom of choice* (ZFC) (Kunen, 2007). However, due to Gödel’s incompleteness theorems (Raatikainen, 2013), there exist theorems that are *unprovable* in any given formal systems, i.e., neither the theorem nor its negation can be proven. We will only focus on theorems where each of them is *provable*, i.e., there exists a proof for the statement, or its negation can be proven for the rest of the thesis.

2.1.1 Formalized proofs

Just as we digitize text, music and many other things to make them more convenient and accessible, there have been many efforts to digitize proofs as machine-readable

codes (Harrison, Urban, and Wiedijk, 2014) since the birth of general-purpose computers in 1945. We shall refer to such proofs as *formalized proofs* from now on. These proofs are written in computer programs called *theorem provers*. There are two types of theorem provers, *automated theorem provers (ATPs)*, such as E (Schulz, 2002), Vampire (Riazanov and Voronkov, 2002) and Z3 (De Moura and Bjørner, 2008), and *interactive theorem provers (ITPs)*, such as Isabelle (Wenzel, Lawrence C Paulson, and Nipkow, 2008), Coq (Bertot, 2008) and Lean (Moura et al., 2015). As their name suggests, ATPs perform automated reasoning to determine if a given theorem is provable. However, there are limitations to their expressivity as the formal systems on which these ATPs are built can represent only statements in first-order logic, i.e., statements that use quantifiers over individuals but not predicates or functions. On the other hand, ITPs, which build on formal systems such as lambda calculus (Church, 1932) and intuitionistic type theory (Martin-Löf and Sambin, 1984), allow users to express and use theorem statements in higher-order logic, i.e., first-order logic plus quantifiers over predicates and functions. The more expressive a programming language is, the harder the automation will be. Thus, ITPs need help from users to guide and assist the proof engine in deriving the proofs interactively through the use of *tactics*. The goal of a tactic is to break down complicated theorem statements into multiple simpler statements that are hopefully easier to complete. This is why ITPs are also known as *proof assistants*. Listing 2.1 shows how a formalized proof of a simple theorem, *sum_to_n*, i.e., for all $n \in \mathbb{N}$, $2(\sum_{i=1}^n i) = n(n+1)$, looks like in the Coq proof assistant.

```

1 Require Import Arith.
2
3 Fixpoint sum_to (n:nat): nat :=
4   match n with
5   | 0 => 0
6   | S k => n + sum_to k
7   end.
8
9 Theorem sum_first_n: forall (n: nat), 2 * sum_to n = n * (n + 1).
10 Proof.
11 induction n.
12 - (* Base case *) trivial.
13 - (* Inductive case *) unfold sum_to; rewrite mult_plus_distr_l;
14   fold sum_to; rewrite IHn; ring.
15 Qed.

```

LISTING 2.1: Formalized proof of *sum_to_n* in Coq

The proof begins with importing the formerly defined definitions and theorems from the module `Arith` (line 1), followed by defining a recursive function `sum_to` to sum natural numbers up to some fixed n (line 3 to 7). The specification of the theorem is then stated (line 9), followed by its proof (line 10 to 15). The proof consists of using the tactic `induction n`, which does the same thing as our usual mathematical induction, applying formerly defined theorem, `mult_plus_distr_l`, assumption, `IHn`, and pre-defined automated proof searches, `trivial` and `ring`, for simplification and completion. As one can see, writing a proof in such system could be an uphill task for an inexperienced user as compare to a pen-and-paper proof, which can be freely expressed in natural language within the boundaries of axioms and assumptions. However, there

is an increase in popularity in using theorem provers recently not for only formalizing old important results, e.g., the formal proof of four-colour theorem (Gonthier et al., 2008), but also for cutting-edge mathematics, e.g., liquid tensor experiment (Scholze, 2021) as discussed in Section 1.1. We will discuss several reasons for the increase in emphasis on formalized proofs.

Why formalized proofs?

Machine verification Verification of a proof requires time especially if it extends to hundreds or even thousands pages. Moreover, there have been occurrences where the proof is found to be erroneous after some time (Avigad and Harrison, 2014), e.g., it took Andrew Wiles and his former student Richard Taylor almost a year to fix Wiles’ first proof of the Fermat’s Last Theorem (Wiles, 1995; Taylor and Wiles, 1995). By formalizing proofs, the computer can automatically verify and guarantee correctness according to the rules admissible within the formal system used by the theorem prover.

Proofs with gaps In ITPs, we can write incomplete proofs with gaps, where parts of the proofs can be left empty for future completion by either the owner or someone else. To illustrate this, we shall use the proof seen in Listing 2.1 as an example. Instead of completing the whole proof, the author of proof could simply initiate the proof with `induction n`, and leave the proofs of both base and inductive cases empty as conjectures for future completion shown in Listing 2.2. We will define such a proof as a *partial proof*, i.e., an entire proof of a theorem which is formally correct modulo specific unproven lemmas/conjectures. We will define *subproof*, as the name suggests, to be proof of a sub-part, which can be a lemma or conjecture, of a larger proof of a theorem. A proof, subproof or partial proof, is considered to be *incomplete* if there exists unproven conjectures within itself. Otherwise, it is considered to be *complete*. For example, line 9 to 15 of Listing 2.1 is a complete proof, line 14 to 18 is an incomplete partial proof in Coq, line 9 and 11 are incomplete subproofs of `sum_to_n`, and `induction n` is a complete subproof of `sum_to_n`. Note that each partial proof or subproof can be arbitrarily long, so it can range from 1 line all the way to few thousands lines long. We will also extend the definition of partial proof and subproof for pen-and-paper proofs as well. The benefit of having partial proofs is that it allows room for collaboration. This is especially clear in the case of formalized proofs. When an author writes a partial proof of a theorem and gets stuck, he/she can write all the unproven lemmas as clearly specified conjectures, like the ones in line 9 and 11 of Listing 2.2. Future contributors who would like to continue his/her proof need not read his entire partial proof as the initial subproof contribution is already formally verified. They instead can go to the well-specified conjectures and proceed from there straightaway. However, as for the case of pen-and-paper proofs, one still needs to take time to verify manually that the initial subproof contribution is valid and find out at which parts of the proof the original author get stuck at, unless they are clearly specified. In Section 4, we will discuss an incentive mechanism can address this issue by allowing reviewers to validate subproofs in a decentralized manner.

```

1 Require Import Arith.
2
3 Fixpoint sum_to (n:nat) : nat :=

```

```

4  match n with
5  | 0 => 0
6  | S k => n + sum_to k
7  end.
8
9  Conjecture sum_to_n_BC: 2 * sum_to 0 = 0 * (0 + 1).
10
11 Conjecture sum_to_n_IC: forall (n : nat), 2 * sum_to (S n) = S n * (S n + 1).
12
13 Theorem sum_to_n: forall (n : nat), 2 * sum_to n = n * (n + 1).
14 Proof.
15 induction n. apply sum_to_n_BC. apply sum_to_n_IC.
16 Qed.

```

LISTING 2.2: Partial proof of `sum_first_n` in Coq

Automated AI tools Although it was previously mentioned that the main purpose of ITPs is to have users interact with machines to search and develop proofs, there has been an increasing number of ITP extensions for automated proof searches, such as Sledgehammer (Meng, Quigley, and Lawrence C Paulson, 2006) for Isabelle, CoqHammer (Czajka and Kaliszyk, 2018) and Tactician (Blaauwbroek, Urban, and Geuvers, 2020) for Coq, and Proof Artifact Co-Training (PACT) for Lean (Han et al., 2021). These tools make use of machine learning methods such as k -NNs, RNNs and neural networks to help with the proof searches. Just as with ATPs but now in higher-order logic, the goal of most of these tools is to have an automated proof search function that generates a proof automatically given some theorem statement, and hence, ease the mathematicians' load in formalizing proofs. The effectiveness, however, of these automated tools is still not as desirable as one would hope. Paulson and Blanchette showed in their experiments that Sledgehammer only managed to automate 52% of the proofs from seven selected Isabelle theories (L. Paulson and Blanchette, 2015). On the brighter side, combinations of different automated tools yield better results than the tools individually. In the experiments run by the developers of Proverbot9001 (Sanchez-Stern et al., 2020), the combination of CoqHammer and their tool is 32% more effective in automated proving than the tool itself. Furthermore, they concluded that CoqGym (Yang and Deng, 2019) and their approaches are complementary as each approach can complete proofs which the other cannot. The goal of the automated tool, Tactician, is to suggest the next best tactic to be applied so to simplify the proof instead of automating the whole proof search with just one tactic or function. This fits well with ITPs' ability to allow proofs with gaps. If there are multiple complementary automated tools that are available, an author can begin with an initial subproof and let these automated tools suggest and simplify the theorem statements into simpler unsolved lemmas for the original author or other users to tackle. As a result, this facilitates collaboration not just between humans but also between automated AI tools. As the state of the AI is constantly improving, one can hope that in the future, he/she can simply state some theorem statements and let different automated AI tools collaborate and solve the problem, without any aid from humans, in some decentralized system. This is precisely one of the main goals of our work, which is to create a blockchain-based system that allows

both humans and automated AI tools to participate and solve problems collaboratively as an unified proof and program synthesizer.

Formal verification of programs and protocols Other than writing proofs for mathematical theorems, theorem provers have also been widely used in many other domains such as systems biology (Rashid et al., 2017), economics (Kerber, Lange, and Rowat, 2016) and self-adaptive systems (Weyns et al., 2012). Out of all of the domains, the two most important domains are arguably computer science and blockchain. In area of computer science, erroneous code can cause serious repercussions to human lives or great financial losses (Nawaz et al., 2019). CompCert (Leroy et al., 2016) is one project that verifies practical compilers to be used for critical embedded software. Furthermore, due to the Curry-Howard proofs-as-programs correspondence (Howard, 1980), ITPs, such as Coq, have extraction tools (Filliâtre and Letouzey, 2004) that allow us to generate usable programs by simply extracting from the formalized proofs. For example, if we want to generate a sorting program for any input list of natural numbers, we can first state its specification as a proposition (Listing 2.3), prove it, and then extract from the proof a working verified algorithm in OCaml language. In the area of blockchain, due to the huge flow of transactions of cryptocurrencies and data, any one attack can be devastating such as the notorious attacks on the DAO (Mehar et al., 2019) and Parity Multisig Wallet (Breidenbach et al., 2017). As a result, there are many companies, such as Certik¹ and Runtime Verification Inc², that work on improving the security of smart contracts (Tolmach et al., 2021) and blockchain protocols (Li et al., 2020).

```
1 Theorem sort_prog:
2   forall (l: list nat), {l': list nat | sorted l' ^ permutation l' l}.
```

LISTING 2.3: Formal specification of a sorting program in Coq

2.1.2 Proof representation

Having the right visualization will help us in understanding the overall structure of a proof, and hence, allowing us to use it for creating an incentive mechanism in Chapter 4. We will first look at how proofs can be visualized as directed acyclic graphs (DAGs), followed by as string diagrams.

Proofs as directed-acyclic graphs (DAGs)

Inspired by Banerjee, Goel, and Kollagunta Krishnaswamy, 2014, a proof of a mathematical statement can be illustrated as a *directed acyclic graph* (DAG), with edges representing subproofs. Figure 2.1a shows a DAG representing a partial proof of a theorem, where the subproofs 0 to m_0 are arranged in a sequential manner as they are meant to be solved one after another. Solid edges are used to denote subproofs that are solved and dotted edges for subproofs that are yet to be solved. Each dotted edge will turn solid if a valid proof is presented for that edge. The goal is to have all paths from node

¹<https://www.certik.com/>

²<https://runtimeverification.com/>

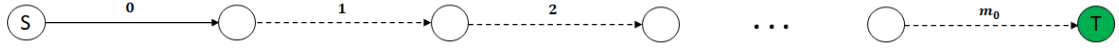
S connected to node T by only solid edges. Note that for this case where subproofs are arranged in sequential manner, subproof $u \in \{0, 1, 2, \dots, m_0\}$ will only be available for solving if all prior subproofs v where $v < u$ are solved. On the other hand, Figure 2.1b shows a DAG representation of a partial proof of a theorem that consists of the set of $\{0\} \cup [m_0]$ subproofs, where subproof 0 is an initial subproof provided and the set $[m_0] := \{1, 2, \dots, m_0\}$ consists the m_0 subproofs/lemmas left unsolved as conjectures required by subproof 0, arranged in a parallel manner as they are independent of each other and can be solved concurrently. One can view the subproof 0 as the proof skeleton of the whole proof, which requires the assumption and completion of unsolved lemmas 1, 2, ..., m_0 .

It is important to note that for any partial proof presented, it can only be of the forms that Figure 2.1a or 2.1b illustrated. This is because at the point where a prover gets stuck while writing a proof, all of his/her work prior to this point is represented by subproof 0 and the remaining lemmas required is either in sequential order or can be done in parallel manner. For example, suppose we have an open conjecture C available for solving. On one hand, if a prover proposes an initial subproof stating $A \implies B \implies C$ (i.e., $A \implies B$ and $B \implies C$) for some lemma A, B and he can only prove A , then the two lemmas required to be solved in order to complete the proof are $A \implies B$ followed by $B \implies C$. Thus, the lemmas are arranged sequentially following the initial subproof as seen in Figure 2.2a. On the other hand, if the prover provides an initial subproof stating $(A \text{ and } B) \implies C$ assuming A and B are true instead, then both lemmas A and B can to be solved independently in order for C to hold. In this case, the lemmas are arranged in parallel following the initial subproof as shown in Figure 2.2b. Further, combination of Figure 2.1a and 2.1b allows us to represent any structure of a whole proof using a DAG with some subgraphs arranged in sequential manner and some in parallel manner.

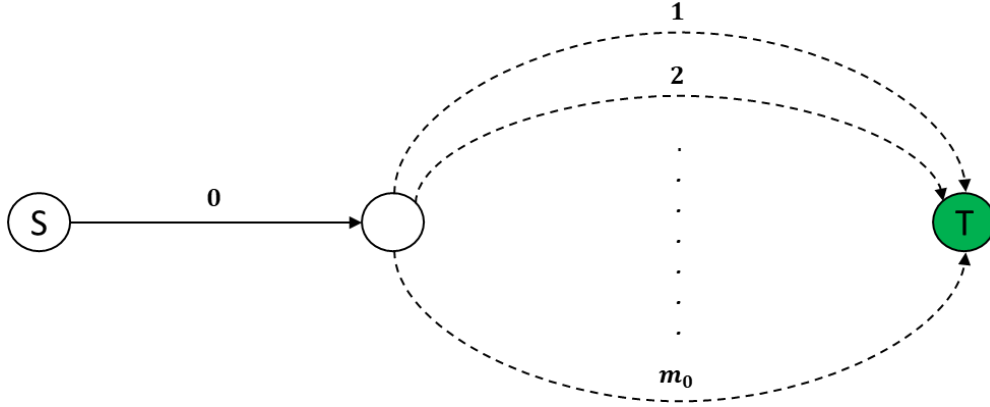
While DAGs have been the predominant method to represent proof structure, its use may be limited due to the rigid nature of the representation. The full structure of the representation can only emerge if all elements of a proof are known. To facilitate the complete representation, flexibility can be incorporated by allowing subproofs along the way as illustrated in Figure 2.1c. For example, some prover presents an initial subproof 0 with the assumption of unsolved subproofs/lemmas $[m_0]$. In Figure 2.1b, the DAG of his partial proof may keep changing until the whole proof has been completed and the DAG representation of a completed proof will look complicated. In contrast, for Figure 2.1c, suppose another prover can only provide an initial subproof (subproof 1.0 in Figure 2.1c) for subproof/lemma 1 instead of its whole proof. This means edge 1 in Figure 2.1b will be deleted and replaced with another DAG (Figure 2.1c). As the process continues and eventually gets completed, the DAG may potentially get massive and complicated. The problem can be compounded if this conjecture turns out to be part of a larger proof. Thus, analysis of the allocation rule of any incentive mechanism would be tedious while using this model representation. In the following, we address these limitations through the use of string diagram representation.

Proofs as string diagrams

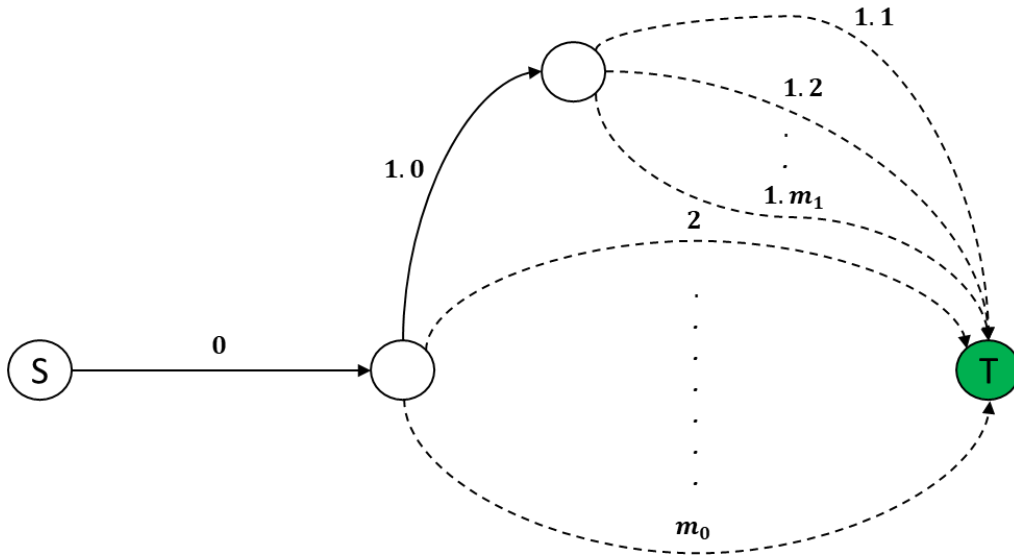
String diagrams, first formally introduced by Hotz, 1965, serve as a tool for studying the relationship between objects or resources, which can be physical or conceptual. Strings



(A) DAG representing a partial proof with subproofs arranged in sequential manner



(B) DAG representing a partial proof with subproof arranged in parallel manner



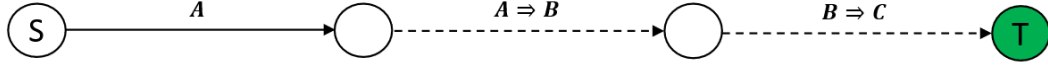
(C) DAG representing the case when a prover presents an initial subproof 1.0 for lemma 1

FIGURE 2.1: Directed acyclic graph (DAG) representation: solid edges - completed subproofs, dotted edges - incompleted subproofs. Goal - all paths from node S to T to be connected by just solid edges.

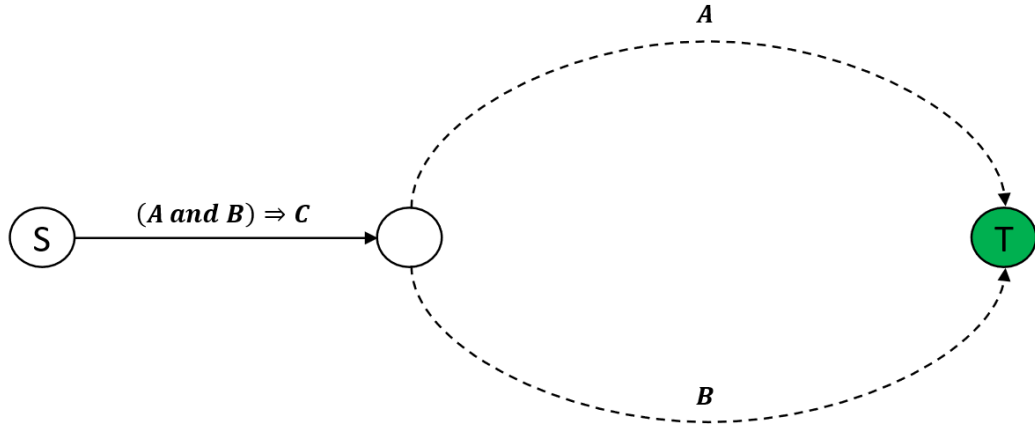
or wires (denoted by lines/edges) are used to represent objects/resources and nodes to denote relationships between objects. There have been numerous work that uses string

$$A \Rightarrow B \Rightarrow C$$

(i.e., $(A \Rightarrow B)$ and $(B \Rightarrow C)$)



(A) Example of DAG representing a partial proof with subproofs arranged in sequential manner



(B) Example of DAG representing a partial proof with subproof arranged in parallel manner

FIGURE 2.2: Examples of how lemmas of a conjecture C can be represented in sequential or parallel manner in a DAG depending on the initial subproof

diagrams to aid understanding, and hence, making progresses in many fields, such as linear algebra (Penrose, 1971b), applied category theory (Fong and Spivak, 2018; Paterson, Spivak, and Vagner, 2021) and quantum theory (Penrose, 1971a; Duncan et al., 2020). The style of constructing a string diagram is dependent on what objects and relationships represent in the framework. In our framework, the objects we are working with are theorem statements/types and relationships are the logical connections between theorems through (sub)proofs. As such, we use strings to represent theorem statements and nodes to represent subproofs. There is no direction drawn for strings as the direction can be viewed either as all going from left to right or the other way round. If the direction is viewed as going from left to right, each node has an input wire (entering from left side of the node) representing the theorem prior to the subproof and output wires (exiting from right side of the node) representing the lemmas required for the subproof. On the other hand, if the direction is viewed as going from right to left, the output wire (formerly viewed as input wire) of a node represents the consequence of the subproof and the input wires (formerly viewed as output wires) represent the assumptions required by the subproof. In our paper, we shall view the string diagram with the left-to-right direction.

As string diagram representations are less common compared to DAG representations, we shall use the (partial) proof of the proposition `sort_prog` to illustrate a simple case before generalizing it. We use dotted nodes to represent incomplete subproofs and solid nodes for completed subproofs. At the beginning, when the proposition is stated without any proof (Listing 2.3), the (empty) proof is represented by a dotted node with input string as the proposition `sort_prog` and output string as the trivial `True` proposition (Figure 2.3a). Once the partial proof (Listing 2.4) is presented, the dotted node will be filled with a (inner) string diagram of the completed and incompleted subproofs as shown in Figure 2.3b. The goal is to have all the paths from the targeted theorem, in this case `sort_prog`, to the `True` proposition, connected with just solid nodes. Each dotted node will eventually turn to solid node by either providing the complete proof of it or if all of the (inner) dotted nodes within the (outer) node are proven, i.e., change to solid nodes. For example, in Figure 2.3b, in order for the dotted node of “proof of `sort_prog`” to be completed and turned into solid node, both the dotted nodes within, i.e., “proof of `sort_prog_base`” and “proof of `sort_prog_IH`”, must be completed and turned into solid nodes first. Similarly, the proofs of the required lemmas are arranged in a parallel manner as both of them need to be proven correct, which can be done concurrently, in order for the whole proof to be valid and they will only be available to solve after the initial subproof, `induction l`, is revealed.

```

1 Conjecture sort_prog_base: {l': list nat | sorted l' ∧ permutation l' []}.
2
3 Conjecture sort_prog_IH: forall (a: nat) (l x: list nat),
4   sorted x → permutation x l
5   → {l': list nat | sorted l' ∧ permutation l' (a :: l)}.
6
7 Theorem sort_prog:
8   forall (l: list nat), {l': list nat | sorted l' ∧ permutation l' l}.
9 Proof.
10 induction l.
11 - apply sort_prog_base.
12 - destruct IHl; destruct a0; eapply sort_prog_IH; eassumption.
13 Qed.

```

LISTING 2.4: Formal partial proof of a sorting program in Coq

Figure 2.4a, 2.4b, 2.5a and 2.5b show the corresponding string diagram representations of the previously mentioned partial proofs illustrated in Figure 2.1a, 2.1b, 2.2a and 2.2b respectively. Similar to the DAG representation, any partial proof representation can only be of the forms illustrated by either of the two figures and any structure of a whole proof can be represented by a string diagram with some sub-diagrams arranged in sequential manner and some in parallel manner.

Two benefits of using string diagrams over DAGs to represent proofs are 1) opacity and 2) composability. First, allowing solid nodes to be opaque, i.e., ignoring the nested string diagrams³ in them, reduces the proof representation complexity, which aids us in defining a proof model for our proposed incentive mechanism in Section 4. For example, suppose we have some reward to be given to the prover or group of provers

³Nested string diagrams can occur when some lemmas of a string diagram can be further broken down into smaller lemmas, and as a result, creating individual string diagrams within nodes.

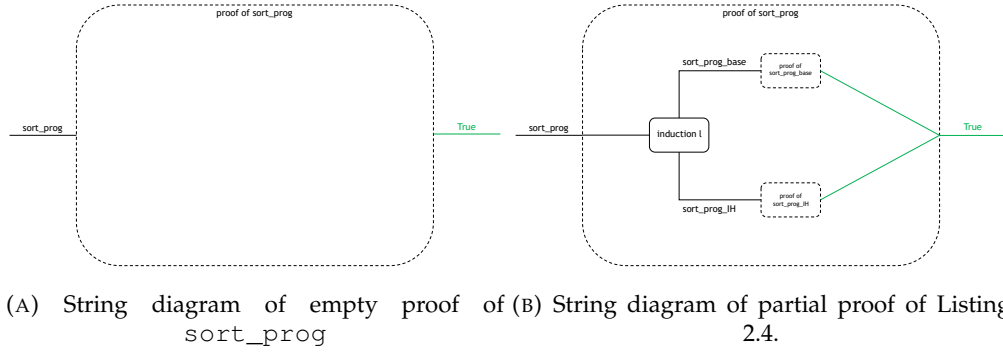


FIGURE 2.3: String diagram representation: strings - theorem statements, solid nodes - completed subproofs, dotted nodes - incompleting subproofs. Goal - all paths from edge `sort_prog` to `True` to be connected by just solid nodes.

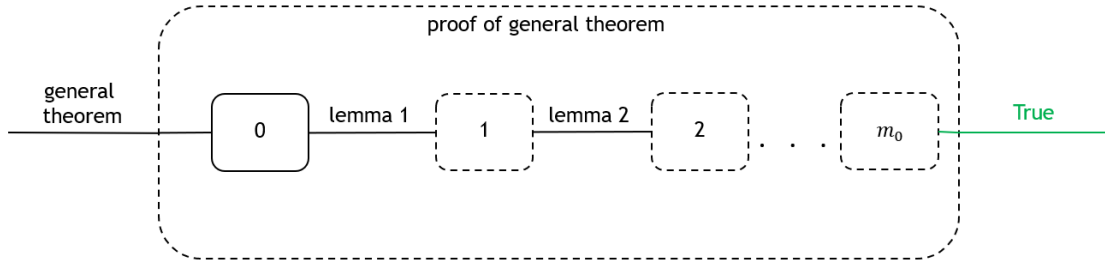
that solve the conjecture illustrated in Figure 2.4b with all its lemmas $1, 2, \dots, m_0$ completed. The string diagram representation can keep each of the solid nodes' contents opaque and not reveal their nested string diagrams (if any). In this way, the incentive mechanism just needs to take into account the relative weights between the $m_0 + 1$ subproofs ($0 \cup [m_0]$) in Figure 2.4b and allocate the rewards accordingly. Each of the lemmas $1, 2, \dots, m_0$ can then apply the incentive mechanism to distribute their own allocated reward through their own individual (inner) string diagrams. As a result, instead of having an incentive mechanism applied to a complicated DAG representation of proofs, the same can be applied recursively to string diagrams of individual nodes. Hence, this will ease the analysis of the incentive mechanism.

Second, the string diagram abstraction also allows easy composition and connection between proofs. For example, if `sort_prog` is proven completely, it will be represented by just a opaque solid node. It can then be composed with other nodes to form part of a larger proof. Figure 2.6 shows how the proof of `sort_prog` can be composed and fitted into a larger proof of a binary search program. This program requires the input list to be sorted and repeatedly split to find some chosen element through `split_search`. Conversely, composing DAGs can result in very complex representations.

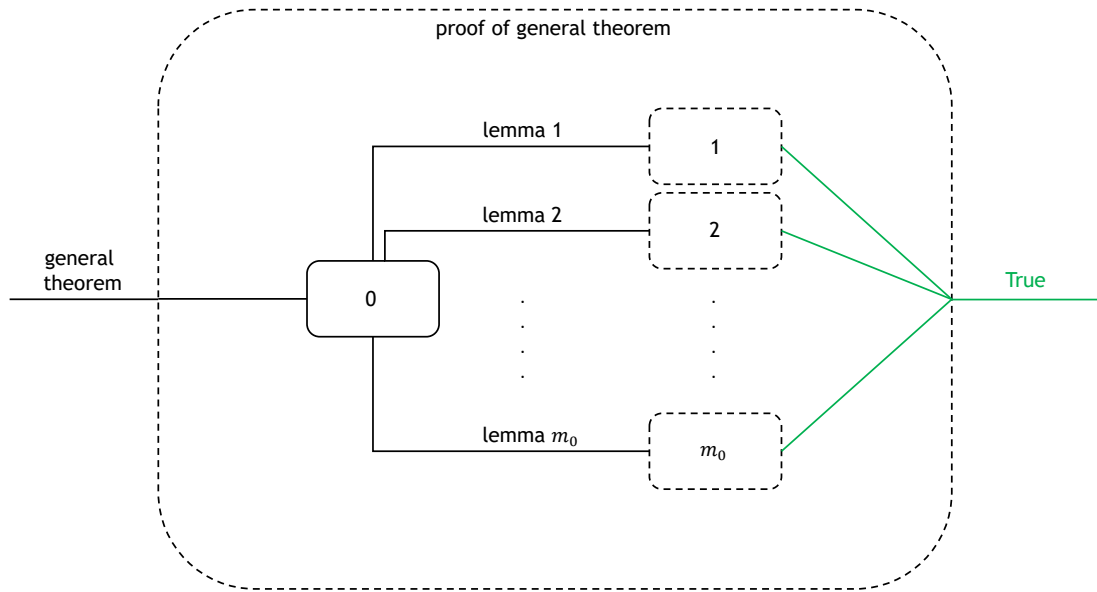
2.2 Blockchains

2.2.1 Blockchain basics

A blockchain is an immutable data structure, where the data is organized in blocks chained together to form one canonical linked list. A public blockchain allows its users to propose the data to be added, e.g., through transactions or interactions with smart contracts (more on these later). However, decision of appending every block to the chain is obtained through a consensus process participated by the nodes (servers) in a peer-to-peer network. Figure 2.7 shows a simplified diagram of what a blockchain is. Consensus rules specify the mechanism selecting a temporary leader to propose new blocks, a fork choice rule to determine the canonical chain, as well as the nature of the data contained in blocks.



(A) String diagram of partial proof with subproofs arranged in sequential manner



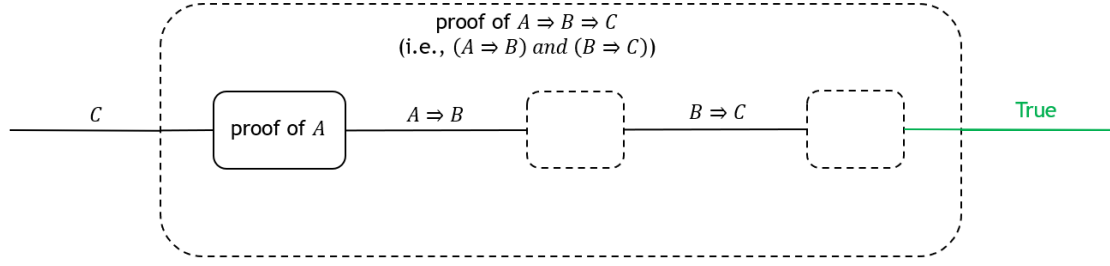
(B) String diagram of partial proof with subproofs arranged in parallel manner

FIGURE 2.4: String diagram representations of partial proofs

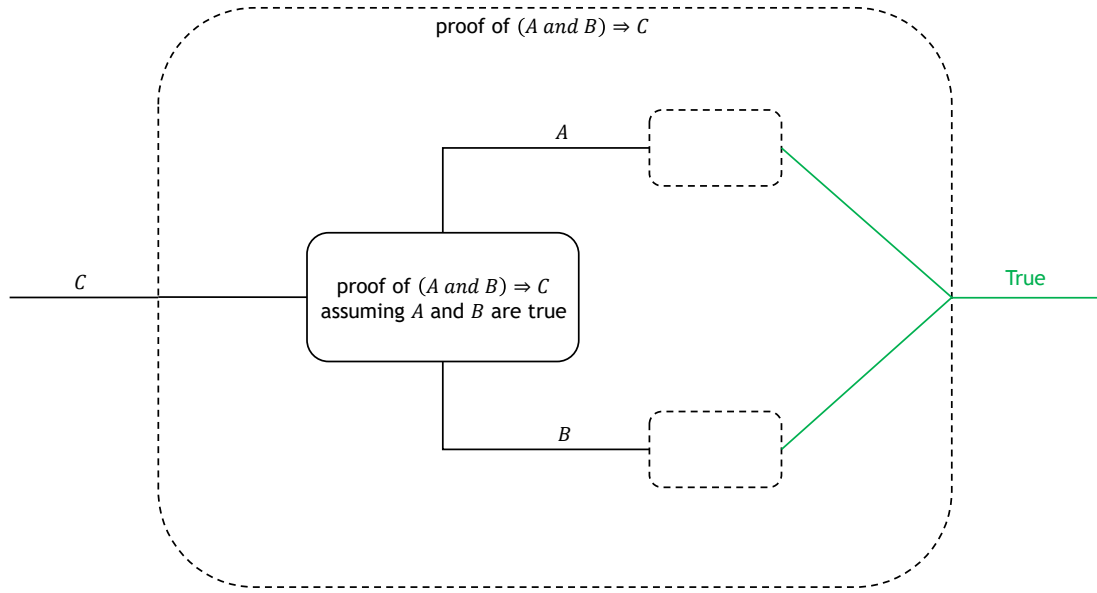
Consensus mechanisms

During run time, there will be a pool of pending data to be added to the chain. A temporary leader, a node in the peer-to-peer network, verifies and selects eligible data from the pool to be contained in a block and adds them to the chain. The leader will receive a *block reward* for every successful block addition.

Two major leader selection mechanisms dominate the landscape of blockchains. The *Proof-of-Work (PoW)* consensus mechanism (e.g., used by Bitcoin (Nakamoto, 2008), Ethereum (Gavin Wood et al., 2014) and others) selects a leader randomly, in proportion to the amount of computational power expended by consensus participants known as *miners*. To mine (i.e., add) the next block on top of an existing block, miners must solve a cryptographic problem by brute force, based on the existing block's hash. The hardness of this problem scales with the total energy expended throughout the network, as measured by the rate of successes, equivalently the speed of the chain's growth. The



(A) String diagram corresponding to the DAG representation shown in Figure 2.2a



(B) String diagram corresponding to the DAG representation shown in Figure 2.2b

FIGURE 2.5: String diagrams corresponding to the DAG representation shown in Figure 2.2

security provided by PoW equally scales with the total hashrate expended, as making alternative histories canonical in the network's view requires matching the energy expended on the current canonical history. Successful miners receive a block reward (in cryptocurrency), which is valuable on the canonical chain history. Block rewards of major cryptonetworks appreciated in "dollar" value dramatically, leading to increased competition to mine the next block and high energy consumption (Golosova and Romanovs, 2018).

Meanwhile, the *Proof-of-Stake* (PoS) consensus mechanism (e.g., to be activated by Ethereum (Buterin, Hernandez, et al., 2020), Cosmos (Kwon and Buchman, 2018) and others) selects a leader based on the amount of stake (in cryptocurrency) deposited by the participant, called *validator*. It does not require high energy consumption as the leader is selected based on the amount of stake and not computational power. The security of the consensus is based on the total amount of stake currently participating, with most PoS consensus mechanisms favoring BFT-style finalization (Castro, Liskov,

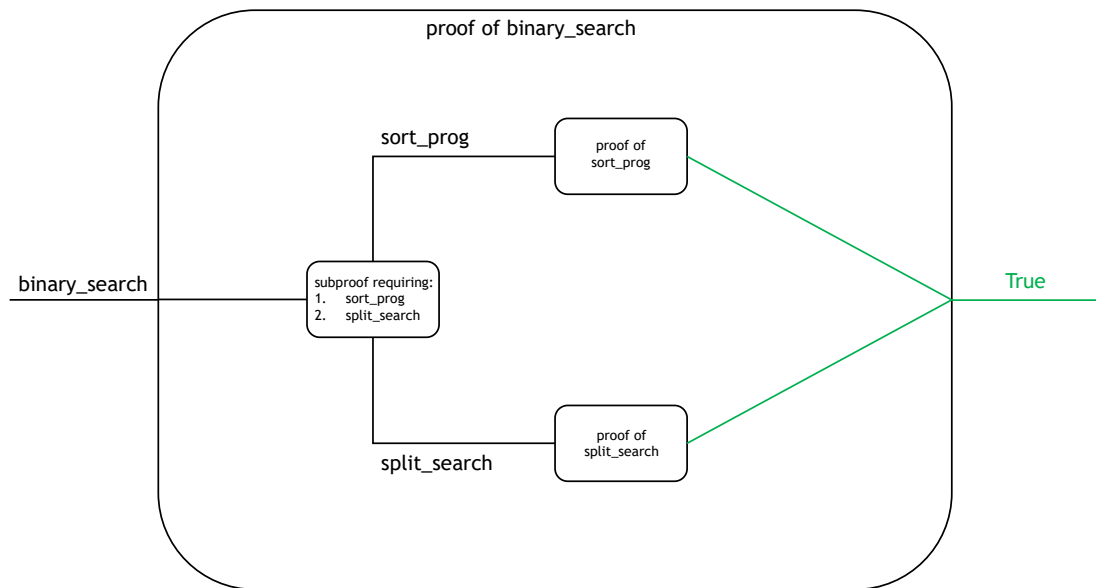


FIGURE 2.6: String diagram representing proof of a binary search specification, which requires the proof of a sorting program and a search through splitting program

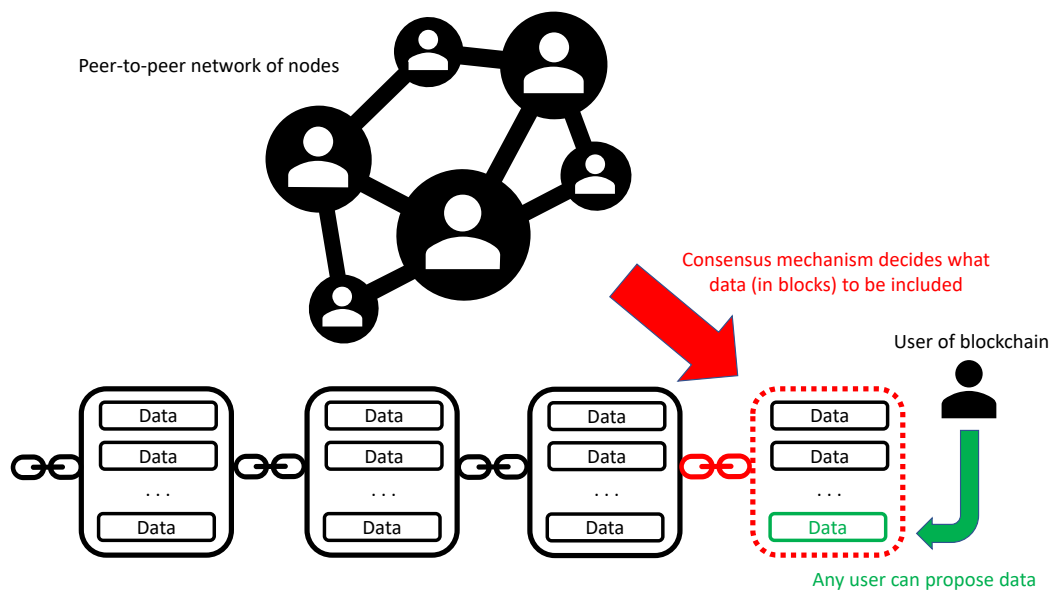


FIGURE 2.7: Simplified diagram of a blockchain

et al., 1999), providing security as long as more than two-thirds of the current stake is honest and active.

The leader selection mechanism used depends if the blockchain is *public* or *private*. Public chains do not restrict the participation set: anyone can either start hashing (in the case of PoW) or deposit some amount of stake (in the case of PoS) to become an

eligible leader. On the other hand, private chains restrict maintenance of the chain to a closed set of participants, who are typically given the responsibility to add blocks to the chain in a round-robin fashion, such as Proof-of-Authority (Wood, 2015), where leader is chosen from a selected set of participants. Private chains are usually used by organizations that possess sensitive data and want to set different permission levels for participants depending on their authorizations. As private chains are limited in sizes, they can process data very fast and do not have to deal with Sibyl attacks. However, as a result, they do not feature a native cryptocurrency to reward the leaders.

Finally, consensus mechanisms must feature a fork choice rule, allowing nodes to decide on the canonical record. For instance, the PoW puzzle can lead to a temporary fork, where two new blocks solved the puzzle of an existing at the same time (Figure 2.8). In PoW, the longest chain rule is often used (Nakamoto, 2008), while PoS networks either rely on choosing the latest finalized block (e.g., Cosmos) or a combination of finality (Buterin and Griffith, 2017) and a heaviest/"most-voted-on" chain rule (e.g., PoS Ethereum) (Buterin, Hernandez, et al., 2020).

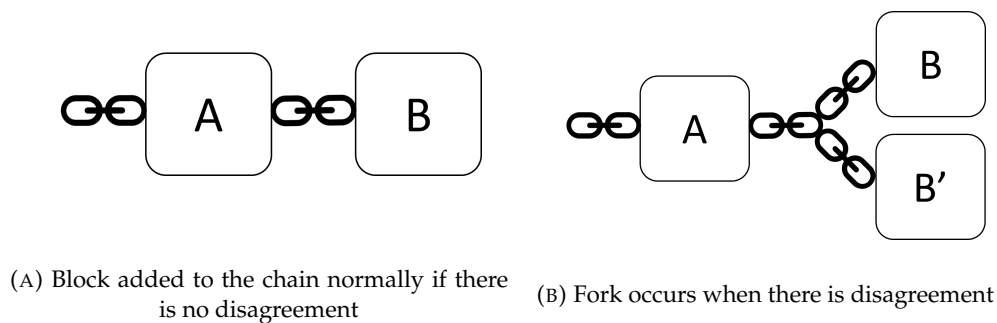


FIGURE 2.8: Diagrams to show the difference between chains with and without fork

Execution model

A blockchain works as a replicated state machine, where the "world" state is maintained and updated by the peer-to-peer network. Depending on the data, the world state can function as a decentralized ledger or even a "world computer". As a result, the scope of blockchain applications has improved from just in financial applications to many other areas such as healthcare, logistics and others (Al-Jaroodi and Mohamed, 2019). Bitcoin is one of the first known use of blockchain, where it is widely used as a cryptocurrency and the data present in the blockchain consists of just transactions between users. The goal is to create an universal digital ledger where the users' account balances are the state of this ledger and transactions are considered as the state transitions from one state to another. These transactions are dealt directly without a need of central authority, such as banks, with transparency and in a decentralized manner.

With the introduction of smart contracts, i.e., program code that are stored on a blockchain as data that contains executable instructions, a blockchain is programmable where transactions sent by users may reference and interact with not only other users but also smart contracts. The execution and transfer of data, e.g., cryptocurrency, will

be dealt accordingly if predetermined conditions implemented by the smart contracts are met. Smart contracts are typically “metered” to circumvent the halting problem, when the instruction set is Turing-complete, e.g., *gas* is the unit of cost in the Ethereum blockchain. Consensus participants execute the instructions triggered by transaction inputs to guarantee correctness of the data, so require upper bounds on the duration of the execution. Users pay for execution proportionally with the computational and storage requirements of their transactions, with the fee accruing to the block proposer who includes their transaction. With the use of smart contracts, applications can be extended to other financial areas such as securities trading and supply chain finance (Swan, 2015). Ethereum is one of the first few blockchains that is implemented with the use of smart contracts. Ethereum is often described as a “world computer” (Aung and Tantidham, 2017) as its smart contracts are used not only to update users’ account balances but also update a virtual state machine, called *Ethereum Virtual Machine* (EVM) (Gavin Wood et al., 2014), present in all nodes of its network. This EVM is the world state that the network is maintaining and each state transition is an program operation from a smart contract. As a result, decentralized applications (dApps) and decentralized autonomous organizations (DAOs) can be created and applied in areas such as in government, health and sciences (Swan, 2015). Figure 2.9 provides a simple illustration to show the differences between the two main execution models of blockchain - universal ledger and world computer.

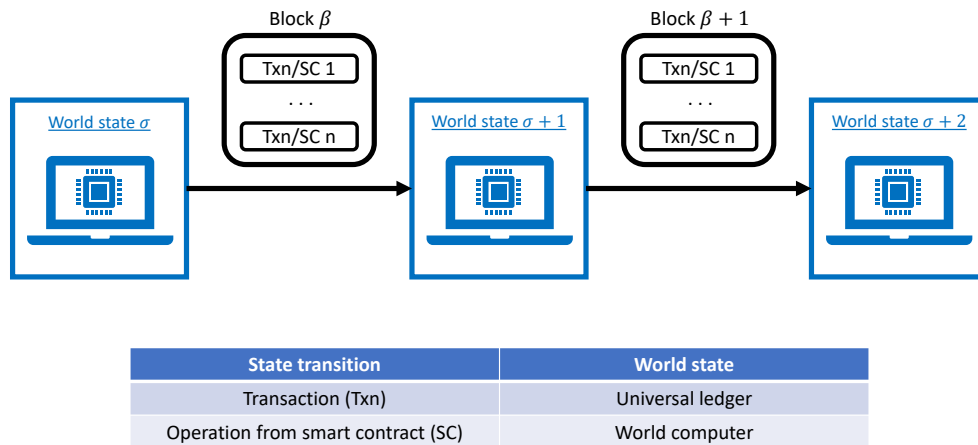


FIGURE 2.9: Simple illustration to show the differences between the two main blockchain execution models

Due to the benefits of what blockchain can provide, we will point out several important reasons on why we believe our system should be implemented within a blockchain environment in end this chapter.

Why blockchains?

The main benefits of blockchain technology include transparency, traceability and security. First, with decentralization, every transaction and flow of data will be transparent for all users as they are responsible for building and validating the blockchain itself, instead of a centralized authority. Second, as the data is immutable once it is considered valid in a blockchain, this accounts for the traceability of data flow, which is particularly useful in applications such as supply chain management (Shakhbulatov et al., 2020). Lastly, enhanced security is ensured as compared to other platforms as every data recorded is agreed upon through some consensus algorithm and each of them is encrypted with a link to the prior data through hashing, i.e., if there is any change in prior data, the hash of the subsequent data will change, which will be noticeable to the users. Building upon these reasons, we will elaborate further on why a blockchain-based approach is preferred over using systems, such as Github⁴ or arXiv⁵, and they are as follows:

Persistence of data through decentralization Although anyone can upload their files available in systems such as Github and arXiv (with endorsement from users who previously posted) and are meant to be open-source for anyone to download, they are all stored in centralized servers (Github's main servers at San Francisco, California, United States and arXiv's at Cornell University). If the servers are hacked and files are not backup by the users or originators, files and its revisions can be lost forever. On the other hand, anyone can participate in a (public) blockchain network and the data stored is maintained in a decentralized peer-to-peer network. This means the data is not just stored in a single server but in every (full) node in the network, which is managed by some user. In this way, the data will not be prone to attacks as even if one server is down, data can still be retrieved from other nodes in the network.

Verified and anonymous authorship for partial progresses ArXiv stores documentation (pdf format) of completed research publication. It does not have a system that allows collaboration of partial progresses. Furthermore, even in a completed work, the authors' names are provided at the beginning of the paper, with no indication of individual contributions within the paper. Having authors' contribution is important if there is any incentive mechanism to split any reward appropriately to be implemented. On the other hand, Github provides records of authors' contributions in each revisions of the files. Similarly, blockchain technology can provide us with such advantage as well. As every transaction must come from a verified user's address in the network, we can use it to track address of every partial proofs submission. Furthermore, as these addresses are produced cryptographically, the identity can stay anonymous. As a result, blinded judgement, e.g., on correctness, novelty or difficulty, can be done on a partial proof if needed.

History of successful and failed partial proofs Another benefit of having partial proofs recorded is to trace the history of proof generation. Although some partial

⁴<https://github.com/>

⁵<https://arxiv.org/>

proofs will eventually lead to the right proofs, some will not. Having a system that keep record of all partial work, allows us to analyze the provers' strategies and thinking from both their successful and failed attempts so to improve on future proof generation. For example, in the automated AI tools for ITPs mentioned previously, the machine learning methods used learn from libraries of formal proofs written by humans to help in their proof searches. However, they are only learning from successful attempts written by humans but not from the failed attempts. This could be one of the reasons why the effectiveness of these automated tools is still not as desirable as one would hope. Thus, having our system to keep track of both successful and failed partial proofs provides these tools with a more comprehensive training dataset to learn from.

Incentive contracts via smart contracts One of the possible reasons that the Polymath project (Gowers, 2009) did not sustain could be due to lack of incentives and incentive mechanisms present to allocate rewards, if any, appropriately. With the use of smart contracts, complicated incentive mechanisms can be implemented as these contracts are essentially executable program code to be run when certain conditions are met. Rewards can be distributed once a proof of a theorem has been completed from a prover or a group of provers. Game theoretical analysis can provide insights on how an incentive mechanism can be structured such that reward can be split appropriately among contributors. We will give a proof-of-concept of such incentive mechanism in Chapter 4.

2.2.2 Blockchain applications and advancement

Due to the potential of what blockchain technology can bring to us, there are many applications and improvements within this ecosystem that are useful and fascinating. Out of these applications and advancement, the ones that we are interested in and will be useful for our approach are decentralized applications (dApps) for storage and rating purposes, token-curated registries (TCRs), non-fungible tokens (NFTs), zero-knowledge proofs (ZKPs) and off-chain executions of transactions and smart contracts.

Decentralized applications (dApps)

With the use of smart contracts, application developers run these code on a decentralized peer-to-peer network, as opposed to running on centralized servers. This provides advantages such as privacy (as no real-world identity of developers or users is needed), resistance to censorship (due to transparency and decentralization) and data integrity (due to immutability and security of what blockchain provides). Many dApps have been developed since the emergence of smart contracts such as in areas of in government, health, sciences and many more (Swan, 2015). We will take a look at some of the applications that are essential to our blockchain-based approach.

Decentralized storage systems To store and access data storage from multiple users or devices, cloud storage systems, such as Dropbox, Microsoft OneDrive and Google Drive, are used as they are less costly and more reliable than local storage (Zeng et

al., 2009). However, there are many security and availability issues arising from this technology, such as lacking of transparency and control over the stored data (Charanya and Aramudhan, 2016). Riding along the benefits of blockchain technology such as transparency and security, decentralized storage systems, such as Storj (Labs., 2018), IPFS (Benet, 2014) and Sia (Vorick and Champine, 2014), were introduced to tackle the issues faced by cloud storage systems. Data is maintained and stored in every node in the peer-to-peer networks of the decentralized systems. They implement some variation of Proof-of-storage/space, where they incentivize users by rewarding them with their native cryptocurrencies in exchange for the proof of storage space used to store the data (Benisi, Aminian, and Javadi, 2020). For IPFS, each content is addressed by a unique hash, a random string of characters, obtained by hashing the file content. Other IPFS users interested in the content will request it by its hash to other peers on the network, until they obtain the file from a peer who is hosting it. The design pattern of hosting longer-form content, such as text, code or media over IPFS while referencing the hash in a blockchain transaction has become commonplace. While not guaranteeing availability (a file could either be missing or held offline), it is sufficient for just one party holding the file to be online for it to be available.

Decentralized rating systems Similar to cloud storage services, online rating systems such as Yelp⁶ can be prone to data modification and removal as data is stored in centralized servers. Decentralized rating systems such as the ones proposed by Shaker, Aliee, and Fotohi, 2021 and Avyukt, Ramachandran, and Krishnamachari, 2021 allows users to rate products transparently without revealing their real-life identities. Additional features, such as rewarding raters with appropriate amount (Avyukt, Ramachandran, and Krishnamachari, 2021) and using Proof of Authority (PoA) consensus mechanisms so that all rating transactions are approved by the authorized users (Shaker, Aliee, and Fotohi, 2021), ensure that ratings are done as fairly as possible.

Token-Curated Registries (TCRs)

A *token-curated registry* (TCR) (Goldin, 2017; Asgaonkar and Krishnamachari, 2018) is a list that is maintained in a decentralized manner through by users, called *curators*, who are incentivized to do so. It is similar to decentralized rating systems where curators have to rate items. However, the ratings that To illustrate how TCR works in general, we will give a simple example.

In most cases, a TCR has two stages, a *registration period*, followed by a *challenge period*, and a token currency for staking. Suppose that a list of "Biggest breakthroughs in formal mathematics in 2021" is created. During the registration period, if Alice, a curator, believes that the paper by Lim et al., 2021 is worthy of being included, she would buy and stake some TCR tokens to register the paper into the list. If all other curators agree to include our paper into the list, all just have to wait for the registration period to be over and the entry will be added. However, if there are curators that disagree with the entry, a challenge period will be initiated. Bob, another curator, can initiate the challenge by putting a stake that match Alice's stake. This will trigger and call for a vote. At this point of time, all token holders will have a say and can vote

⁶<https://www.yelp.co.uk>

whether Alice's entry to be included in the list. At the end of the challenge period, if Alice's entry is voted out of the list, Alice's stake will be lost and redistributed to Bob and other curators who voted to exclude the entry. On the other hand, if Alice's entry is voted in, Bob's stake will be lost and redistributed to Alice and those that have voted to include the entry.

There are several advantages of using TCRs over centralized lists created by big companies such as Google and Yelp. First, owner of a centralized list can arbitrary add or remove items from the list, whereas for TCRs, as they are created in a decentralized way, items in the lists are solely based on curators' votes. As a result, they would provide a better representation of the population opinion. Second, although it is free for us to vote in places like Yelp, voters are actually helping the centralized entities to sell advertising space without getting rewarded. On the other hand, TCRs incentivize curators to vote for or against items to be included in such lists through rewards, and thus, having a win-win situation for the curators and the users accessing the lists. Examples of TCRs that leverage on these advantages include adChain (Gebron, 2017) and Messari (Selkis and McArdle, 2018). In particular, Kosmarski and Gordiychuk, 2020 discussed the idea of having a TCR for scholarly journal, which would be useful in our blockchain-based approach.

Non-fungible tokens (NFTs)

To understand what is a *non-fungible tokens* (NFTs), we need to understand the concept of fungibility. An asset is fungible if its individual units are interchangeable and replaceable with asset/s, and each of whose parts are indistinguishable from one another. A piece gold is fungible as it can be replaced by the another piece of gold of the same weight. Other examples of fungible goods include commodities and dollar bills. On the other hand, a non-fungible good is unique and is irreplaceable with other asset/s of equal value. An art piece, a one-of-a-kind toy figurine and diamonds are examples of non-fungible goods as there is nothing of equal value that can replace each of the items. As a result, when selling non-fungible goods, the selling price can vary a lot as it is based on the the valuation of the buyer, e.g., one may value an art piece for \$1 while another may the same art piece for \$1 million.

NFTs are non-fungible digital assets, such as photos, videos and artworks, that are stored in a blockchain. Popular examples include Cryptokitties⁷, where you can own, breed and play with digital kitties, and NBA Top Shot⁸, where you can buy and sell short digital videos of NBA moments. Due to the cryptographic nature of blockchain, as each NFT is unique, NFTs are uniquely identifiable through hashing. Proofs of ownership and certificates of authenticity can be issued to the addresses of the owners in a NFT ledger, a blockchain ledger to track transaction of NFTs. NFTs has been increasingly gaining popularity and people are selling their NFTs for tokens in marketplaces such as OpenSea⁹ and Rarible¹⁰, so that their digital works are sold to suitable buyers who appreciate the works and are willing to exchange them with appropriate cryptocurrencies, which in turn can be exchanged to real-life currencies.

⁷<https://www.cryptokitties.co/>

⁸<https://nbatopshot.com/>

⁹<https://opensea.io/>

¹⁰<https://rarible.com/>

The opportunities of expanding on what constitutes a NFT are incredibly huge as seen from the NFT sale of co-founder of Twitter Jack Dorsey's first tweet for more than \$2.9 millions (Lambert, 2021). Some intriguing fields that could benefit from NFTs include gaming industry, where game items and their ownership can be sold as NFTs, and metaverse, where users live in a virtual world that mimics our daily lives of owning properties and doing activities digitally (Wang et al., 2021).

There are multiple benefits of owning digital NFTs over physical non-fungible goods (such as art pieces and figurines). Besides allowing owners to keep up with the latest digital trends and earning from trading of NFTs, key benefits also include the persistency and tracibility of such products. Unlike physical items, digital NFTs will not wear and tear. As such, the value of a NFT will be determined by the market and will not diminish due to defects. Each NFT will always stay at its original form and persist as long as there is a record of it somewhere in the network. Thus, NFTs are often stored in a decentralized storage system such as IPFS to ensure that their records are always present. Moreover, the benefit of traceability in blockchain technology is extended to NFTs. Due to the immutability of records in blockchains, ownership and other important information regarding each NFT are traceable. This may not be the case for physical goods as information may be lost if any manual recording or tampering occurs.

Zero-knowledge proofs (ZKPs)

Zero-knowledge proof (ZKP) is a method where one party (the prover) can prove the validity of some specific information to another party (the verifier) without disclosing any additional information. ZKP is first introduced by Goldwasser, Micali, and Rackoff, 1989 and is usually used in cases when the prover's information is meant to be kept as private. For example, ZKPs will be useful in obtaining loan or health insurance without disclosing his/her full credit score or health information respectively. The three main properties that all ZKPs must satisfy are:

- **Completeness:** If validity of the information is true, then the verifier can indeed certify that the prover possesses the information and the required input.
- **Soundness:** If the validity of the information is false, then no (dishonest) prover can prove to a (honest) verifier that he/she has the information and the required input when it is not the case.
- **Zero-knowledge:** The verifier will not know any other information from the prover other than the validity of the information.

To illustrate how ZKP could work, we will use the famous Ali Baba cave example, first introduced by Quisquater et al., 1989. Suppose there are a Prover (he) and a Verifier (she) are in front of a cave as seen in Figure 2.10. The goal is for Prover to prove to the Verifier that he has the secret password key to open the Door at the end of the cave, without revealing the content of the key. For the Verifier to verify this, the following procedure is done:

1. Verifier stands at location A. At this location, she cannot see anything that is happening in the cave.

2. Prover, whose initial location is at B, chooses to either walk to location C or D.
3. Verifier walks to location B and asks the Prover to walk to location B from direction w or e.
4. Prover, if necessary, use the secret key to open the Door, and walks to location B from the direction, if possible, that was instructed by Verifier in step 3.
5. Prover and Verifier repeats the step 1 to 4 for x number of times.

In this procedure, as long as the Prover is unable to go to B from the direction instructed by Verifier just once, it shows that Prover does not have the secret key. On the other hand, for large x , if Prover can go to B from the directions instructed by Verifier for x number of times, this provides statistical evidence that Prover indeed has the secret key without revealing its content.

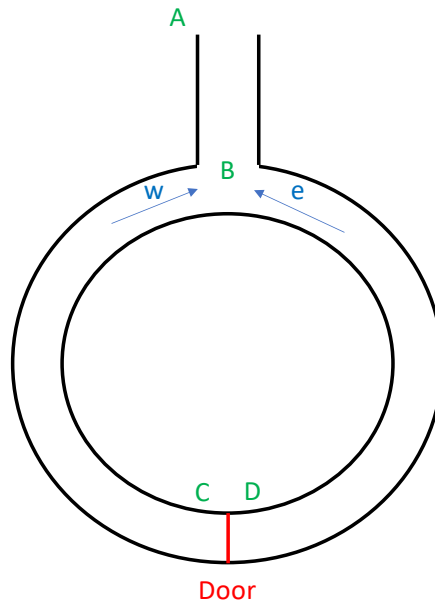


FIGURE 2.10: The Ali Baba cave

The two main kinds of ZKPs are interactive (Goldwasser, Micali, and Rackoff, 1989) and non-interactive proofs (Blum, Feldman, and Micali, 2019). An interactive ZKP involves series of interactions between the prover and verifier and the validity of the information is usually verified through statistical guarantees. The Ali Baba cave example is an interactive ZKP as multiple rounds of tasks need to be done in order for the Verifier to ensure that the Prover indeed has the secret key to the Door in the cave. On the other hand, a non-interactive ZKP does not require any interaction between the prover and the verifier but requires the use of cryptography.

Applications of ZKPs include authentication systems (Kurmi and Sodhi, 2015) and blockchain technology (Sun et al., 2021). For example, Zcash (Sasson et al., 2014) is first widespread application of zk-SNARKs, a form of ZKP, in the world of blockchains to provide privacy for its users and their transactions. Transactions in Zcash are shielded

in the sense that they are fully encrypted on the blockchain but still verifiable under the network's consensus rules by using zk-SNARK proofs. zk-SNARK stands for "Zero-knowledge Succinct Non-Interactive Argument of Knowledge". Other than being non-interactive, zk-SNARK proofs are succinct in a way that they can be verified within a very short period of time, i.e., a few milliseconds, and each of them can be of only a few hundred bytes in size even for statements about programs that are very large. As a result, it does not only provide privacy to the users, but also better efficiency and scalability for the system.

Off-chain executions of transactions and smart contracts

An on-chain transaction, or simply transactions, occurs when one user transfers some cryptocurrency to another and is reflected on the blockchain itself once it has been validated via some consensus mechanism discussed previously. As it requires time and energy to validate blocks of transactions, the speed of transactions to be added to the blockchain will be compromised and transaction fees are required to be given to the miners/validators. On the other hand, off-chain transactions are transactions that happen outside of the blockchain and are not validated or reflected directly on the chain itself. Methods of executing off-chain transactions include having some form of transfer agreement or guarantor to honor these transactions. For example, suppose there are two transactions where Alice transfers Bob say 5 ETH, followed by Bob transfers Alice 3 ETH. If the transactions happen on-chain, both transactions will be validated and recorded directly on the blockchain itself. However, if the transactions happen off-chain, the guarantor can keep track of the transaction between them (privately), and then record a single transaction without revealing any internal transactions between them and simply updating the final state of their balances, i.e., Alice's balance will decrease by 2 ETH and Bob's will increase by 2.

Seen from the simple example above, off-chain transactions provide more security and privacy of the users and their transactions as compared to on-chain transactions because the former does not track every detail directly on the blockchain itself. As off-chain transactions do not require validations by miners/validators on-chain, they can be executed instantly, which tackles the scalability issue of blockchain (Chauhan et al., 2018), and require little to no transaction fee. As such, Lightning network (Poon and Dryja, 2016) and Plasma (Poon and Buterin, 2017) are examples of off-chain protocol for transactions in Bitcoin and Ethereum respectively.

This idea of executing transactions off-chain can be further expanded to smart contracts. Smart contracts are computer programs that inhabit on the blockchain and will be executed automatically once preset conditions are met. As smart contracts are "metered" by e.g., gas in Ethereum, every implementation or transaction executed by the smart contracts requires cost. Thus, it is impractical to run computationally intensive applications such as image recognition and zero-knowledge proofs through smart contracts (Sariboz et al., 2021) as it would take up a lot of time and cost. To address this problem of computationally intensive smart contracts, Eberhardt and Tai, 2018 and Sariboz et al., 2021 propose off-chain execution of smart contracts through the use of zk-SNARKs and offloading them to a computationally powerful third party using an incentive mechanism respectively.

Chapter 3

Blockchain architecture

In this chapter, we will describe the architecture of our proposed blockchain-based system. We assume the availability of a public, programmable blockchain, where smart contracts are deployed. We call *prover* an account (or address) on the blockchain participating in the protocol. A human or AI participant (prover using automated AI tool) may operate several prover accounts. For simplicity, we consider the case where one entity corresponds to one prover only. Our system architecture can be broken down into three interconnected layers: a) the *data layer*, b) the *client layer* and c) the *incentive layer*. Firstly, the data layer describes what and how mathematical proofs can be recorded in the blockchain. Secondly, the client layer tells us how the proofs is presented and accessible to the users. Lastly, the incentive layer details how users can be rewarded for their proofs. Note that we will be presenting two variants of our system: i) one that takes in formalized proofs, and ii) one that can take in pen-and-paper proofs. Thus, for each of the sections in the rest of this chapter, we will first provide details for the formalized proof version before addressing the similarities and differences in the pen-and-paper proof version. Fig. 3.1 and 3.2 show flowcharts of how a prover's contribution gets recorded and eventually rewarded in our blockchain-based system across these three layers for the formalized proof and pen-and-paper proof variants of our system respectively.

3.1 Data layer

The data layer accepts any record and contribution of formalized mathematical objects, i.e., partial or completed proofs, conjectures, definitions and tactics. However, contributions that contain syntactically incorrect code must be disregarded by users of the protocol. There are two approaches in checking the correctness of each code.

The first approach is to have the smart contracts to be written in the language of the theorem prover used so that provers will write the formalized proofs directly as smart contracts itself. This would require the blockchain we are implementing on to have its virtual state machine compatible with the theorem prover language. Otherwise, creation of a new independent blockchain is needed. The data layer has to run the checks required to decide whether a contribution is *valid* or not, as it would have to run the theorem prover interpreter to find out the validity of the code. While this is theoretically possible in a Turing-complete environment, the cost of such operation would be prohibitively high especially for long and complicated proof code. To address this issue, we could have off-chain execution of these computationally intensive smart contracts as proposed by Sariboz et al., 2021. The idea is to offload them to other

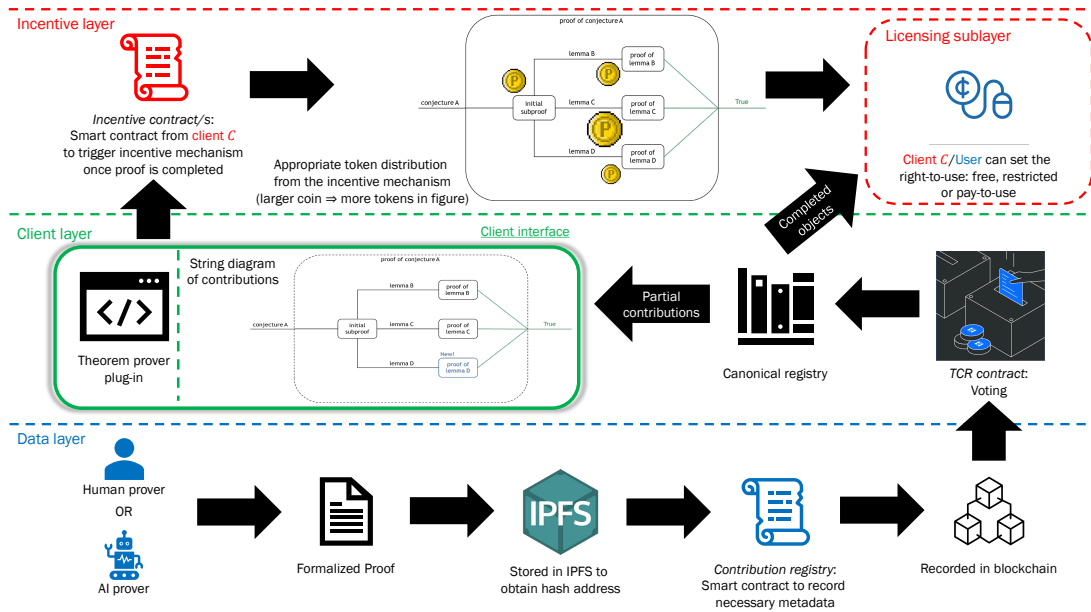


FIGURE 3.1: Flowchart of how a prover's formalized proof contribution gets recorded and eventually rewarded in our blockchain-based system across the three layers of our system architecture. Images' sources: [IPFS](#), [TCR](#), [right-to-use](#)

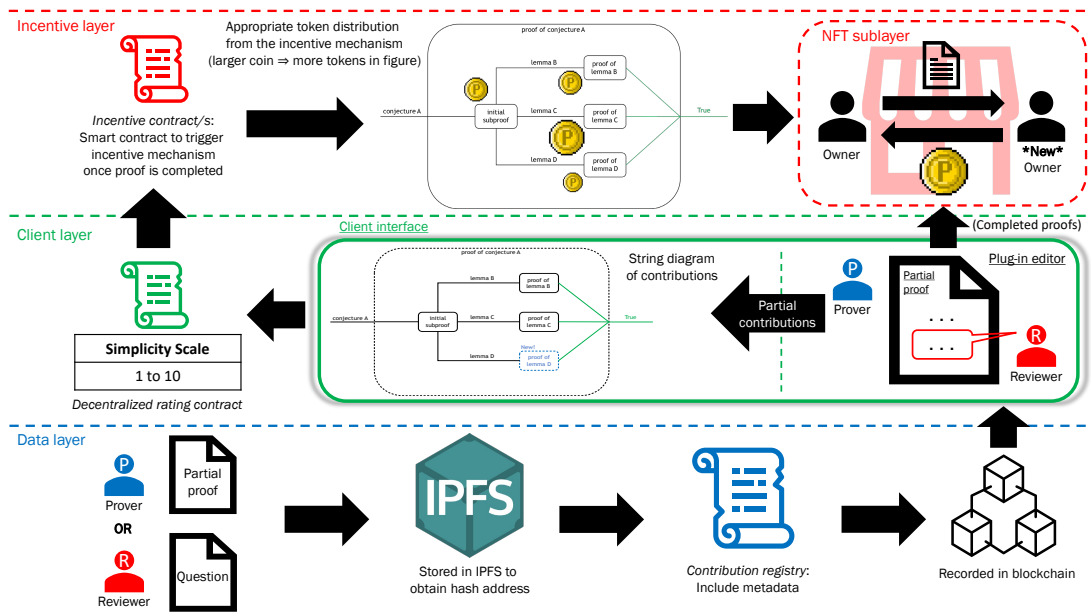


FIGURE 3.2: Flowchart of how a prover's pen-and-paper proof contribution gets recorded and eventually rewarded across the three layers of our system architecture.

computationally powerful entities using an incentive mechanism. After verifying the correctness of a smart contract, the computationally powerful entity uploads the proof

of correctness back to the blockchain for the miner or validator of the implemented consensus mechanism to verify, instead of executing the computationally intensive smart contract itself.

The other approach, which we prefer, is to make use of the following two elements: a *decentralized file storage* and a deployed smart contract on the blockchain, *contribution registry* (as seen in Figure 3.1). When a prover (human or AI system) produces a formalized proof, he would store the file in the decentralized file storage, say IPFS, and obtains the hash address of the content. Each formalized proof can be of any type: a) open conjecture statement for provers to prove, b) subproof addressing to a conjecture or continuation of other subproof, or c) completed mathematical objects such as new definitions, tactics and theorems. The prover will then have to record an entry, stating the necessary information of the formalized proof contribution, to the contribution registry. Minimally, each record contains the prover's address and the hash address of the formalized proof contribution. Additional metadata attributes include the formalized proof type and references to previous relevant contributions (e.g., "imports") can be included as well to provide more information for all provers. As only recording of minimal information of the formalized proof on the contribution registry smart contract is required, we could implement our system using existing blockchains that have smart contract capability such as Ethereum. Using decentralized file storage also incurs low energy and cost for the users to run as only the hash address of the formalized proof is required. However, miners/validators, who need to validate the correctness of every formalized proof, to download the files through the hash address and compile and check the code through their local computers manually. To address this issue, zk-SNARKs (mentioned in Chapter 2) could be utilized. With a zk-SNARK scheme in place for our system, provers can outsource the verification of their formalized proofs to users of the zk-SNARK scheme, called *zk-provers*, to verify and produce zero knowledge proofs of correctness, which are succinct and non-interactive. Although zk-SNARKs for such formalized proof validation is still a theoretical idea and does not exist yet, assuming they exist, it would ideal as miners/validators just have to check these zk-SNARK proofs instead of executing the formalized proofs on their own.

Regardless of the approach taken, as long as the formalized proofs are verified to be correctly written, i.e., syntactically correct, they will be recorded in the blockchain through the contribution registry. There ought to be both good and bad attempts of conjectures, subproofs, definitions and tactics recorded, but this is what we want our system to do. Just like in art where one may value an artwork more than others, every piece of formalized mathematical object may be valued by someone eventually. The goal is to have this ever-growing database of formalized mathematics so that any prover can look up for something that might be valuable to him/her. Furthermore, this database will include both good and bad attempts of subproofs. which automated AI systems could use for their training data as discussed in Chapter 2. However, there are bound to be spam and duplicates of proof code. Although there is some form of deterrence from the metered fee required for deployment of smart contract, we will discuss more on how we can effectively filter out these submissions in the client layer.

There are also several considerations concerning the data layer before rolling out

our system. First, it is to fix the theorem prover the formalized proof code are generated from. Although there are many types of theorem provers, it is necessary to fix one as the code from different theorem provers are incompatible with one another as they are designed upon different formal systems. In Chapter 5, our illustrative example uses Coq codes as the formalized proofs the system will take in. Unless there is universal theorem prover that every computer scientist and mathematician agrees to use, one can imagine that each theorem prover can have its own blockchain-based system to record all contributions of proofs. Second, the contribution registry can be easily bootstrapped with the current standard library of the theorem prover chosen, e.g., Coq standard library¹ for Coq, providing the foundation for new proofs and tactics to be added to the registry. All of the standard libraries' documents are stored in the decentralized file storage and provers can records all the hash addresses of "imports" that their formalized proofs require. As a result, analysis such as dependency of the files can be done as all contributions will be referenced and recorded as metadata in the contribution registry.

Pen-and-paper proof version

One key difference in the pen-and-paper proof variant of our system is the introduction of another account type: *reviewer*. Unlike formalized proofs which are verified automatically by the theorem prover software itself, if we want to expand our system to allow submissions of pen-and-paper (partial) proofs, we need this new reviewer account to review and verify the correctness of proofs manually. Note that any account can be a prover or a reviewer, depending if the user of the account wants to post a subproof or a set of questions to address issues in some subproof that is already recorded in the blockchain system. Thus, the process will begin with either a prover posting a partial (pen-and-paper) proof or a reviewer posting a set of questions, both of which are considered as a record in the contribution registry. Similar to the formalized proof version, the subproof or the set of questions will be stored in a decentralized file storage and its hash address, together with other necessary information will be recorded in the blockchain through the contribution registry. As compare to the formalized proof version, the processing of data, i.e., proofs and its necessary details, in this version is more straightforward as there is no need for zk-SNARK scheme or bootstrapping with some standard library. However, the responsibility of checking the correctness of proofs will be shift to humans and it is crucial that the client interface allows smooth interactions between provers and reviewers (more on this in the client layer). Furthermore, to prevent both provers and reviewers to spam proofs and questions respectively, the incentive mechanism will be more complicated than the one used for formalized proof version (more on this in Chapter 4).

3.2 Client layer

As mentioned earlier, the data layer accepts any record or contribution from provers as the goal is to have this huge database of formalized mathematical objects for any user

¹<https://coq.inria.fr/library/>

to utilize. However, our system will also need to deal with the issue of spam and duplicates. Despite the requirement of smart contract fees, contributions of dubious quality may well be registered, as well as identical copies of previous statements, copies aggregating multiple previous contributions or copies with refactored variable names. Heuristics may be developed to “flag” copies by comparing their types against previously published contributions, a first step towards creating a *canonical registry*. One can think of the canonical registry as the standard library of *novel* and *useful* formalized mathematical objects that the system is trying to maintain, similar to the practices of the different theorem provers’ standard standard libraries.

To achieve that, we add an incentive for provers in the protocol to *vote* on the inclusion of newly registered statements into the canonical registry. This incentive is deployed as a Token-Curated Registry (TCR), a smart contract where provers stake some amount of tokens (becoming *bonded provers*) to earn the right of voting in or voting out contributions from the canonical registry. These bonded provers may adopt the role of contributors or curators, the former when they add a record to the registry, the latter when they vote on the inclusion of a record to the canonical registry. As contributors, provers attach an inclusion stake along with their record. Any prover may challenge the inclusion of the record to the canonical registry by posting a dispute stake. As curators, provers are rewarded for voting on the correct outcome, determined by a majority rule, in the event of a challenge. If the record is voted out, the challenger receives part of the inclusion stake, while the remainder is divided among voters who rejected the record. If the record is voted in, the dispute stake of the challenger is lost to the contributor and curators who voted to include the record. Bonded provers may choose to participate in a vote or not. After some delay period, either the record enters the canonical registry unchallenged, or following a successful vote in favor of inclusion. It is important to note that as zk-SNARKs for formalized proof validation does not exist in the market yet, bonded provers have to access the proof codes via its hash address and check its validity by running the theorem prover software themselves. As a result, every entry in the canonical registry can be trusted as each of them has been verified and agreed upon by many decentralized bonded provers. Thus, this registry can be served as a form of *common knowledge* (Fagin et al., 2003) in the field of (formalized) mathematics.

Although “canonical” appears to refer to a binary property, i.e., either voted in or out, in effect a quantitative measure of weight distinguishes one contribution from another, as a varying number of curators engage in the voting process for each incoming record. Uncontroversial inclusion ought to bring more positive votes from curators, who need not expend a lot of time to identify novelty or usefulness, as would uncontroversial rejection, e.g., in the case of an invalid Coq code, which is also trivial to detect assuming bonded provers run theorem prover software themselves. Furthermore, in case of a subproof contribution, additional details such as simplicity of the subproof relative to other subproofs addressing the same theorem/conjecture statement could be required from the bonded provers so to facilitate the incentive mechanism that we will describe in Chapter 4.

The client layer is responsible for presenting the current state of the canonical registry to the provers, as an interface to the data layer. Essentially, the client layer could

function as a plug-in to the chosen theorem prover editors, allowing provers to maintain and track the contributions they are working with, associated with their weights to provide contextual information on their validity, novelty and usefulness. One such example is jsCoq², an online editor for Coq the proof assistant, by Arias, Pin, and Jouvelot, 2017. If zk-SNARK scheme is not implemented, the client layer additionally runs the engine of the theorem prover to decide on the validity of a contribution, as well as heuristics on the types of the statement to determine their potential novelty. These checks allow the bonded validators to input their votes to the TCR via the client layer.

The client remains loosely coupled with the data layer. Any interface capable of reading data from the registry, verifying validity of the contributions and publishing progress to the registry functions as a client, allowing for diversity in implementations and features. Furthermore, to display current progress in each conjecture's proof and imports of previous contributions, the client holds a string diagram (mentioned in Chapter 2), linking a contribution to its ancestors. This string diagram could provide the structure necessary for the definition of incentives explored in the Chapter 4. In a future iteration, it may be possible to compose the TCR instantiation described above with the contextual information received from the string diagram of proofs, akin to the citation graph of Ito and Tanaka, 2019. Contributions building on previously registered items would then add weight to their ancestors, indirectly recognising their usefulness and/or novelty.

Pen-and-paper proof version

As mentioned previously, the main difference in the pen-and-paper version is the necessity of manual verification of a reviewer account. A client is an interface responsible for downloading the necessary records and contributions in a structured manner, e.g., by following the string diagram representation of contributions. A client interface may be built, e.g., as a plug-in to text or PDF editors similar to Google Docs³, allowing provers to add structured metadata as well as publishing records and contributions to the blockchain. Through the text editor, provers and reviewers can interact with each other by providing subproofs of conjectures/theorems and leaving review questions/comments respectively. One such blockchain-based reviewing model that we could adapt from is Ants-Review (Trovò and Massari, 2021) where smart contracts are used to liaise between provers and reviewers, who would be rewarded for their reviews. Additionally, the interface would present data, such as the stakes, bounties and rewards, derived from the incentive layer (more details in Chapter 4).

In the formalized version, TCR is used to consolidate novel, useful and valid submissions in the canonical registry. However, in the pen-and-paper variant, the quality and validity of each proof is regulated by the incentivized debate system between the provers and the reviewers (more details in Chapter 4). Furthermore, the novelty and valuation of each mathematical proof/paper can be indirectly measured by its sales in the NFT sublayer, which will be described in the incentive layer. Just like selling popular NFTs, such as Bored Ape Yacht Club⁴ and CryptoPunks⁵, as the selling price of a

²<https://coq.vercel.app/>

³<https://docs.google.com/>

⁴<https://boredapeyachtclub.com/>

⁵<https://www.larvalabs.com/cryptopunks>

mathematical proof/paper NFT increases, it could mean that buyers value the importance of the result and are willing to pay more to own this mathematical piece of work. As a result, a TCR contract is not needed in the pen-and-paper version.

For subproof contributions, we will see that measuring the relative simplicity of a subproof to the rest of the subproofs is important in our incentive mechanism. In the formalized version, bonded provers can cast their votes and ratings through the TCR contract. However, as argued in the last paragraph, TCR is not needed in the pen-and-paper version. Thus, mechanisms, such as what the Graph⁶ or the proposed decentralized review system by Avyukt, Ramachandran, and Krishnamachari, 2021 is using, can be deployed to score such simplicity rating according to some agreed upon criteria.

Although a canonical registry is not presented in Figure 3.2, it can be implemented such that it will only register subproofs that are verified and valid. Similar as the canonical registry described for the formalized version, this will serve as a form of common knowledge within the field of mathematics where every entry can be trusted as it went through rounds of validations from many decentralized reviewers.

3.3 Incentive layer

The data layer and the client layer ensure consensus over the canonical registry. Consistent with our modular architecture, the incentive layer is once again loosely coupled with the previous client layer. Value representation is handled natively by public, programmable blockchains, allowing the protocol to maintain for each agent a *balance* denominated in *tokens*. Our protocol uses incentives in a flexible manner: any agent can deploy some incentive mechanism as a smart contract to assign or distribute reward, which makes reference to contribution records. It can be either of their own conception or from existing templates.

We start with the simplest possible template, a fixed prize payment for the completion of a single proof. This prize payment can be funded from organizations or blockchain-based public good funding models such as Quadratic Funding (Weiss, 2021) and retroactive funding (Optimism and Buterin, 2021). In this scenario, an entity deploys a smart contract holding the prize, with a single signer (“owner”) deciding to offer the prize to a contribution that formally proves the proposed statement. The mechanism records the identifier of the winning contribution and the author of the contribution receives the prize money. Although this template is simple to implement, it requires trust assumptions on the signer deciding on the winner. It is easily extended to a multi-signature implementation, where some set of signers must approve the winner before the prize money is unlocked. Further, it may be extended to the whole set of bonded provers, where these provers stake some amount of tokens to vote for the winner similar to what is described in the TCR contract from the client layer. This quorum of signers or bonded provers can also choose to split the reward via some allocation rule. In Chapter 4, we will take a look at a game theoretical analysis of how we can design an incentive mechanism to ensure provers to share their partial progresses immediately without delay.

⁶<https://thegraph.com/docs/curating>

More complex incentive systems may be added later on in a compatible manner, as long as they make reference to the string diagram representation of the original contributions. For instance, there is an open conjecture that is yet to be proven and more than one branch of partial proofs, which are significantly different from one another, are present. The incentive mechanism can allow staking of tokens on one of these branches, which could entitle stakers to a share of future rewards obtained should that branch terminate with a completed proof of the statement, with the final prover earning a larger share in proportion to the amount staked. Via this partial ownership, stakers are rewarded for detecting promising branches, while provers are induced to progress on these promising branches.

With the flexibility of implementing complex incentive mechanisms via smart contracts, we can reward the contributions within a proof and across different proofs appropriately. Thus, it is important to have the right incentive mechanism/s to encourage users to not only generate a single proof but also multiple proofs of a common problem collaboratively. For example, one could implement an incentive mechanism similar to what was studied by Babaioff et al., 2012 via a smart contract. That is, he/she will reward the first completed proof with reward of x tokens, followed by the second completed proof with $\frac{x}{2}$ tokens, then the third completed proof with $\frac{x}{4}$ tokens and so on. This mechanism of halving the reward for each completed proof will eventually cost him/her a maximum of $2x$ tokens.

3.3.1 Licensing sublayer (for formalized version)

Up to this point, we have addressed how we can implement incentive mechanisms via smart contracts to encourage provers to collaborate and solve common open conjectures. This leads to the next question: how can we incentivize users to post new formalized definitions, theorems with completed proofs and tactics? To answer this, we will have a *licensing sublayer* that allows the author to set the *right-to-use* for each of his or her formalized mathematical object. This idea was introduced in one of the prior works, Qeditas (White, 2016), where the right-of-use can be categorized into three types of uses - *free-to-use* (free to use without payment), *restricted-to-use* (unable to use at all) and *pay-to-use* (pay some x tokens in order to use). This gives the provers flexibility in keeping their formalized mathematical objects open-source, confidential or trademarked for other provers to pay to use respectively. The rights are set to free-to-use by default, and any change to this setting will have to be done by the author. We include this sublayer within the incentive layer as rewards in terms of payments can be involved to the authors of completed objects.

However, since the codes are stored and accessible in the decentralized storage system, it is possible that one could simply duplicate another prover's formalized mathematical objects, which are meant to be restricted-to-use or pay-to-use, by copying the code and create a new copy of them, setting it as free-to-use. This defeats the purpose of licensing these objects. One solution is to use zk-SNARK as discussed before. If a zk-SNARK scheme is implemented, it can provide privacy for the proofs and yet verify the validity of the code. As a result, provers can just show the specifications of their completed definitions, theorems and tactics without revealing any details or proofs. Other provers will not be able to copy or duplicate proofs as details of the original copy can be censored and its right-to-use can be maintained. This is made possible

as most of the theorem provers, e.g., Coq, allow users to use existing theorem statements/types without knowing the details of their proofs. For example, in Listing 2.2, line 15 uses the conjecture statements/types `sum_to_n_BC` and `sum_to_n_IC` without having their proofs completed. One can think of it as if the proof in line 15 uses both statements/types as “black-box” lemmas, where their proofs can be omitted.

Pen-and-paper proof version

Similar to the formalized version, any agent who wishes to give out some reward for a prover or a set of provers for solving some open conjecture can do it via a smart contract implementing an incentive mechanism. However, as mentioned before, in the pen-and-paper version, the incentive mechanism would be more complicated as other than finding a “good” allocation rule for the set of provers, it should preferably include some form of incentives for reviewers to post questions and comments for verifying and reviewing the proofs’ correctness and novelty. Moreover, it should ensure that provers and reviewers do not spam proofs and questions in hope to earn the incentives and gain the system due to time-outs in response time, i.e., provers/reviewers fail to respond before the response period closes. In Chapter 4, we will discuss about how the introduction of stakes, bounties and rewards can address these issues. Another difference is that instead of having a licensing sublayer, there is a *NFT sublayer* in the pen-and-paper version, where completed mathematical results can be sold as non-fungible tokens (NFTs).

3.3.2 NFT sublayer (for pen-and-paper version)

Unlike in the formalized version, mathematical results and proofs are written with natural language (usually in English) instead of code in the pen-and-paper version. Through natural language, a prover can freely express and argue his/her mathematical result through explaining its novelty and proof. Such form of expression through mathematical reasoning can be seen as a form of art (King, 2006; Devlin, 2000). As a result, the importance and impact of mathematical results can be subjective depending on the user/bearer. This is where NFTs marketplace comes into place. Just like how digital images, videos and audios are sold in marketplaces such as OpenSea and Rarible, scientific manuscripts such as mathematical proofs and papers can be sold as NFTs as well (Stephenson, 2021).

In our incentive layer, there is a NFT sublayer which consists of a marketplace for owners to sell their mathematical papers as NFTs. To make things clear, we will define *author/s* of a mathematical paper NFT as the original author/s of the paper and *owner/s* as the one/s who has the current ownership of the NFT property. After a complete mathematical paper containing the mathematical result with completed proof is generated through the process described in the data and client layer, the paper will be up for sale at the NFT marketplace. At this point, the owner/s of the paper are either the agent/s that posted the original open conjecture (which is then solved) with reward or the authors of the paper (in the case where they collaborate in a decentralized manner without reward from any entity for solving a conjecture). After each sale of the paper, its ownership is transferred to the buyer in exchange for tokens. Small portion, say 10%, of every sale as a form of remuneration can be given and split among the authors

according to the allocation rule, e.g., the incentive mechanism described in Chapter 4. In this way, through the NFT sublayer, even if there is no initial reward allocated for a mathematical result for completed proof, authors still may receive rewards through sales in this NFT marketplace.

Although earning from trading of NFTs may seem to be attractive for owners and authors of a mathematical paper NFT, the primary goal is to keep digital representation of the work that has been done, a trace of the work in the broader lineage of what it is building on. Thus, such NFT serves as the canonical record of the paper, while trading is almost secondary. This goal is further supported and aligned with the aims discussed in Stephenson, 2021 and Gresham, 2021. In recent years, there is a rise in research group auctioning NFTs (Jones, 2021) following the sale of the first Open Science NFT⁷ selling for 13 ETH. Other than using NFTs, works by Mackey et al., 2019 show that there also has been efforts to use blockchain as a medium to keep digital records and connections between scientific publications.

⁷<https://opensea.io/assets/ethereum/0x495f947276749ce646f68ac8c248420045cb7b5e/41443491289334730858714141368268395642829177615924976808738423867555824271361>

Chapter 4

Incentive mechanism

As mentioned, it is important to incentivize not only the provers but also reviewers (for pen-and-paper version) to collaborate and verify mathematical proofs respectively. This is what the incentive layer is designed to do via smart contracts. This chapter provides a proof-of-concept of such an incentive mechanism that incentivize provers to share their partial results immediately and reviewers to verify them to ensure correctness and novelty. We will provide some basic game theory concepts and definitions necessary in understanding our analysis before explaining our mechanism model and its analysis. The model is described for the pen-and-paper version where reviewers participation is taken into consideration. However, we will see that this model is just a generalization and can be narrowed down to a model for the formalized version where there is no reviewer. We will take a look at the basic concepts and definitions required before discussing the model and its analysis in details. We will end off the chapter with some discussions regarding the assumptions made in the model and their validities.

4.1 Game theory concepts and definitions

Game theory is the study of strategic behaviours among agents in some sort of scenario. The scenario is modelled mathematically and is defined as a *game*. The agents are assumed to be rational, which means each of them would optimize some form of utility (usually defined through a function). Based on their utility-maximizing behaviours, equilibrium concepts can be discussed which are dependent of the description of the game. In this section, we will first discuss how a form of game called *extensive game with perfect information* can be expressed mathematically and its equilibrium concepts. After which, we will see how this form of game can be extended to a *Bayesian extensive game with observable actions* and its equilibrium concept, *perfect Bayesian equilibrium (PBE)*. PBE is important and extensively used in the analysis of our incentive mechanism, which is discussed in Section 4.2. Concepts and definitions described in this section follow closely to the notes by Osborne and Rubinstein, 1994. Readers who have the prior knowledge of PBE can skip this section and move directly to the next section.

4.1.1 Nash and subgame Nash equilibrium

An extensive game is an explicit representation of the sequential structure of the decision choices between players of a game. If every player has all the information of previously occurred events when making any decision, the game is said to be with *perfect information*. Otherwise, there is *imperfect information*. We will restrict our attention

to games in which there is no simultaneous moves (i.e., no two players make decisions at the same time) as it is not required in the analysis of our incentive mechanism. We will look at the equilibrium concepts of games with perfect information before moving on to equilibrium in games where there is imperfect information in Section 4.1.2.

Definition 4.1.1. *An extensive game with perfect information has the following components:*

- (Players) A set N .
- (Histories) A set H of sequences (finite or infinite) that satisfies the following three properties:
 - (Initial history) The empty sequence $\emptyset$ is a member of H .
 - If $(a^k)_{k=1,\dots,K} \in H$ (where K may be infinite) and $L < K$ then $(a^k)_{k=1,\dots,L} \in H$.
 - If an infinite sequence $(a^k)_{k=1}^\infty$ satisfies $(a^k)_{k=1,\dots,L} \in H$ for every positive integer L then $(a^k)_{k=1}^\infty \in H$.

Each member of H is a **history** where each component of a history is an **action** taken by a player. A history $(a^k)_{k=1,\dots,K} \in H$ is **terminal** if it is infinite or if there is no a^{K+1} such that $(a^k)_{k=1,\dots,K+1} \in H$. The set of terminal histories is denoted Z .

- (Player function) A function P that assigns to each nonterminal history (i.e., each member of $H \setminus Z$) a member of N . P is defined as the **player function**, $P(h)$ being the player who takes an action after the history h .
- (Preferences) For each player $i \in N$ a preference relation $\succsim_i$ on Z , which is defined as the **preference relation** of player i .

An extensive game with perfect information is denoted by $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ whose components satisfy the conditions in the Definition 4.1.1. Note that if the set H of possible histories is finite then the game is *finite*. If the longest history is finite then the game has a *finite horizon*. After any nonterminal history h , player $P(h)$ chooses an action from the set which we defined as follows:

$$A(h) := \{a : (h, a) \in H\}$$

where (h, a) denote the history consisting of h followed by action a . Further, there are many ways in defining preference relation. The usual way is to specify, for each possible pair of terminal histories, $z_1, z_2 \in Z$, for some player i , his preference relation as $z_1 \succsim_i z_2$ if and only if $u_i(z_1) \geq u_i(z_2)$ where u_i is the (expected) utility function for player i . We shall take a look at an example to better understand Definition 4.1.1.

Example 4.1.2. Consider the two player game illustrated in Figure 4.1 where the leaves of the tree denoted by $(u_1(z), u_2(z))$ represent the payoff received by player 1 and 2 respectively for some $z \in Z$.

An extensive game with perfect information that models the individuals' predicament in Example 4.1.2 is $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ where

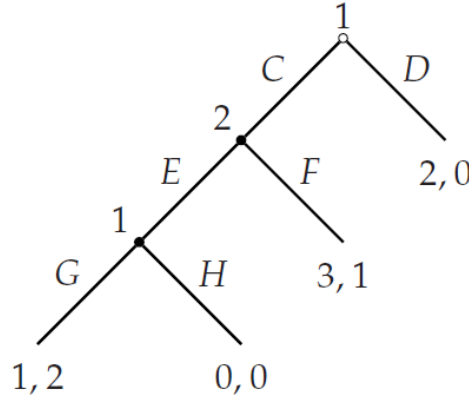


FIGURE 4.1: An extensive game in which player 1 moves both before and after player 2.

- $N = \{1, 2\}$;
- H consists of the seven histories $\emptyset, C, D, (C, E), (C, F), (C, E, G), (C, E, H)$ and $Z = \{D, (C, F), (C, E, G), (C, E, H)\}$;
- $P(\emptyset) = 1, P(C) = 2$ and $P((C, E)) = 1$;
- $z_1 \succsim_i z_2$ iff $u_i(z_1) \geq u_i(z_2)$ for $i \in N$.

To understand the solution concept, *Nash equilibrium*, of such game, we need to understand the term *strategy* for every player. A strategy can specify the action chosen by a player for *every* history after which it is his turn to move, even for histories that, if the strategy is followed, are never reached.

Definition 4.1.3. A *strategy of player* $i \in N$ in an extensive game with perfect information $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ is a function that assigns an action in $A(h)$ to each nonterminal history $h \in H \setminus Z$ for which $P(h) = i$.

In Example 4.1.2, player 1 has four strategies, CG, CH, DG and DH . That is, her strategy specifies an action after the history (C, E) even if it specifies that she chooses B at the beginning of the game. On the other hand, player 2 only has two strategies E and F .

For each strategy profile $s = (s_i)_{i \in N}$ in the extensive game $\langle N, H, P, (\succsim_i) \rangle$, the **outcome** $O(s)$ of s is defined to be the terminal history that results when each player $i \in N$ follows the precepts of s_i . That is, $O(s)$ is the (possibly infinite) history $(a^1, \dots, a^K) \in Z$ such that for $0 \leq k < K$ we have $s_{P(a^1, \dots, a^k)}(a^1, \dots, a^k) = a^{k+1}$. In Example 4.1.2, we can see that the set of strategy profiles of eight profiles $(CG, E), (CH, E), (DG, E), (DH, E), (CG, F), (CH, F), (DG, F)$ and (DH, F) . The respective outcomes are $(C, E, G), (C, E, H), D, D, (C, F), (C, F), D$ and D .

Up till now, the strategies we have defined are *pure strategies* where the players fix a particular strategy and play it with a probability of 1. However, each player can also play a *mixed strategy* where he/she chooses what to play according to a probability distribution over his actions. As the solution concepts that we discuss for extensive game

with perfect information serve just as preliminaries for PBE (which is used extensively in our incentive mechanism), we will defer the concept of mixed strategy to the next subsection.

The first solution concept, *Nash equilibrium* we define for an extensive game with perfect information ignores the sequential structure of the game. It treats the strategies as choices that are made once and for all simultaneously among players before the game begins. We will introduce another solution concept, *subgame perfect equilibrium*, later on that considers the strategies in between a game.

Definition 4.1.4. A *Nash equilibrium* of an extensive game with perfect information $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ is a strategy profile $s^* = (s_{-i}^*, s_i^*)$ such that for every player $i \in N$ we have

$$O(s_{-i}^*, s_i^*) \succsim_i O(s_{-i}^*, s_i)$$

for every strategy s_i of player i where s_{-i} represents the sequence of strategy profiles of all players except for player i .

In other words, if all players are in the state of Nash equilibrium, every player does not wish to deviate from this state as his/her utility will not increase if he/she changes strategy. A matrix/table form representation of the utilities of the game's outcomes, called **normal form**, provides us a easier way in finding Nash equilibria as compared to analyzing through an extensive form representation. From Table 4.1, we can see easily that (CH, F) , (DG, E) and (DH, E) are the Nash equilibria. At the state (CH, F) , player 1's utility = 3 and changing to any other strategy will not increase his/her utility given player 2 plays F . Same can be said for player 2 as his/her utility of 1 will not increase by choosing other strategy given player 1 plays CH . With the same reasoning, readers can check that (DG, E) and (DH, E) are Nash equilibria too.

	E	F
CG	(1,2)	(3,1)
CH	(0,0)	(3,1)
DG	(2,0)	(2,0)
DH	(2,0)	(2,0)

TABLE 4.1: Normal form of the game in Example 4.1.2. Row player = player 1. Column player = player 2.

For the solution concept, subgame perfect equilibrium, that takes into account of the sequential structure of an extensive game, the concept of *subgame*.

Definition 4.1.5. The *subgame* of the extensive game with perfect information $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ that follows the history h is the extensive game $\Gamma(h) = \langle N, H|_h, P|_h, (\succsim_i|_h) \rangle$, where $H|_h$ is the set of sequences h' of actions for which $(h, h') \in H$, $P|_h$ is defined by $P|_h(h') = P(h, h')$ for each $h' \in H|_h$, and $\succsim_i|_h$ is defined by $h' \succsim_i|_h h''$ if and only if $(h, h') \succsim_i (h, h'')$.

The notion of equilibrium we now define requires that the action prescribed by each player's strategy be optimal, given the other players' strategies, after every history. Given a strategy s_i of player i and a history h in the extensive game Γ denote by $s_i|_h$ the strategy that s_i induces in the subgame $\Gamma(h)$ (i.e. $s_i|_h(h') := s_i(h, h')$ for each $h' \in H|_h$); denote by O_h the outcome function of $\Gamma(h)$.

Definition 4.1.6. A *subgame perfect equilibrium* of an extensive game with perfect information $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ is a strategy profile s^* such that for every player $i \in N$ and every nonterminal history $h \in H \setminus Z$ for which $P(h) = i$ we have

$$O_h(s_{-i}^*|h, s_i^*|h) \succsim_i |h O_h(s_{-i}^*|h, s_i)$$

for every strategy s_i of player i in the subgame $\Gamma(h)$.

The following two facts should be apparent from the above definition:

1. Every subgame perfect equilibrium is a Nash equilibrium.
2. A subgame perfect equilibrium is a strategy profile s^* in Γ such that for any history h the strategy profile $s^*|_h$ is a Nash equilibrium of the subgame $\Gamma(h)$ (i.e. s^* induces a Nash equilibrium in every subgame).

In Example 4.1.2, the Nash equilibria (CH, F) and (DH, E) are not subgame perfect equilibria: in the subgame following the history (C, E) , player 1's strategies CH and DH induce the strategy H , which is not optimal. On the other hand, the Nash equilibrium (DG, E) is a subgame perfect equilibrium: (a) it is a Nash equilibrium, so player 1's strategy is optimal at the start of the game, given player 2's strategy, (b) in the subgame following the history C , player 2's strategy E induces the strategy E , which is optimal given player 1's strategy, and (c) in the subgame following the history (C, E) , player 1's strategy DG induces the strategy G , which is optimal.

To verify that a strategy profile s^* is a subgame perfect equilibrium, Definition 4.1.6 requires us to check, for every player i and every subgame, that there is no strategy that leads to an outcome that player i prefers. The following result shows that in a game with a finite horizon we can restrict attention, for each player i and each subgame, to alternative strategies that differ from s_i^* in the actions they prescribe after just one history. Specifically, a strategy profile is a subgame perfect equilibrium if and only if for each subgame the player who makes the first move cannot obtain a better outcome by changing only his initial action. For an extensive game Γ denote by $l(\Gamma)$ the length of the longest history in Γ ; we refer to $l(\Gamma)$ as the *length* of Γ .

Lemma 4.1.7 (The one deviation property). *Let $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ be a finite horizon extensive game with perfect information. The strategy profile s^* is a subgame perfect equilibrium of Γ if and only if for every player $i \in N$ and every history $h \in H$ for which $P(h) = i$ we have*

$$O_h(s_{-i}^*|h, s_i^*|h) \succsim_i |h O_h(s_{-i}^*|h, s_i)$$

for every strategy s_i of player i in the subgame $\Gamma(h)$ that differs from $s_i^*|_h$ only in the action it prescribes after the initial history of $\Gamma(h)$.

Proof. Proof can be found in book by Osborne and Rubinstein, 1994. □

The following gives a result on the existence of subgame perfect equilibrium for finite extensive game with perfect information.

Theorem 4.1.8 (Kuhn's theorem). *Every finite extensive game with perfect information has a subgame perfect equilibrium.*

Proof. Let $\Gamma = \langle N, H, P, (\succsim_i) \rangle$ be a finite extensive game with perfect information. We construct a subgame perfect equilibrium of Γ by induction on $l(\Gamma(h))$; at the same time we define a function R that associates a terminal history with every history $h \in H$ and show that this history is a subgame perfect equilibrium outcome of the subgame $\Gamma(h)$.

If $l(\Gamma(h)) = 0$ (i.e. h is a terminal history of Γ) define $R(h) = h$. Now suppose that $R(h)$ is defined for all $h \in H$ with $l(\Gamma(h)) \leq k$ for some $k \geq 0$. Let h^* be a history for which $l(\Gamma(h^*)) = k+1$ and let $P(h^*) = i$. Since $l(\Gamma(h^*)) = k+1$, we have $l(\Gamma(h^*, a)) \leq k$ for all $a \in A(h^*)$. Define $s_i(h^*)$ to be a $\succsim_i$ -maximizer of $R(h^*, a)$ over $a \in A(h^*)$, and define $R(h^*) = R(h^*; s_i(h^*))$. By induction we have now defined a strategy profile s in Γ ; by Lemma 4.1.7 this strategy profile is a subgame perfect equilibrium of Γ . $\square$

The procedure used in this proof is often referred to as *backwards induction*, which is important in proving the results in the analysis of our incentive mechanism. In addition to being a means by which to prove the proposition, this procedure is an algorithm for calculating the set of subgame perfect equilibria of a finite game. In Example 4.1.2, at the end of the tree (i.e., $l(\Gamma) = 0$), player 1's utility for choosing $G = 1$ is greater than the utility for choosing H . Thus, G is optimal in this subtree. Moving up the tree (i.e., $l(\Gamma) = 2$), player 2's utility of choosing $F = 1$ is less than the utility of choosing E , which is equal to 2, given player 1 chooses G . So in this subtree, (G, E) is optimal. Lastly, at the top of the tree (i.e., $l(\Gamma) = 2$), player 1's utility of choosing $D = 2$ exceeds the utility of choosing C , which is equal to 1, given that the optimal strategy profile of the previous subtree is (G, E) . Thus, the only subgame perfect equilibrium of the extensive game in this example is (DG, E) .

In this subsection, we discussed solution concepts for extensive game with perfect information. However, there are many cases where not every previously occurred events will be known to all players. As a result, players have to make decision based on partial/imperfect information. In particular, our incentive mechanism requires players (provers and reviewers) to make decisions in such circumstances. We will go through the solution concept, perfect Bayesian equilibrium (PBE), in the next subsection as it is crucial in our analysis of the incentive mechanism discussed in Section 4.2.

4.1.2 Perfect Bayesian equilibrium (PBE)

To deal with games with imperfect information, we need to generalize Definition 4.1.1 so to allow players to be partially informed about prior events when making decisions. It also allows for exogenous uncertainty, that is, some moves may be made by "chance" (i.e., each of these move is made based on some probability distribution over a set of possible moves and is not dictated by players).

Definition 4.1.9. An *extensive game* has the following components:

- (Players) A set N of players as defined in Definition 4.1.1.
- (Histories) A set H of histories as defined in Definition 4.1.1.
- (Player function) A function P that assigns to each nonterminal history (i.e., each member of $H \setminus Z$) a member of $N \cup \{c\}$. P is defined as the **player function**, $P(h)$ being the player who takes an action after the history h . If $P(h) = c$, then chance determines the action taken after history h .

- (Chance function) A function f_c that associates with every history h for which $P(h) = c$ a probability measure $f_c(\cdot|h)$ on $A(h)$, where each such probability measure is independent of every other such measure. $f_c(a|h)$ is the probability that a occurs after the history h .
- (Information partitions) For each player $i \in N$, a partition $\mathcal{I}_i$ of $\{h \in H : P(h) = i\}$ with the property that $A(h) = A(h')$ whenever h and h' are in the same member of the partition. For $I_i \in \mathcal{I}_i$, we denote by $A(I_i)$ the set $A(h)$ and by $P(I_i)$ the player $P(h)$ for any $h \in I_i$. $\mathcal{I}_i$ is the **information partition** of player i and a set $I_i \in \mathcal{I}_i$ is an **information set** of player i .
- (Preferences) For each player $i \in N$ a preference relation $\succsim_i$ on a probability distribution over Z , which is defined as the **preference relation** of player i , that can be represented as the expected value of a payoff function defined on Z .

The above definition differs from Definition 4.1.1 in two ways: (1) chance moves and (2) information partitions. First, other than the set of players N , the player function could map to c , called *chance*. If $P(h) = c$, chance takes an action, a , from $A(h)$ with probability $f_c(a|h)$. For example, the black edges in Figure 4.2 the possible moves by chance with their associated probabilities. Due to the chance moves and possibly mixed strategies of the players (we have briefly mentioned mixed strategy previously but will be going through more in details later), the preference relation $\succsim_i$ for each player i is defined on a probability distribution over Z , which is usually represented as expected payoff function on Z . Second, the information partition $\mathcal{I}_i$ groups indistinguishable histories of player i into sets I_i 's. That is, if $h, h' \in I_i$ for some $I_i \in \mathcal{I}_i$, $A(h) = A(h')$. In Example 4.1.14, the three information sets for player 1 are the sets of histories prior to 1^A , 1^K and 1^Q respectively (in this case, they are all empty sets), and the three information sets of player 2 are the sets of histories prior to nodes connecting via dotted lines of 2^A , 2^K and 2^Q respectively.

Definition 4.1.3 tell us what a player's strategy is defined as for extensive game with perfect information (Definition 4.1.9). We have a extension of that definition for a general extensive game (Definition 4.1.9) and it is called *pure strategy*. Furthermore, there are two other ways to model the possibility that a player's actions depend upon random factors. The formal definitions are as follow:

Definition 4.1.10. In an extensive game $\langle N, H, P, f_c, (\mathcal{I}_i), (\succsim_i) \rangle$, for player i , a **pure strategy** is a function that assigns an action in $A(I_i)$ to each information set $I_i \in \mathcal{I}_i$, a **mixed strategy** is a probability measure over the set of his pure strategies and a **behavioral strategy** is a collection $(\beta_i(I_i))_{I_i \in \mathcal{I}_i}$ of independent probability measures, where $\beta_i(I_i)$ is a probability measure over $A(I_i)$ (i.e., for any history $h \in I_i \in \mathcal{I}_i$ and action $a \in A(h)$ we denote by $\beta_i(h)(a)$ the probability $\beta_i(I_i)(a)$ assigned by $\beta_i(I_i)$ to the action a).

Given a set of player i 's pure strategies, a mixed strategy is a probability measure over this set. On the other hand, a behavioral strategy specifies a probability measure over the actions available to player i at each of his information sets. Thus, a behavioral strategy comprises of mixed strategies, one for each of his information sets. To see how behavioral strategies look like, one can refer to the derivation of PBE in Example 4.1.14. Just like how we define Nash equilibrium in (pure) strategies in Definition 4.1.4, we can define Nash equilibrium in mixed and behavioral strategies.

Definition 4.1.11. A *Nash equilibrium in mixed strategies* of an extensive game is (similar to Definition 4.1.4) a profile σ^* of mixed strategies with the property that for every player $i \in N$, we have

$$O(\sigma_{-i}^*, \sigma_i^*) \succeq_i O(\sigma_{-i}^*, \sigma_i)$$

for every mixed strategy σ_i of player i . A *Nash equilibrium in behavioral strategies* is defined analogously.

With the necessary preliminary concepts and definitions, we are ready to define the equilibrium concept, PBE, is. To understand PBE, we need to narrow down to a family of games called *Bayesian extensive games with observable actions*. Such a game models a situation in which every player observes the action of every other player and the only uncertainty (and thus imperfect information) is about an initial move of chance that distributes payoff-relevant personal information among the players. Moreover, the information received by each player does not reveal any information about any of the other players. We say that chance selects *types* for the players and refer to player i after he receives the information θ_i as *type* θ_i . the formal definition is as follows:

Definition 4.1.12. A *Bayesian extensive game with observable actions* is a tuple $\langle \Gamma, (\Theta_i), (p_i), (u_i) \rangle$ where

- $\Gamma = \langle N, H, P \rangle$ is an extensive game form with perfect information and simultaneous moves (where N is as defined in Definition 4.1.1 but P is a function that assigns to each nonterminal history a set of players instead of a single player, further, H and P jointly satisfy the condition that for every nonterminal history h there is a collection $\{A_i(h)\}_{i \in P(h)}$ of sets for which $A(h) = \{a : (h, a) \in H\} = \times_{i \in P(h)} A_i(h)$).

and for each player $i \in N$

- Θ_i is a finite set (which refers to the set of possible types of player i) and we write $\Theta = \times_{i \in N} \Theta_i$
- p_i is a probability measure on Θ_i for which $p_i(\theta_i) > 0$ for all $\theta_i \in \Theta_i$, and the measures p_i are stochastically independent (i.e., $p_i(\theta_i)$ is the probability that player i is selected to be of type θ_i).
- $u_i : \Theta \times Z \rightarrow \mathbb{R}$ is a von Neumann-Morgenstern (expectation-maximizing) utility function (i.e., $u_i(\theta, h)$ is player i 's payoff when the profile of types is θ and the terminal history of Γ is h).

The definition above describes a game in which chance selects the types of all players, who are subsequently fully aware at all points of all moves taken previously. This form of game can also be described as an extensive game (with imperfect information and simultaneous moves) in which the set of histories consists of $\{\emptyset\} \cup (\Theta \times H)$ and each information set of each player i takes the form $I(\Theta_i, h) = \{((\theta_i, \theta'_{-i}), h) : \theta'_{-i} \in \Theta_{-i}\}$ for $i \in P(h)$ and $\theta_i \in \Theta_i$ (so that the number of histories in $I(\Theta_i, h)$ is the number of members of Θ_{-i}).

An equilibrium candidate of such a game is a pair $((\sigma_i), (\mu_i)) = ((\sigma_i(\theta_i))_{i \in N, \theta_i \in \Theta_i}, (\mu_i(h))_{i \in N, h \in H \setminus Z})$, where each $\sigma_i(\theta_i)$ is a behavioral strategy of player i in Γ (i.e., the

strategy used by type i of player i) and each $\mu_i(h)$ is a probability measure on Θ_i (i.e., the common belief of all players other than i about player i 's type after history h). The profile (σ_i) describes the information in a profile of all players' behavioral strategies and the profile (μ_i) provides us the their beliefs given that every player has perfect information but only uninformed about the other players' types. To define formally the equilibrium concept, PBE, of Bayesian extensive game with observable actions, we additionally need the following definitions:

Let (σ_i) be a profile of behavioral strategies in Γ . We define $O(\sigma_{-i}, s_i, \mu_{-i}|_h)$ to be the probability measure on the set of terminal histories of Γ given that player i uses the strategy s_i in Γ , each type θ_j of each player j uses the strategy $\sigma_j(\theta_j)$, the game has reached h , and the probability that i assigns to θ_{-i} is derived from $\mu_{-i}(h)$.

Definition 4.1.13. Let $\langle \Gamma, (\Theta_i), (p_i), (u_i) \rangle$ be a Bayesian extensive game with observable actions, where $\Gamma = \langle N, H, P \rangle$. A pair $((\sigma_i), (\mu_i)) = ((\sigma_i(\theta_i))_{i \in N, \theta_i \in \Theta_i}, (\mu_i(h))_{i \in N, h \in H \setminus Z})$, where $\sigma_i(\theta_i)$ is a behavioral strategy of player i in Γ and $\mu_i(h)$ is a probability measure on Θ_i , is a **perfect Bayesian equilibrium** of the game if the following conditions are satisfied:

- (Sequential rationality) For every nonterminal history $h \in H \setminus Z$, every player $i \in P(h)$, for every $\theta_i \in \Theta_i$ the probability measure $O(\sigma_{-i}, \sigma_i(\theta_i), \mu_{-i}|_h)$ is at least good for type θ_i as $O(\sigma_{-i}, s_i, \mu_{-i}|_h)$ for any strategy s_i of player i in Γ .
- (Correct initial beliefs) $\mu_i(\emptyset) = p_i$ for each $i \in N$.
- (Action-determined beliefs) If $i \notin P(h)$ and $a \in A(h)$ then $\mu_i(h, a) = \mu_i(h)$; if $i \in P(h)$, $a_i \in A(h)$, $a'_i \in A(h)$ and $a_i = a'_i$ then $\mu_i(h, a_i) = \mu_i(h, a'_i)$.
- (Bayesian updating) If $i \in P(h)$ and a_i is in the support of $\sigma_i(\theta_i)(h)$ for some θ_i in the support of $\mu_i(h)$ then for any $\theta'_i \in \Theta_i$ we have

$$\mu_i(h, a_i)(\theta'_i) = \frac{\sigma_i(\theta'_i)(h)(a_i) \cdot \mu_i(h)(\theta'_i)}{\sum_{\theta_i \in \Theta_i} \sigma_i(\theta_i)(h)(a_i) \cdot \mu_i(h)(\theta_i)}.$$

The following are the interpretation for each of the above conditions:

Sequential rationality The behavioral strategy $\sigma_i(\theta_i)$ of each type θ_i of each player i must be optimal for type θ_i after every sequence of events.

Correct initial beliefs The other players' initial beliefs about the type of each player i is given by p_i .

Action-determined beliefs Only a player's actions can influence the other players' beliefs about his/her type. That is, (a) if player i does not have to move at the history h then the actions taken at h do not affect the other players' beliefs about player i 's type and (b) if player i is one of the players who takes an action at h then the other players' beliefs about player i 's type depend only on h and the action taken by player i , not on the other players' actions.

Bayesian updating Player i 's action at the history h is consistent with the other players' beliefs about player i at h , given σ_i . This means in addition to the condition of action-determined beliefs, the players' beliefs are derived using Bayes' rule based on player i 's actions. Players update their beliefs about player i via Bayes' rule until his behavior contradicts his strategy σ_i . If such contradiction happens, they form new conjecture about player i 's type and use it as the basis for future Bayesian updating until another contradiction happens again.

We shall end this section with an example to see how PBE can be derived.

Example 4.1.14 (Three-Cards Poker). This is a card game where it involves two players - player 1 (he) and 2 (she), and three cards - Ace (A), King (K) and Queen (Q). The ordering of the cards is as follows: Ace beats King and Queen, and King beats Queen. Before the start of the game, each player places 1 dollar on the table. When the game starts, the three cards are faced down and the two players choose 1 card each (without revealing) randomly. First, depending on the card player 1 has chosen, he decides to bet (B) or fold (F). If he decides to bet, he needs to place another 1 dollar on the table. Otherwise, the game ends and player 2 gets the money on the table if he folds. Next, player 2 decides to call (C) or fold (F) based on the card she has chosen. If she decides to call, both players reveal their cards and depending on the ordering of their cards, the winner will get all the money on the table. Otherwise, player 1 gets the money. Figure 4.2 shows the payoff tree of the game. We denote player i with card j as type i^j where $i \in \{1, 2\}$ and $j \in \{A, K, Q\}$.

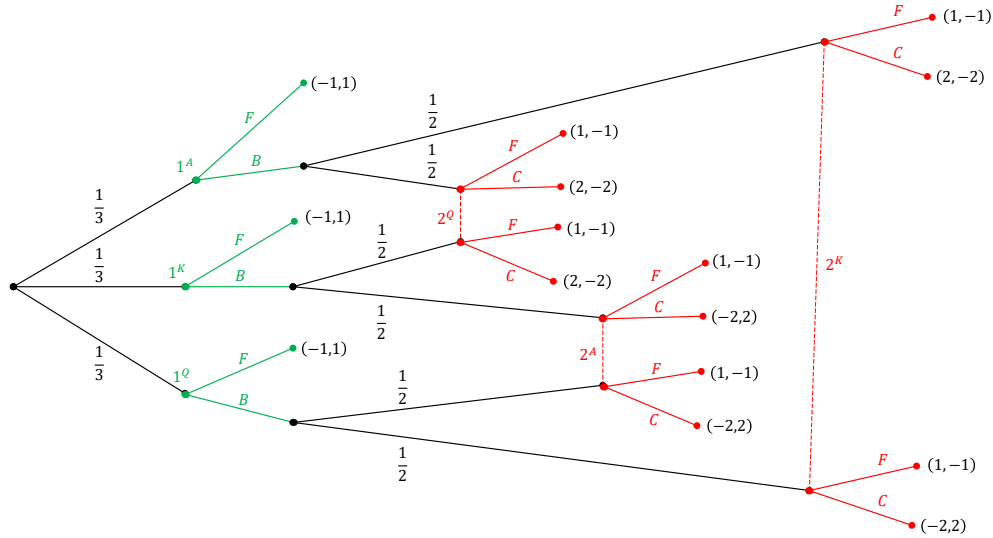


FIGURE 4.2: Payoff tree of Example 4.1.14

We will go through the obvious strategies for both players in the game first. For player 1, type 1^A will always bet as regardless of what card player 2 gets, he will win the game. For player 2, with the same reasoning, type 2^A will always call. On the other hand, type 2^Q will always fold with her queen as she will definitely lose the game. Next, we will go through what strategy player 1^K and 1^Q will choose.

If player 1 has the King card, it is obvious that the expected payoff for player 1 if he folds is -1. Given that the information set of type 1^K , both the probabilities that player 2 is type 2^A and 2^Q are $\frac{1}{2}$. Suppose he bets, player 2^A will call and his payoff is -2. Otherwise, player 2^Q will fold and his payoff will be +1. From here, we can calculate his expected payoff from betting: $\frac{1}{2}(-2) + \frac{1}{2}(+1) = -\frac{1}{2}$. Thus, betting is the sequential rational choice given his beliefs of player 2's strategy.

For the case if player 1 has the Queen card, we shall see that player 1^Q must play a mixed strategy of folding and betting. If player 1^Q always folds, then it is sequentially rational for player 2^K to always fold as she knows that player 1 will only bet if he is of type 1^A . However, if player 2^K always folds, player 1^Q should not fold as her expected payoff from betting would be $-\frac{1}{2}$, which is greater than her payoff of -1 if she folds. This leads to a contradiction that player 1^Q always fold. Thus, player 1^Q cannot always fold. On the other extreme end, if player 1^Q always bets, player 2^K must believe that player 1 is equally likely to be type 1^A or 1^Q . As a result, her sequential rational choice would be to call as her expected payoff of calling (i.e., 0) is greater than that of folding (i.e., -1). But if player 2^K always calls, then it is clear that player 1^Q should not bet, and hence, a contradiction.

Player 1^Q must be indifferent between betting and folding in the PBE. We will let p represents the probability of him betting and $1 - p$ represents the probability of him folding. Similarly, player 2^K must be indifferent between folding and calling. We will let q be the probability of her calling and $1 - q$ be the probability of her folding. We will solve for the p and q in the PBE. The expected payoff of player 1^Q betting is as follows:

$$\begin{aligned}
 & \mathbb{E}(1^Q \text{ bets}) \\
 &= \frac{1}{2}(\text{payoff if player 2 is type } 2^A) + \frac{1}{2}(\text{payoff if player 2 is type } 2^K) \\
 &= \frac{1}{2}(\text{payoff when } 2^A \text{ always call}) + \frac{1}{2}(q \cdot \text{payoff when } 2^K \text{ calls} + (1 - q) \cdot \text{payoff when } 2^K \text{ folds}) \\
 &= \frac{1}{2}(-2) + \frac{1}{2}q(-2) + \frac{1}{2}(1 - q)(1)
 \end{aligned}$$

Since the payoff for player 1^Q folding is -1, for him to be indifferent between betting and folding, we have:

$$\begin{aligned}
 \mathbb{E}(1^Q \text{ bets}) &= \mathbb{E}(1^Q \text{ folds}) \\
 \frac{1}{2}(-2) + \frac{1}{2}q(-2) + \frac{1}{2}(1 - q)(1) &= -1 \\
 q &= \frac{1}{3}
 \end{aligned}$$

Given the belief of player 1^Q 's indifferent condition, player 1^K calls the bet $\frac{1}{3}$ of the time. Next, to find the value of p , suppose player 2^K is indifferent between calling and folding, we have the following:

$$\begin{aligned}\mathbb{E}(2^K \text{ calls}) &= \mathbb{E}(2^K \text{ folds}) \\ \mathbb{P}(1^A | 2^K \wedge 1 \text{ bets})(-2) + \mathbb{P}(1^Q | 2^K \wedge 1 \text{ bets})(2) &= -1\end{aligned}$$

To find the expression of $\mathbb{E}(2^K \text{ calls})$ in terms of p , we need to use the Bayesian updating condition as follows:

$$\begin{aligned}\mathbb{P}(1^A | 2^K \wedge 1 \text{ bets}) &= \frac{\mathbb{P}(2^K \wedge 1 \text{ bets} | 1^A)\mathbb{P}(1^A)}{\mathbb{P}(2^K \wedge 1 \text{ bets} | 1^A)\mathbb{P}(1^A) + \mathbb{P}(2^K \wedge 1 \text{ bets} | 1^Q)\mathbb{P}(1^Q)} \\ &= \frac{\frac{1}{2} \cdot \frac{1}{3}}{\frac{1}{2} \cdot \frac{1}{3} + \frac{1}{2} \cdot p \cdot \frac{1}{3}} \\ &= \frac{1}{1+p} \\ \Rightarrow \mathbb{P}(1^Q | 2^K \wedge 1 \text{ bets}) &= 1 - \frac{1}{1+p} \\ &= \frac{p}{1+p}\end{aligned}$$

Thus, we can solve for p :

$$\begin{aligned}\mathbb{P}(1^A | 2^K \wedge 1 \text{ bets})(-2) + \mathbb{P}(1^Q | 2^K \wedge 1 \text{ bets})(2) &= -1 \\ \frac{1}{1+p}(-2) + \frac{p}{1+p}(2) &= -1 \\ p &= \frac{1}{3}\end{aligned}$$

This means that player 1^Q "bluffs" by betting with a Queen card $\frac{1}{3}$ of the time.

The PBE for Example 4.1.14 is as follows:

- σ_1 - 1^A : bets w.p. 1, 1^K : bets w.p. 1, 1^Q : bets w.p. $\frac{1}{3}$ and folds w.p. $\frac{2}{3}$
- σ_2 - 2^A : calls w.p. 1, 2^K : calls w.p. $\frac{1}{3}$ and folds w.p. $\frac{2}{3}$, 2^Q : folds w.p. 1
- μ_1 - 1^A believes player 2 is 2^K w.p. $\frac{1}{2}$ and 2^Q w.p. $\frac{1}{2}$, 1^K believes player 2 is 2^A w.p. $\frac{1}{2}$ and 2^Q w.p. $\frac{1}{2}$, 1^Q believes player 2 is 2^A w.p. $\frac{1}{2}$ and 2^K w.p. $\frac{1}{2}$
- μ_2 - (her information sets follow from player 1 betting) 2^A believes player 1 is 1^K w.p. $\frac{3}{4}$ and 1^Q w.p. $\frac{1}{4}$, 2^K believes player 1 is 1^A w.p. $\frac{3}{4}$ and 1^Q w.p. $\frac{1}{4}$, 2^Q believes player 1 is 1^A w.p. $\frac{1}{2}$ and 1^K w.p. $\frac{1}{2}$

4.2 Model of the incentive mechanism

With all the necessary game theory concepts and mechanisms covered, we are ready to take a deep dive into how a game designer can come out with an incentive mechanism design that would encourage provers to share their partial progresses immediately while ensuring correctness through incentives for the reviewers. First, we shall

take a look at how we can view the process of solving a common problem/conjecture from partial progresses of provers as a model we call *Proof completion game* and we will assume that the conjecture is provable¹. Next, as the process involves interactions between reviewers and provers, we shall also see how we can incentivize this "debate" process between the two parties. Lastly, we shall look at the remaining dynamics and strategies required for the analysis, which will be done in the next section.

4.2.1 Proof completion game model

A Proof completion game is the process of solving subproofs, which eventually lead to completion of a whole proof of a mathematical problem, which comprises of the two components, *string diagram representation of partial proof* and players, where we call them *provers*.

(I) String diagram representation of partial proof A partial proof of a mathematical theorem/conjecture provided by a prover can be illustrated using a string diagram either as shown in Figure 2.4a or Figure 2.4b. It consists of a set of $\{0\} \cup [m_0]$ subproofs, where subproof 0 is the initial subproof the prover solved (represented by solid node) and the set $[m_0] := \{1, 2, \dots, m_0\}$ consists the m_0 subproofs/lemmas left unsolved (represented as dotted nodes) as conjectures required by subproof 0, arranged either in sequential nodes or in parallel nodes respectively. For unsolved subproofs arranged in sequential manner, any subproof $u \in [m_0]$ will only be available to solve if all preceding subproof/s $v < u$ is/are solved. For unsolved subproofs arranged in parallel manner, any subproof $u \in [m_0]$ will only be available to solve after subproof 0 is solved. Note that the node covering all subproofs $\{0\} \cup [m_0]$ (labelled as "proof of general theorem") is dotted as the whole proof has not been completed yet. Once all subproofs $\{0\} \cup [m_0]$ are solved, where the individual nodes will turn solid, this node then will turn solid, indicating that the whole proof has been solved.

(II) Provers There exists a set of $[n] := \{1, 2, \dots, n\}$ of provers, working independently, with distinct *prover-subproof aptitudes*, $a_i(u)$, which represents the aptitude of Prover i for subproof u , and are committed to complete the whole proof.

There are several details to take note of regarding the string diagram representation of a partial proof. First, once a prover, say P , provides an initial subproof 0 of a conjecture, the lemmas following this initial subproof in the string diagram are fixed and assumed to be true. This means that we assume that all provers are committed to complete the whole proof following P 's approach and will not try to provide an alternate approach of solving the conjecture, which will result in an alternate string diagram (different from P 's) that the incentive mechanism will be deployed on. We will discuss some considerations and limitations due this assumption in Chapter 6. Second, in the paper by Banerjee, Goel, and Kollagunta Krishnaswamy, 2014, each subproof is considered as "solved" once a prover provided the proof. However, we know that this is

¹Disclaimer: The proposed incentive mechanism for proof collaboration in this chapter works for provable conjecture. We will discuss the possibility of expanding this incentive mechanism to include cases where the conjecture turns out to be false and unprovable when we conclude in Chapter 6.

not the case for a pen-and-paper proof, as it needs to get validated through a reviewing process, similar to the format in conferences' submissions, before it can be considered as solved. Otherwise, if it gets invalidated, it will still remain as unsolved. To motivate reviewers, we describe in the next section how the reviewing process is incentivized, fostering debate on the validity of a given claim.

4.2.2 Incentivized reviewing process

Building on the approach of Carré et al., 2021, we will start by describing the process with just two agents, Prover and Reviewer, debating the correctness of a single proof claim, and using machine-level verification (formalized proofs) as the final judgement if needed. Each debate has a fixed maximum number of challenge rounds, say L . The challenge round decreases starting from L all the way to 0 if the debate did not stop at any earlier step. We use Q_l, C_l for $l \in \{0, \dots, L\}$ to denote the question posted by Reviewer and the claim of proof provided by Prover at step l , respectively. Every Q_l comes with a bounty cost of β_l and every C_l comes with a stake pair's cost of $\sigma_l^\uparrow, \sigma_l^\downarrow$ for $l \in \{0, \dots, L\}$. Each Q_l has either a status of "unanswered" or "answered" and each C_l has either a status of "validated" or "invalidated" for $l \in \{0, \dots, L\}$. Prover and Reviewer alternate between turns to respond in a debate, and begin with either:

1. Reviewer posts an open conjecture, i.e., an question, with status "unanswered", Q_L , for any prover to provide a claim of proof, or
2. Prover posts a claim of proof, C_L , for some mathematical statement without addressing to any question.

Thus, the turns taken in a full debate can be viewed as a sequence,

$$(Q_L, C_L, Q_{L-1}, C_{L-1}, \dots, Q_1, C_1, Q_0, C_0),$$

where in this case, the debate starts off with Reviewer posting a mathematical question/conjecture. On the other hand, if the debate starts off with a claim of proof of some mathematical statement, Q_L will not exist, i.e.,

$$(C_L, Q_{L-1}, C_{L-1}, \dots, Q_1, C_1, Q_0, C_0).$$

If the debate terminates at any challenge step, say $l' \in \{0, 1, \dots, L-1, L\}$, the list will look like $(Q_L/C_L, \dots, Q_{l'}, C_{l'})$ if there is no response from Reviewer or $(Q_L/C_L, \dots, C_{l'+1}, Q_{l'})$ if there is no response from Prover.

Prover may take the following actions:

- For $l \in \{1, \dots, L\}$ (i.e., intermediate and initial rounds):
 - Prover can provide a claim of proof C_l answering Q_l with a stake pair's cost of $\sigma_l^\uparrow$ (an up-stake bet for answering Q_l) and $\sigma_l^\downarrow$ (a down-stake bet for Reviewer to post Q_{l-1}).
 - If the debate begins with C_L , then the Prover is not answering any prior question, and thus, $\sigma_L^\uparrow = 0$.

- For $l = 0$ (i.e., final round):
 - Prover must provide a formal proof C_0 addressing Q_0 a stake cost of $\sigma_0^\uparrow$ ($\sigma_0^\downarrow = 0$ as the debate terminates at step 0).
 - If the formal proof is verified, C_0 gets the status "validated", otherwise, it gets the status "invalidated".
- For all cases $l \in \{0, \dots, L\}$ (i.e., all rounds):
 - All claims of proof addressing Q_l must be proposed within a response time τ_l of Q_l 's publication.
 - If a claim of proof C_l addressing Q_l gets the status "validated", then this claim is said to be answering Q_l and Q_l gets the status "answered"; if no such claim exists, then Q_l gets the status "unanswered".

Reviewer may take the following actions for $l \in \{1, \dots, L\}$:

- If the debate begins with Q_L , the question is not addressing any previous claim and is instead an open problem for any prover with a bounty cost of β_L .
- Reviewer can dispute claim C_{l+1} by asking question Q_l with a bounty cost of β_l regarding any part of the claim.
- All questions about C_l must be asked within the verification time θ_l of C_l 's publication.
- If the final status of a question for a claim is "unanswered", then this question is said to be a defeating question, and the claim gets the status "invalidated"; if no such question exists, the claim gets the status "validated".

Up to this point, we have only described the debate between two agents (Prover and Reviewer). With the right incentive mechanism, the roles of Prover and Reviewer are completely decentralized, e.g., through a blockchain environment, a market of provers and reviewers is enabled, where an agent can play both roles at various levels. Every proof posted by a prover is subjected to questions from all agents in the system. Similarly, every question/conjecture posted by a reviewer is subjected to responses from all agents in the system. The incentive mechanism for the system is based on bounties $(\beta_l)_l$, with stakes $(\sigma_l^\uparrow, \sigma_l^\downarrow)_l$ as follows:

- If a claim C_l addressing a question Q_l is the first one to get the status "validated", then C_l receives the bounty β_l from Q_l and $\sigma_l^\uparrow, \sigma_l^\downarrow$ is refunded back to C_l .
- If a claim C_l addressing a question Q_l gets the status "invalidated", then Q_l receives the stake $\sigma_l^\uparrow$ from C_l and β_l is refunded back to Q_l .
- If a question Q_l disputing a claim C_{l+1} is the first one to get the status "unanswered", then Q_l receives the stake $\sigma_{l+1}^\downarrow$ from C_l and β_l is refunded back to Q_l .

Together with all the details of the incentivized reviewing process described above, our model includes the following assumptions:

(III) Stakes, bounties and rewards Every claim of subproof u at challenge step l posted by a prover requires a stake pair cost of $(\sigma_l^u)^\uparrow, (\sigma_l^u)^\downarrow$ and every question posted by a reviewer requires a bounty cost of β_l^u . A claim of subproof u that is validated at challenge step l has associated reward $R_l^u \geq 0$, awarded to the first claim that is publicly shared and validated by the reviewing process. We assume that these rewards are fixed exogenously. On every subproof, only the first attempt of a prover's claim is eligible for stakes, bounties and rewards. If her claim of some subproof gets invalidated at the first attempt, she can only post formalized proof of that subproof and does not require to put down any stake or receive any reward for her subsequent attempts.

4.2.3 Dynamics and strategies of the model

The following model's dynamics and strategies are also assumed:

(IV) Rate of finding a claim A claim of subproof u is found by Prover i after a random time interval drawn from an exponential distribution, i.e., $T_i^u \sim \text{Exp}(a_i(u))$.

(V) Confidence of a claim Upon finding a claim of subproof u , Prover i receives a (random) signal $P_i^u \in [0, 1]$, uniform on $[0, 1]$, such that $\mathbb{E}[X|P_i^u] = P_i^u$ where $X = \mathbb{1}_{\{\text{Claim is formalizable}\}}$, i.e., one can think of P_i^u as Prover i 's belief/confidence that her claim of proof u is formalizable. We assume this signal P_i^u is independent from her aptitude to the proof, $a_i(u)$. Prover decides when to share, and hence, publish the claim, which is thereafter subjected to validation by the incentivized reviewing process described in Section 4.2.2.

(VI) Strategy space The game proceeds in continuous time - for ease of notation, we suppress the dependence of quantities on time. If Prover i finds a claim of subproof u , then she can decide when to publish and share her claim after some time t . If she posts her claim immediately at $t = 0$, we say that she follows the **partial progress sharing (PPS)** policy. Otherwise, Prover follows a non-PPS policy. We assume that the duration for the validation process by the reviewing process is insignificant compared with the time used to generate any subproof. Thus, we assume that time "freezes" during any validation process. A subproof u is considered "solved" if a claim for it is validated. The game ends once every subproof up to the last one is solved. In the event where a prover publishes a set of consecutive subproofs, $A \subseteq \{0\} \cup [m_0]$, each of these subproofs is subjected to validation individually and simultaneously. When a prover i 's claim of subproof u gets invalidated, either the prover needs to produce a formalized proof or some other prover's claim for u needs to get validated before she can attempt another subproof.

In our model, we assume provers know the complete aptitude matrix but we will see that this assumption can be relaxed if separable-aptitudes (SA) model is assumed: A model is a **separable-aptitudes (SA) model** if the prover-subproof aptitudes decompose as $a_i(u) = a_i \cdot s_u$, where a_i is the *innate ability* of prover i , which is independent of subproofs and s_u is the *simplicity* of subproof u , which is independent of provers.

The aims of a game designer are:

1. To *ensure PPS* - for any open problem, choose the "right" rewards, stakes' and bounties' amounts such that provers are incentivized to publicly share any initial partial proof without any delay.
2. To *minimize the makespan* - ensuring the proof of open problems is completed in the least time.

It is not difficult to see that the first objective is a necessary precondition for the second. Thus, we focus on the analysis of the model to ensure PPS.

4.3 Analysis of the incentive mechanism

To ease our syntactic load, we analyze a core model, satisfying conditions **(I) - (VI)**, where the reviewing process has a maximum amount of challenge steps $L = 2$. Moreover, we assume the interactions within each debate of a subproof claim are limited to the Prover (she) who provides the claim and a single Reviewer (he). We first look at the analysis on a single edge in the subproof-network before moving on to the analysis on the whole subproof-network. For ease of notation, we suppress the dependence of quantities on time, T_i^u , and aptitude, $a_i(u)$, for all $i \in [n]$ and $u \in \{0\} \cup [m_0]$. The main results of our analysis are Corollary 4.3.4 for Proof completion game with multiple sequential subproofs, and Corollary 4.3.8, together with Corollary 4.3.7, for Proof completion game with multiple parallel subproofs. They state that PPS is the best strategy for any prover after scaling of some parameter sets of the subproofs.

4.3.1 Analysis on string diagram of single subproof

We will first look at the case where the proof of a given conjecture cannot be broken down into smaller subproofs. Thus, any prover would immediately post and share her work once she finds a claim of the proof with high enough confidence.

We assume the debate starts with Prover i posting a claim of proof u of some mathematical statement. We denote $R_2^u, R_1^u, R_0^u \geq 0$ as the rewards Prover gets if her proof is validated at level 2, 1 and 0, respectively. Since there is a single Reviewer, we can let $\sigma_2^u = (\sigma_2^u)^\downarrow$ and $\sigma_1^u = (\sigma_1^u)^\uparrow + (\sigma_1^u)^\downarrow$, both potentially earned by the reviewer. Thus, σ_2^u, σ_1^u represents the stakes required by Prover i at challenge step 2 and 1, respectively. β_1^u, β_0^u represents the bounties required for questions asked by Reviewer at step 1 and 0, respectively. Figure 4.3 summarizes the payoffs between Prover i and Reviewer in this core model.

Lemma 4.3.1 (Proposition 22 of Carré et al., 2021²). *Consider subproof u in a single node string diagram (i.e., of the form illustrated in Figure 2.3a). The game possesses a unique perfect Bayesian equilibrium (PBE). Depending on the parameter set $\Theta^u := \{R_0^u, R_1^u, R_2^u, \sigma_1^u, \sigma_2^u, \beta_0^u, \beta_1^u\}$, this PBE takes one of three types detailed below. For all types, there is a threshold $\pi_*^u := \pi_*^u(\Theta^u) \in [0, 1)$ such that Prover i publishes if and only if $P_i^u \geq \pi_*^u$ and:*

- **Type 1:** Reviewer always challenges the initial proof, Prover replies if $X = 1$ and replies with probability $p^u := p^u(\Theta^u)$ if $X = 0$. Conditional on reply, Reviewer challenges w.p. $q_1^u := q_1^u(\Theta^u)$ (and Prover successfully terminates the process if and only if $X = 1$).

²Proof can be found in the original paper.

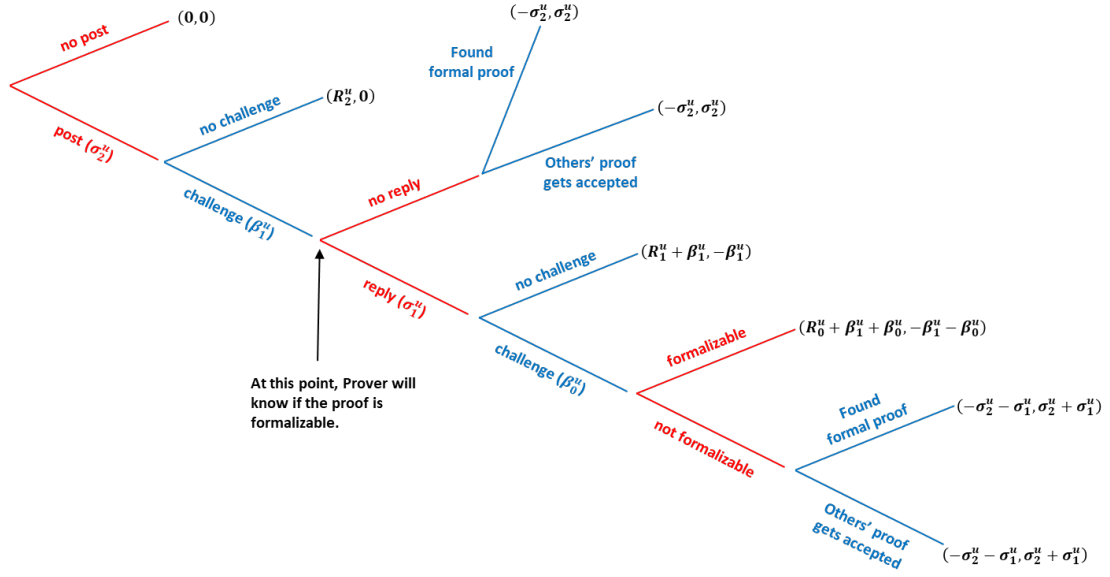


FIGURE 4.3: Reviewing model with $L = 2$. $(U_{Prover}, U_{Reviewer})$ represents the payoffs of Prover and Reviewer respectively at each outcome of the game.

- **Type 2:** Reviewer challenges the initial proof with probability $q_2^u := q_2^u(\Theta^u) \in (0, 1)$. Then actions unfold as in Type 1.
- **Type 3:** $\pi_*^u = 0$ and Reviewer never challenges.

The following are the necessary conditions in terms of Θ^u for each type of PBE to occur:

Type 1 PBE

$$\begin{aligned} \pi_1^u &> \pi_e^u \\ \phi^u(\pi_e^u) &> 0 \\ \pi_*^u &= \frac{\sigma_2^u}{q_1^u(R_0^u + \beta_1^u + \beta_0^u) + (1 - q_1^u)(R_1^u + \beta_1^u) + \sigma_2^u} \end{aligned} \quad (4.1)$$

Type 2 PBE

$$\begin{aligned} \pi_1^u &> \pi_e^u \\ \phi^u(0) &\geq 0 \\ q_2^u &= \frac{R_2^u}{R_2^u - \pi_*^u(q_1^u(R_0^u + \beta_1^u + \beta_0^u) + (1 - q_1^u)(R_1^u + \beta_1^u)) + (1 - \pi_*^u)\sigma_2^u} \end{aligned} \quad (4.2)$$

with π_*^u from equation set 4.1 and $0 < q_2^u < 1$

Type 3 PBE

$$\begin{aligned}\pi_*^u &= 0 \\ \phi^u(0) &< 0\end{aligned}\tag{4.3}$$

where

$$\begin{aligned}\pi_1^u &= \frac{\sigma_2^u + \sigma_1^u + \beta_1^u}{\sigma_2^u + \sigma_1^u + \beta_1^u + \beta_0^u} \\ \pi_e^u &= \frac{1}{2}(1 + \pi_*^u) \\ \phi^u(\pi_e^u) &= \pi_e^u(-q_1^u(\beta_1^u + \beta_0^u) - (1 - q_1^u)\beta_1^u) + (1 - \pi_e^u)((1 - p^u)\sigma_2^u - p^u\beta_1^u) \\ p^u &= \frac{\pi_e^u(1 - \pi_1^u)}{\pi_1^u(1 - \pi_e^u)} \\ q_1^u &= \frac{R_1^u + \sigma_2^u + \beta_1^u}{R_1^u + \sigma_2^u + \beta_1^u + \sigma_1^u}\end{aligned}\tag{4.4}$$

In layman's term, Lemma 4.3.1 tells us that by setting the "right" values in the parameter set Θ^u , Type 1, 2 or 3 PBE would hold for each subproof u in a Proof-completion game on a general subproof-network. One can think of π_*^u as the minimum confidence level the game designer wants the prover to have, p_u as the probability that she wants the prover to respond at round 1 and q_2^u, q_1^u as the probabilities that she wants the reviewer to ask questions after round 2 and 1 respectively.

Moreover, from the payoff tree shown in Figure 4.3, suppose $P_i^u \geq \pi_*^u$, Prover i would post her proof and her expected payoff, $V_x(\Theta^u, P_i^u)$, (Equation 4.5) can be derived based on the type of PBE, x , the model is in as function of parameter set Θ^u and P_i^u . Her expected payoffs under different types of PBE will be essential in analyzing the case for multiple-subproofs case. Note that Prover's expected payoff does not change even if the first attempt of her claim gets invalidated as her subsequent attempt does not require any stake or can receive any reward, which is stated in condition (III).

$$\begin{aligned}V_1(\Theta^u, P_i^u) &= -\sigma_2^u(1 - P_i^u)(1 - p^u) + (R_1^u + \beta_1^u)(P_i^u + (1 - P_i^u)p^u)(1 - q_1^u) \\ &\quad + (R_0^u + \beta_0^u + \beta_1^u)P_i^u q_1^u + (-\sigma_1^u - \sigma_2^u)(1 - P_i^u)p^u q_1^u \\ V_2(\Theta^u, P_i^u) &= R_2^u(1 - q_2^u) + V_x(\Theta^u, P_i^u)q_2^u \\ V_3(\Theta^u, P_i^u) &= R_2^u\end{aligned}\tag{4.5}$$

4.3.2 Analysis on string diagram of multiple sequential subproofs

With the expected payoff functions in Equation 4.5, in addition to the conditions from Lemma 4.3.1, we can derive conditions such that PPS is the best strategy for Prover i in multiple sequential subproof case.

Theorem 4.3.2. *Consider the Proof-completion game on a string diagram of multiple sequential subproofs as shown in Figure 2.4a with the incentive reviewing process described in Section 4.2.2 in place. Suppose for each subproof $u \in \{0\} \cup [m_0]$, conditions on Θ^u in Lemma 4.3.1 is*

satisfied such that prover i only posts if and only if $P_i^u \geq \pi_*^u$ and Type x PBE for $x \in \{1, 2, 3\}$ is achieved. In addition, suppose the following condition hold:

- Each Prover i that participates in the game has her own fixed confidence level, $P_i = P_i^u$ for all $u \in \{0\} \cup [m_0]$

For any pair of subproof k, l such that k precedes l , if the parameter sets Θ^k, Θ^l satisfies:

$$V_x(\Theta^k, P_i) \geq c_{i,k,l} V_x(\Theta^l, P_i)$$

where $c_{i,k,l} = \frac{a_i(l)a_{-i}(l)}{a(l)a_{-i}(k)}$, $V_x(*, *)$ for $x \in \{1, 2, 3\}$ is the expected payoff function defined in equation set 4.5 and for any subproof u , we define $a_{-i}(u) := \sum_{j \neq i} a_j(u)$, $a(u) = a_i(u) + a_{-i}(u)$, then every prover follows PPS policy .

Proof. We will be numbering subproofs with the first subproof denoted as 0 and the last as m_0 as shown in Figure 2.4a. We will also fix prover $i \in [n]$ and assume other provers follow PPS. $G_{k,l}^i, 0 \leq k \leq l \leq m_0$ is defined as subgame where prover i has found claims for subproofs u with high enough confidence (i.e., $P_i \geq \pi_*^u$) for $u \in \{k, k+1, \dots, l-1\}$ (refer this set of subproofs as *captive subproofs* of prover i) and is finding claim for subproof l , while subproofs $0, 1, \dots, k-1$ are all solved and all other provers are finding the claim of subproof k . Note that prover i starts either on par or ahead of others as they follow PPS. When $k < l$, prover i has not shared the claims for her captive subproofs, otherwise, the others would not be stuck at task k . We define a new ordering $\preceq$ for subgames is defined as $G_{k,l}^i \preceq G_{p,q}^i$ if $k > p$ OR $k = p$ and $l > q$.

First, assuming the invariant (i.e., PPS is the best strategy for every prover) is true and Type x PBE for $x \in \{1, 2, 3\}$ to hold for every subproof u , the expected reward earned by prover i in subgame $G_{k,l}^i$ can be derived and is given by:

$$R(G_{k,l}^i) = \sum_{u=k}^{l-1} V_x(\Theta^u, P_i^u) + \sum_{u=l}^{m_0} \frac{a_i(u)}{a(u)} \cdot V_x(\Theta^u, \pi_e^u)$$

where $\frac{a_i(u)}{a(u)}$ is the probability of the event when prover i finds a claim of subproof u faster than other provers and with confidence $P_i \geq \pi_*^u$ before subproof u is solved by any other prover, which is derived due to a standard property of the minimum of independent exponential random variables.

Next, we will prove the invariant by backward induction on the ordering $\preceq$.

Base case: Consider subgame G_{k,m_0+1}^i for any $k \in \{0\} \cup [m_0]$, which is defined as the game where prover i finds the claims of all subproofs $k, k+1, \dots, m_0$ with high enough confidence while the rest of the provers $j \neq i$ are still working on subproof k . It is obvious that PPS ($t = 0$) is the best strategy.

Inductive case: We will split the remaining subgames into two cases: $k < l$ and $k = l$.

Case I. ($G_{k,l}^i, 0 \leq k < l$): Assume game $G_{k,l}^i$ starts at $t = 0$ and the invariant holds for all smaller subgames.

Subcase PPS. Suppose prover i follows PPS (i.e., shares claims for his captive subproofs). Even if there are some subproofs in his captive subproofs gets invalidated, this will reduce $G_{k,l}^i$ to $G_{l,l}^i$ eventually. To see why this is true, assume there are $0 \leq h \leq l-k$ subproofs in his captive subproofs gets invalidated. Let subproofs $u_1, u_2, \dots, u_h$ be the captive subproofs that are invalidated and $u_x < u_y$ if $x < y$. As stated in condition (I), any subproof u will only be available to solve if all preceding subproof/s $v < u$ is/are solved. Thus, all provers can only work on u_1 before u_2 . After u_1 is solved, with the same reasoning, u_2 must be solved before u_3 becomes available for provers and so on. Thus, $u_1, u_2, \dots, u_h$ need to be solved by the provers before moving on to subproof k . For each of these invalidated captive subproofs, if prover i finds a formalized proof before other agents solve it, it is indifferent for him to follow or defect from PPS as his expected payoff for this second attempt will be 0 in either case as stated in condition (III). As a result, we can assume that he follows PPS for $u_1, u_2, \dots, u_h$. Once all these invalidated captive subproofs are eventually solved by either prover i 's formalized proof/s or by other provers, the game will be reduced to $G_{l,l}^i$. By inductive hypothesis, the invariant holds for all subgames.

Subcase Non-PPS. On the other hand, suppose prover i chooses to defect from PPS until the subgame $G_{k,l}$ changes its state. Let τ_1 be the first time when either prover i finds a claim of subproof l with high enough confidence or some agent $j \neq i$ solves subproof k . From here, it is sufficient to consider the deviations from PPS of the sort: prover i chooses a fixed delay $\tau > 0$, and publicly shares and posts the claims to captive subproofs at time $t = \min\{\tau_1, \tau\}$ from the following argument (*):

- *Scenario I.* Suppose $\tau_1 < \tau$ and prover i finds a claim of subproof l with high enough confidence before other prover $j \neq i$ solves a claim of subproof k . This reduces the game to $G_{k,l+1}$ and by induction hypothesis, the invariant holds for all smaller games.
- *Scenario II.* Suppose $\tau_1 < \tau$ and some prover $j \neq i$ solves subproof k before i finds a claim of subproof l . This reduces the game to $G_{k+1,l}^i$ and by induction hypothesis, the invariant holds for all smaller games.
- *Scenario III.* Suppose $\tau_1 < \tau$. This will be the same scenario as *Subcase PPS* where the subgame will eventually reduce to $G_{l,l}^i$.
- Any randomized choice of τ can be written as a linear combination of these scenarios.

So it suffices for us to show that $\tau = 0$ is the best strategy for i in the induction step for $G_{k,l}^i$. Let $U_{k,l}^i(\tau)$ be the expected reward of agent i if she chooses delay τ . So for $\tau = 0$, we have:

$$\begin{aligned}
 U_{k,l}^i(0) &= R(G_{k,l}^i) \\
 &= \sum_{u=k}^{l-1} V_x(\Theta^u, P_i) + \sum_{u=l}^{m_0} \frac{a_i(u)}{a(u)} \cdot V_x(\Theta^u, P_i) \\
 &= V_x(\Theta^k, P_i) + \Delta + \frac{a_i(l)}{a(l)} \cdot V_x(\Theta^l, P_i) + R(G_{l+1,l+1})
 \end{aligned}$$

where $\Delta = \sum_{u=k+1}^{l-1} V_x(\Theta^u, P_i)$ and $R(G_{l+1,l+1})$ is the expected reward earned by agent i in the sub-game $G_{l+1,l+1}$ assuming the induction hypothesis.

Let $a(i, k, l) = a_i(l) + a_{-i}(k)$ and hence, $\frac{a_i(l)}{a(i, k, l)}$ represents the probability that prover i finds a claim of subproof l with high enough confidence before other prover $j \neq i$ solves subproof k , which is again derived from the standard property of the minimum of independent exponential random variables. For any $\tau > 0$, the following can be derived from the argument (*):

$$\begin{aligned}
U_{k,l}^i(\tau) &= \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_i(l)}{a(i, k, l)} \cdot \left(\sum_{u=k}^{l-1} V_x(\Theta^u, P_i) + V_x(\Theta^l, P_i) + R(G_{l+1,l+1}) \right) \\
&\quad + \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_{-i}(k)}{a(i, k, l)} \cdot R(G_{k+1,l}^i) \\
&\quad + \mathbb{P}[\tau_1 \geq \tau] \cdot R(G_{k,l}^i) \\
&= \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_i(l)}{a(i, k, l)} \cdot \left(V_x(\Theta^k, P_i) + \Delta + V_x(\Theta^l, P_i) + R(G_{l+1,l+1}) \right) \\
&\quad + \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_{-i}(k)}{a(i, k, l)} \cdot \left(\Delta + \frac{a_i(l)}{a(l)} \cdot V_x(\Theta^l, P_i) + R(G_{l+1,l+1}) \right) \\
&\quad + \mathbb{P}[\tau_1 \geq \tau] \cdot \left(V_x(\Theta^k, P_i) + \Delta + \frac{a_i(l)}{a(l)} \cdot V_x(\Theta^l, P_i) + R(G_{l+1,l+1}) \right) \\
&= V_x(\Theta^k, P_i) \left(1 - \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_{-i}(k)}{a(i, k, l)} \right) + \Delta \\
&\quad + V_x(\Theta^l, P_i) \left(\frac{a_i(l)}{a(l)} + \mathbb{P}[\tau_1 < \tau] \cdot \frac{a_i(l)}{a(i, k, l)} \cdot \frac{a_{-i}(l)}{a(l)} \right) + R(G_{l+1,l+1})
\end{aligned}$$

Subtracting the last equality from the expression for $U_{k,l}^i(0)$, we will have:

$$U_{k,l}^i(0) - U_{k,l}^i(\tau) = V_x(\Theta^k, P_i) \left(\mathbb{P}[\tau_1 < \tau] \cdot \frac{a_{-i}(k)}{a(i, k, l)} \right) - V_x(\Theta^l, P_i) \left(\mathbb{P}[\tau_1 < \tau] \cdot \frac{a_i(l)}{a(i, k, l)} \cdot \frac{a_{-i}(l)}{a(l)} \right)$$

Thus, if Θ^k, Θ^l , where k precedes l , are set up such that $V_x(\Theta^k, P_i) \geq \frac{a_i(l)a_{-i}(l)}{a(l)a_{-i}(k)} V_x(\Theta^l, P_i)$, PPS is the best strategy for agent i , i.e.,:

$$\begin{aligned}
V_x(\Theta^k, P_i) &\geq \frac{a_i(l)a_{-i}(l)}{a(l)a_{-i}(k)} V_x(\Theta^l, P_i) \\
\implies V_x(\Theta^k, P_i) (a_{-i}(k)) &\geq \frac{a_i(l)a_{-i}(l)}{a(l)} V_x(\Theta^l, P_i) \\
\implies V_x(\Theta^k, P_i) (a_{-i}(k)) - \frac{a_i(l)a_{-i}(l)}{a(l)} V_x(\Theta^l, P_i) &\geq 0 \\
\implies V_x(\Theta^k, P_i) \left(\mathbb{P}[\tau_1 < \tau] \cdot \frac{a_{-i}(k)}{a(i, k, l)} \right) - V_x(\Theta^l, P_i) \left(\mathbb{P}[\tau_1 < \tau] \cdot \frac{a_i(l)}{a(i, k, l)} \cdot \frac{a_{-i}(l)}{a(l)} \right) &\geq 0 \\
\implies U_{k,l}^i(0) - U_{k,l}^i(\tau) &\geq 0 \\
\implies U_{k,l}^i(0) &\geq U_{k,l}^i(\tau)
\end{aligned}$$

Case II. ($G_{l,l}^i, l \leq m_0$): Assume the invariant holds for all smaller games. If some agent $j \neq i$ solves subtask l first, this reduces the game to $G_{l+1,l+1}^i$. On the other hand, if agent i finds a claim of subtask l with high enough confidence first, this reduces the game to $G_{l,l+1}^i$. In both of these smaller subgames, PPS is the best response for prover i due to our inductive hypothesis. Hence, this completes our induction. $\square$

We can derive a few consequences from Theorem 4.3.2. First, suppose Type 3 PBE holds where $\pi_*^u = 0$ for any subproof u , this narrows down to the main result for linear subtask-networks in Banerjee, Goel, and Kollagunta Krishnaswamy, 2014.

Corollary 4.3.3 (Theorem 2.1 of Banerjee, Goel, and Kollagunta Krishnaswamy, 2014). *Consider the Proof-completion game on a string diagram of multiple sequential subproofs as shown in Figure 2.4a with the incentive reviewing process described in Section 4.2.2 in place. Suppose for each subproof $u \in \{0\} \cup [m_0]$, conditions on Θ^u in Lemma 4.3.1 is satisfied such that Type 3 PBE is achieved (i.e., $\pi_*^u = 0$ for any subproof u). For any pair of subproof k, l such that k precedes l , if the parameter sets Θ^k, Θ^l satisfies:*

$$\frac{R_2^k a_{-i}(k)}{R_2^l a_{-i}(l)} \geq \frac{a_i(l)}{a(l)}$$

where for any subproof u , we define $a_{-i}(u) := \sum_{j \neq i} a_j(u)$, $a(u) = a_i(u) + a_{-i}(u)$, then PPS policy is a PBE for prover i .

Proof. The inequality can be derived immediately when we substitute $V_x(\Theta^u, *)$ with R_2^u and $\pi_*^u = 0$ for subproof $u \in \{k, l\}$ in the inequality in Theorem 4.3.2. $\square$

Second, Theorem 4.3.2 provide insights to a game designer on how stakes, bounties and rewards should be assigned to incentivize provers to share their partial progresses immediately and for reviewers to review the claims in a Proof completion game represented by a string diagram with multiple sequential subproofs. However, the theorem assume provers know the complete aptitude matrix, i.e., prover i knows every other provers aptitude for each subproof, $a_j(u)$ for $j \neq i$, which is not the case most of the time. Moreover, there are many possible ways for the game designer to set the parameter sets such that the inequality condition in the theorem holds with respect to the provers' expected payoffs, $V_x(*, *)$. As such, the following corollary shows that by scaling the parameter set Θ^l by some factor of Θ^k , where subproof k precedes subproof l and assuming SA model, we can provide a simpler condition for PPS to be a PBE. Note that we identify Θ_u as a vector, i.e., $(R_0^u, R_1^u, R_2^u, \sigma_1^u, \sigma_2^u, \beta_0^u, \beta_1^u)$ instead of a set. For any subproof u, v , we use the notation $\Theta_u \leq \Theta_v$ if and only if $R_0^u \leq R_0^v, \dots, \beta_1^u \leq \beta_1^v$ and $a\Theta_u = (aR_0^u, aR_1^u, aR_2^u, a\sigma_1^u, a\sigma_2^u, a\beta_0^u, a\beta_1^u)$ for any constant a .

Corollary 4.3.4. *Consider the Proof-completion game on a string diagram of multiple sequential subproofs as shown in Figure 2.4a with the incentive reviewing process described in Section 4.2.2 in place. Suppose for each subproof $u \in \{0\} \cup [m_0]$, conditions on Θ^u in Lemma 4.3.1 is satisfied such that prover i only posts if and only if $P_i^u \geq \pi_*$, for some fixed $\pi_* := \pi_*^u$ for all $u \in [m_0]$, and Type x PBE for $x \in \{1, 2, 3\}$ is achieved. In addition, suppose the following condition hold:*

- Each Prover i that participates in the game has her own fixed confidence level, $P_i = P_i^u$ for all $u \in \{0\} \cup [m_0]$
- SA model is assumed

For any pair of subproof k, l such that k precedes l , if the parameter sets Θ^k, Θ^l satisfies:

$$\Theta^k \geq \alpha \cdot s_{l,k} \cdot \Theta^l$$

where $\alpha = \max_{i \in [n]} \frac{a_i}{\sum_{j \in [n]} a_j}$, which represents the best prover innate ability as compared to all provers and $s_{l,k} = \frac{s_l}{s_k}$, which represents the relative simplicity of subproof l as compared to subproof k , then every prover follows PPS policy.

Proof. For any subproof u, v , $\Theta^u = d \cdot \Theta^v$ where $d \geq 0$ can be any constant, if Θ^v satisfies Type x PBE for $x \in \{1, 2, 3\}$ then so would Θ^u . To see why this is true, notice that once the parameters in Θ^v are assigned, π_1^u and q_1^u from equation set 4.4 are invariant under multiplication of d on Θ^v , i.e., $\pi_1^u = \pi_1^v$ and $q_1^u = q_1^v$. As a result, $\pi_*^u, q_2^u, \pi_e^u, p^u$ from equation sets 4.1, 4.2, 4.3 and 4.4 respectively are invariant under the multiplication of d on Θ^v as well. Thus, we can assume $\pi_* := \pi_*^w, \pi_e := \pi_e^w, \pi_1 := \pi_1^w, q_1 := q_1^w, q_2 := q_2^w, p := p^w$ for any subproof w . The only variable that scales with d is $\phi^u(\pi_e)$, i.e., $\phi^u(\pi_e) = d\phi^v(\pi_e)$. However, in the case of Type 1 PBE, if $\phi_v(\pi_e) \geq 0$, then $\phi^u(\pi_e) = d\phi^v(\pi_e) \geq 0$ as well. Similarly for Type 2 and 3 PBE, if $\phi_v(0) \geq 0$ or $\phi_v(0) < 0$, then $\phi^u(0) = d\phi^v(0) \geq 0$ or $\phi^u(0) = d\phi^v(0) < 0$ as well respectively. Thus, by checking the conditions in the equation sets 4.1, 4.2 and 4.3, we can see that if Θ^v satisfies Type x PBE for $x \in \{1, 2, 3\}$ then so would Θ^u with some fixed $\pi_* := \pi_*^u = \pi_*^v$. Furthermore, we can also derive that for any $x \in \{1, 2, 3\}$ and some fixed P_i that is greater than or equal to π_*^u and π_*^v , $V_x(\Theta^u, P_i) = V_x(d \cdot \Theta^v, P_i) = d \cdot V_x(\Theta^v, P_i)$ as p, q_1 and q_2 are invariant under multiplication of d on Θ^v .

Therefore, given that the conditions in Corollary 4.3.4 holds and $\Theta^k = d \cdot \Theta^l$ where $d = \alpha \cdot s_{l,k}$, then we have:

$$\begin{aligned} V_x(\Theta^k, P_i) &= V_x(d \cdot \Theta^l, P_i) \\ &= d \cdot V_x(\Theta^l, P_i) \\ &\geq \alpha \cdot s_{l,k} \cdot V_x(\Theta^l, P_i) \\ &\geq \frac{a_i \cdot s_l}{(\sum_{j \in [n]} a_j) \cdot s_k} \cdot V_x(\Theta^l, P_i) \\ &= \frac{(a_i \cdot s_l)((\sum_{j \neq i \in [n]} a_j) \cdot s_l)}{((\sum_{j \in [n]} a_j) \cdot s_l)((\sum_{j \neq i \in [n]} a_j) \cdot s_k)} \cdot V_x(\Theta^l, P_i) \\ &= \frac{a_i(l)a_{-i}(l)}{a(l)a_{-i}(k)} V_x(\Theta^l, P_i) \end{aligned}$$

By Theorem 4.3.2, PPS policy is a PBE for prover i . □

4.3.3 Analysis on string diagram of multiple parallel subproofs

We will next look at the case where the partial proof is of the form shown in Figure 2.4b, where it is represented by a string diagram of multiple parallel subproofs. For

our analysis, we denote the whole string diagram as shown in Figure 2.4b as G and a subdiagram which comprises of only the parallel subproofs $1, 2, \dots, m_0$ as G' as shown in Figure 4.4.

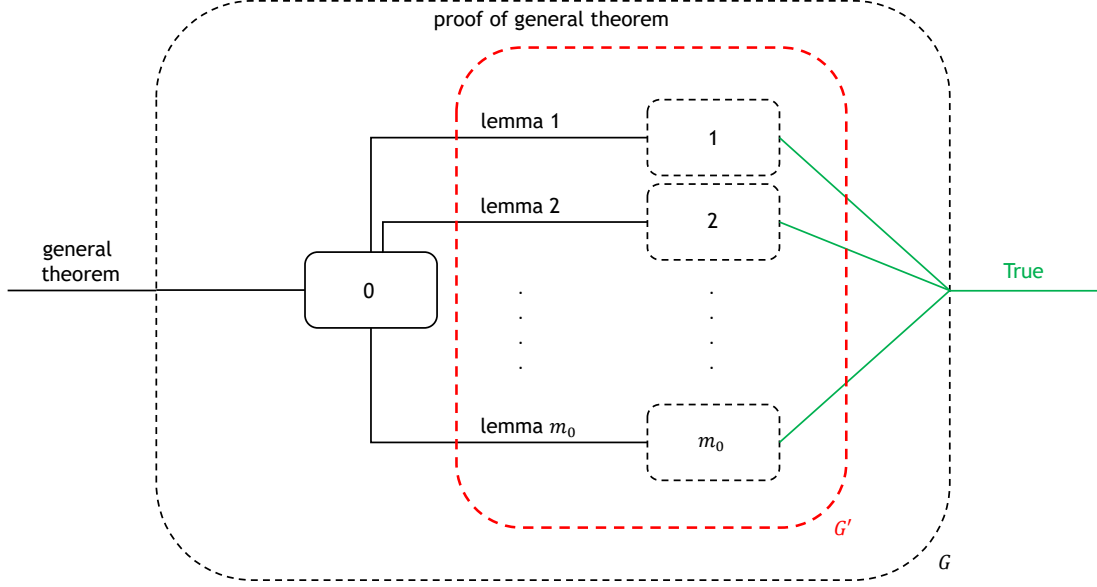


FIGURE 4.4: String diagram G comprises of all subproofs $0, 1, \dots, m_0$ while G' comprises of only the parallel subproofs $1, 2, \dots, m_0$

Theorem 4.3.5. Consider the Proof-completion game on a string diagram G' . Suppose for each subproof u , conditions on Θ^u in Lemma 4.3.1 is satisfied such that any Prover $i \in [n]$ only posts if and only if $P_i^u \geq \pi_*^u$, i.e., high enough confidence, and Type x PBE for $x \in \{1, 2, 3\}$ is achieved. In addition, suppose the following conditions hold:

1. Each Prover i that participates in the game has her own fixed confidence level, $P_i = P_i^u$ for all $u \in [m_0]$,
2. Parameter sets in $\{\Theta^w\}_{w \in [m_0]}$ satisfy the following monotonicity condition: there is a total ordering $\prec$ on the set $[m_0]$ such that for every Prover i and pair of subproofs u, v , we have $v \prec u \iff \gamma_i(v) < \gamma_i(u)$ where we define $\gamma_i(w)$ as the expected virtual reward of subproof w and

$$\gamma_i(w) := \frac{a_i(w)a_{-i}(w)}{a(w)} V_x(\Theta^w, P_i),$$

where $V_x(*, *)$ for $x \in \{1, 2, 3\}$ is the expected payoff function defined in equation set 4.5 and for any subproof w , we define $a_{-k}(w) := \sum_{j \neq k} a_k(w)$, $a(w) = a_k(w) + a_{-k}(w)$. Then the following strategies together constitute a PBE:

- Every Prover follows PPS policy
- Suppose $M_0 \subseteq [m_0]$ is the set of available subproofs. Every Prover i chooses to work on the unique subproof $u^* = \arg \max_{u \in M_0} \gamma_i(u)$, i.e., herding.

Proof. Fix a Prover $i \in [n]$ and assume all other provers $j \neq i$ follow PPS policy. Let $S, T \subseteq [m_0]$ be the set of available subproofs for other provers $j \neq i$ and Prover i has yet to find the claims of respectively. We assume that i is ahead of or on par with any j , and thus, $T \subseteq S$. S^c, T^c represent the set of subproofs j and i found claims to, respectively. We define $T^c \setminus S^c$ as Prover i 's captive subproofs, i.e., the additional subproofs she has ahead of the rest. We let $G_{S,T}$ represents the game where Prover i has yet to find the claims of subproofs in T while the rest are still working on subproofs in S . Note that as all other provers $j \neq i$ are following PPS policy, once a claim is found, these provers would have posted the claim immediately. We define an ordering $\prec$ such that $G_{S,T} \prec G_{S',T'}$ if and only if $|S| \leq |S'|$ with $|T| = |T'|$ or $|T| \leq |T'|$. We prove this theorem via backward induction.

Base case: Consider the game $G_{S,T}$ where $|T| = 0$. This means that Prover i has found claims of all subproofs. It is obvious that PPS ($t = 0$) is her best strategy.

Inductive case: Suppose the game starts at $G_{S,T}$. By induction hypothesis, Prover i follows PPS for games smaller than $G_{S,T}$. We consider two cases where: (i) $1 \leq |T| < |S| \leq m_0$ and (ii) $1 \leq |T| = |S| \leq m_0$.

Case (i). Consider $G_{S,T}$ with $1 \leq |T| < |S| \leq m_0$, i.e., other provers $j \neq i$ are working on some subproof $u^* \in T^c \setminus S^c$ (i.e., one of i 's captive subproofs) while i works on some $v \in T$. Note that $u^* \in T^c$, i.e., i has found the claim for u^* .

Subcase PPS. If Prover i follows PPS, her expected payoff is:

$$U_{S,T}^i = \sum_{u \in T^c \setminus S^c} V_x(\Theta^u, P_i) + \sum_{u \in T} \frac{a_i(u)}{a(u)} V_x(\Theta^u, P_i).$$

Note that when some subproofs in her captive subproofs are invalidated, $G_{S,T}$ is reduced to $G_{T,T}$ eventually. To see why this is true, we let $Z \subseteq T^c \setminus S^c$ represent the set of subproofs in i 's captive subproofs that are invalidated. As mentioned in the condition (III), we assume that for each of these invalidated captive subproofs, i needs to find a formalized proof for the subproof or some other prover's claim gets validated before eventually moving on to her remaining subproof set T . In either case, it is indifferent for him to follow or defect from PPS as her expected payoff for this second attempt is 0 in either case. Thus, we can assume i follows PPS for the subproof set Z . Once all these invalidated captive subproofs are eventually solved by either i 's formalized proof/s or by other provers, the game is reduced to $G_{T,T}$. By induction hypothesis, as $G_{T,T} \prec G_{S,T}$, all provers including i follow PPS + herding, and thus, $\sum_{u \in T} \frac{a_i(u)}{a(u)} V_x(\Theta^u, P_i)$ is derived.

Subcase Non-PPS. In the case where Prover i chooses to follow non-PPS policy, either Prover i finds some $v \in T$ first or some $j \neq i$ finds some u^* first. Similarly, in either case, Prover i chooses PPS later on by induction hypothesis. Thus, her expected reward is:

$$U_{S,T}^{i,dev} = \frac{a_{-i}(u^*)}{a(i, v, u^*)} (U_{S,T} - V_x(\Theta^{u^*}, P_i)) + \frac{a_i(v)}{a(i, v, u^*)} (U_{S,T} + \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i)),$$

where $a(i, v, u^*) = a_i(v) + a_{-i}(u^*)$, derived similarly by the standard property of the minimum of independent exponential random variables. We obtain the following:

$$\begin{aligned}
& U_{S,T}^i - U_{S,T}^{i,dev} \\
&= \frac{a_{-i}(u^*)}{a(i, v, u^*)} V_x(\Theta^{u^*}, P_i) - \frac{a_i(v)}{a(i, v, u^*)} \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) \\
&\geq \frac{a_{-i}(u^*)}{a(i, v, u^*)} \left(\frac{a_i(u^*)}{a(u^*)} \right) V_x(\Theta^{u^*}, P_i) - \frac{a_i(v)}{a(i, v, u^*)} \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) \\
&= \frac{\gamma_i(u^*) - \gamma_i(v)}{a(i, v, u^*)}
\end{aligned}$$

Since Prover i worked on and found the claim of u^* before v , by the monotonicity condition, all provers have the same ordering on expected virtual rewards and $\gamma_i(u^*) \geq \gamma_i(v)$. Hence, we have $U_{S,T}^i \geq U_{S,T}^{i,dev}$.

Case (ii). Consider $G_{S,T}$ with $1 \leq |T| = |S| \leq m_0$. In this case, all subproofs in T^c are solved and all provers are working on some subproofs in T . If Prover i chooses herding, i.e., works on $u^* := \arg \max_{u \in T} \gamma_i(u)$, her expected reward is:

$$U_{T,T}^i = \sum_{u \in T} \frac{a_i(u)}{a(u)} V_x(\Theta^u, P_i).$$

If she chooses to deviate to some other subproof v other than u^* , her expected reward becomes:

$$U_{T,T}^{i,dev} = \frac{a_{-i}(u^*)}{a(i, v, u^*)} (U_{T,T} - \frac{a_i(u^*)}{a(u^*)} V_x(\Theta^{u^*}, P_i)) + \frac{a_i(v)}{a(i, v, u^*)} (U_{T,T} + \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i))$$

Thus, we can derive:

$$\begin{aligned}
& U_{T,T}^i - U_{T,T}^{i,dev} \\
&= \frac{a_{-i}(u^*)}{a(i, v, u^*)} \frac{a_{-i}(u^*)}{a(u^*)} V_x(\Theta^{u^*}, P_i) - \frac{a_i(v)}{a(i, v, u^*)} \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) \\
&= \frac{\gamma_i(u^*) - \gamma_i(v)}{a(i, v, u^*)}
\end{aligned}$$

Since $u^* := \arg \max_{u \in T} \gamma_i(u)$, $\gamma_i(u^*) \geq \gamma_i(v)$. Thus, $U_{T,T}^i \geq U_{T,T}^{i,dev}$ and this completes the proof. $\square$

Having settled the behavior over the parallel nodes of G' shown in Figure 4.4, the last step in our understanding is to settle the behavior of the provers for the first node, subproof 0, to complete the analysis for string diagram G .

Theorem 4.3.6. *Consider the Proof-completion game on a string diagram G . Suppose all the conditions in Theorem 4.3.5 hold, then in the scenario where some prover i has found the claim of first subproof 0 while the other provers $j \neq i$ did not find any claim yet, i follows PPS, and hence, posts the claim of subproof 0 immediately if the following condition holds: for any pair of subproofs 0, w where $w \in [m_0]$, Θ^0, Θ^w satisfy:*

$$V_x(\Theta^0, P_i) \geq \max_{w \in [m_0]} c_{i,w,0} V_x(\Theta^w, P_i)$$

$$\text{where } c_{i,w,0} = \frac{a_i(w)a_{-i}(w)}{a(w)a_{-i}(0)}.$$

Proof. Let $S = [m_0] \cup \{0\}$ and $T \subseteq [m_0]$ be the sets of available subproofs for other provers $j \neq i$ and Prover i has yet to find the claims to, respectively. $S^c, T^c, G_{S,T}$ are defined as in the proof of Theorem 4.3.5. However, for the ordering $\prec$, we define $G_{S,T} \preceq G_{S,T'}$ if and only if $|T| \leq |T'|$, as S is fixed to be $[m_0] \cup \{0\}$. We again prove the theorem with backward induction.

Base case: Consider the game $G_{S,T}$ where $|T| = 0$. Prover i has found claims to all subproofs. It is obvious that PPS ($t = 0$) is her best strategy.

Inductive case: Suppose the game starts at $G_{S,T}$.

Subcase PPS. If Prover i follows PPS, her expected payoff is:

$$U_{S,T}^i = V_x(\Theta^0, P_i) + \sum_{u \in T^c \setminus S^c} V_x(\Theta^u, P_i) + \sum_{u \in T} \frac{a_i(u)}{a(u)} V_x(\Theta^u, P_i).$$

Note that after Prover i follows PPS, the game is reduced from $G_{S,T}$ to $G_{T,T}$ with similar reasoning as seen in the proofs of Theorem 4.3.2 and 4.3.5. By Theorem 4.3.5, every prover follows PPS + herding, and hence, $\sum_{u \in T} \frac{a_i(u)}{a(u)} V_x(\Theta^u, P_i)$ is derived.

Subcase Non-PPS. Suppose Prover i chooses to deviate from PPS and work on some $v \in T$ instead of posting all her claims in $T^c \cup \{0\}$. Note that either Prover i finds some $v \in T$ before some prover $j \neq i$ finds 0 or vice versa. The game is reduced to $G_{S,T \setminus \{v\}}$ or $G_{S \setminus \{0\}, T}$, respectively. In the prior case, Prover i chooses PPS by the induction hypothesis. In the latter case, Prover i chooses PPS by Theorem 4.3.5. Thus, the expected reward for Prover i is:

$$U_{S,T}^{i,dev} = \frac{a_{-i}(0)}{a(i,v,0)} (U_{S,T} - V_x(\Theta^0, P_i)) + \frac{a_i(v)}{a(i,v,0)} \left(U_{S,T} + \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) \right).$$

where $a(i,v,0) = a_i(v) + a_{-i}(0)$, derived similarly by the standard property of the minimum of independent exponential random variables.

Subtracting from $U_{S,T}^i$:

$$U_{S,T}^i - U_{S,T}^{i,dev} = \frac{a_{-i}(0)}{a(i,v,0)} V_x(\Theta^0, P_i) - \frac{a_i(v)}{a(i,v,0)} \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i)$$

To ensure PPS, we need $U_{S,T}^i - U_{S,T}^{i,dev} \geq 0$. Thus, Prover i follows PPS when:

$$\begin{aligned}
V_x(\Theta^0, P_i) &\geq \max_{w \in [m_0]} \frac{a_i(w)a_{-i}(w)}{a(w)a_{-i}(0)} V_x(\Theta^w, P_i) \\
\implies V_x(\Theta^0, P_i) &\geq \frac{a_i(v)a_{-i}(v)}{a(v)a_{-i}(0)} V_x(\Theta^v, P_i) \\
\implies a_{-i}(0)V_x(\Theta^0, P_i) &\geq \frac{a_i(v)a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) \\
\implies a_{-i}(0)V_x(\Theta^0, P_i) - \frac{a_i(v)a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) &\geq 0 \\
\implies \frac{a_{-i}(0)}{a(i, v, 0)} V_x(\Theta^0, P_i) - \frac{a_i(v)}{a(i, v, 0)} \frac{a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) &\geq 0 \\
\implies U_{S,T}^i - U_{S,T}^{i,dev} &\geq 0
\end{aligned}$$

In our analysis, T is chosen arbitrarily, and thus, the inequality above must hold for all $T \subseteq [m_0]$, in particular when $T = [m_0]$. Thus, if the inequality condition in Theorem 4.3.6 holds, i posts subproof 0 immediately once she has found the claim of it. $\square$

Similar to the case for multiple sequential subproofs, combination of Theorem 4.3.5 and 4.3.6 provide conditions for a game designer to ensure partial progress sharing and for reviewers to review the claims in a Proof completion game represented by a string diagram with multiple parallel subproofs. However, both theorems also assume provers know the complete aptitude matrix and there are many ways for the parameter sets to satisfy the inequality conditions in the theorems with respect to the provers' expected payoffs, $V_x(*, *)$. Through similar approach as done in the case for multiple sequential subproofs, the following corollaries provide simpler conditions for PPS to be a PBE in the case for multiple parallel subproofs.

Corollary 4.3.7 (Corollary of Theorem 4.3.5). *Consider the Proof-completion game on a string diagram G' . Suppose for each subproof u , conditions on Θ^u in Lemma 4.3.1 are satisfied such that any prover $i \in [n]$ only publishes if and only if $P_i^u \geq \pi_*$, for some fixed $\pi_* := \pi_*^u$ for all $u \in [m_0]$, and Type x PBE for $x \in \{1, 2, 3\}$ is achieved. In addition, suppose the following conditions hold:*

1. *Each prover i that participates in the game has her own fixed confidence level, $P_i = P_i^u$ for all $u \in [m_0]$, and*
- 2*. *Parameter sets in $\{\Theta^w\}_{w \in [m_0]}$ satisfy the following monotonicity condition: there is a total ordering $\prec$ on the set $[m_0]$ such that for every prover i and pair of subproofs u, v , we have $v \prec u \iff s_v \cdot \Theta^v < s_u \cdot \Theta^u$*
3. *SA model is assumed*

where $s_w :=$ the simplicity subproof $w \in [m_0]$. Then the following strategies together constitute a PBE:

- Every prover follows PPS policy

- Suppose $M_0 \subseteq [m_0]$ is the set of available subproofs. Every prover i chooses to work on the unique subproof $u^* = \arg \max_{u \in M_0} s_u \cdot \Theta^u$, i.e., herding.

Proof. Consider condition 2 in Theorem 4.3.5 under SA model, for every prover i and subproofs u, v such that $v \prec u$, we have:

$$\begin{aligned}
& \gamma_i(v) < \gamma_i(u) \\
\iff & \frac{a_i(v)a_{-i}(v)}{a(v)} V_x(\Theta^v, P_i) < \frac{a_i(u)a_{-i}(u)}{a(u)} V_x(\Theta^u, P_i) \\
\iff & \frac{a_i \cdot s_v \cdot (\sum_{j \neq i} a_j) \cdot s_v}{(\sum_{j \in [n]} a_j) \cdot s_v} V_x(\Theta^v, P_i) < \frac{a_i \cdot s_u \cdot (\sum_{j \neq i} a_j) \cdot s_u}{(\sum_{j \in [n]} a_j) \cdot s_u} V_x(\Theta^u, P_i) \\
\iff & s_v \cdot V_x(\Theta^v, P_i) < s_u \cdot V_x(\Theta^u, P_i)
\end{aligned}$$

Under the condition 2*, we have:

$$\begin{aligned}
& s_v \cdot \Theta^v < s_u \cdot \Theta^u \\
\iff & \Theta^v < \frac{s_u}{s_v} \cdot \Theta^u \\
\implies & V_x(\Theta^v, P_i) < \frac{s_u}{s_v} \cdot V_x(\Theta^u, P_i), \forall i \in [n] \\
& \text{(same reasoning as seen in the proof of Corollary 4.3.4)} \\
\iff & s_v \cdot V_x(\Theta^v, P_i) < s_u \cdot V_x(\Theta^u, P_i), \forall i \in [n]
\end{aligned}$$

Thus, by Theorem 4.3.5, this completes the proof. $\square$

Corollary 4.3.8 (Corollary of Theorem 4.3.6). *Consider the Proof-completion game on a string diagram G . Suppose all the conditions in Corollary 4.3.7 hold. Then, when some Prover i has found the claim of first subproof 0 while any other prover $j \neq i$ has not found any claim yet, Prover i follows PPS and publishes the claim of subproof 0 immediately if the following condition holds: for any pair of subproofs $0, w$ where $w \in [m_0]$, Θ^0, Θ^w satisfy:*

$$\Theta^0 \geq \max_{w \in [m_0]} \alpha \cdot s_{w,0} \cdot \Theta^w$$

where $\alpha = \max_{i \in [n]} \frac{a_i}{\sum_{j \in [n]} a_j}$, which represents the best prover innate ability as compared to all provers and $s_{w,0} = \frac{s_w}{s_0}$, which represents the relative simplicity of subproof w as compared to subproof 0.

Proof. Consider the inequality condition from Theorem 4.3.6 under the SA model, for some prover i and any pair of subproofs $0, w$, we have:

$$\begin{aligned}
& V_x(\Theta^0, P_i) \geq \max_{w \in [m_0]} \frac{a_i(w)a_{-i}(w)}{a(w)a_{-i}(0)} V_x(\Theta^w, P_i) \\
\iff & V_x(\Theta^0, P_i) \geq \max_{w \in [m_0]} \frac{a_i \cdot s_w \cdot (\sum_{j \neq i} a_j) \cdot s_w}{(\sum_{j \in [n]} a_j) \cdot s_w (\sum_{j \neq i} a_j) \cdot s_0} V_x(\Theta^w, P_i) \\
\iff & V_x(\Theta^0, P_i) \geq \max_{w \in [m_0]} \frac{a_i}{\sum_{j \in [n]} a_j} \cdot s_{w,0} V_x(\Theta^w, P_i)
\end{aligned}$$

Under the inequality condition in Corollary 4.3.8, we have:

$$\begin{aligned}
\Theta^0 &\geq \max_{w \in [m_0]} \max_{i \in [n]} \frac{a_i}{\sum_{j \in [n]} a_j} \cdot s_{w,0} \cdot \Theta^w \\
\iff \Theta^0 &\geq \max_{w \in [m_0]} \frac{a_i}{\sum_{j \in [n]} a_j} \cdot s_{w,0} \cdot \Theta^w, \forall i \in [n] \\
\implies V_x(\Theta^0, P_i) &\geq \max_{w \in [m_0]} \frac{a_i}{\sum_{j \in [n]} a_j} \cdot s_{w,0} V_x(\Theta^w, P_i), \forall i \in [n] \\
&\text{(same reasoning as seen in the proof of Corollary 4.3.4)}
\end{aligned}$$

Thus, by Theorem 4.3.6, this completes the proof. $\square$

Corollary 4.3.8, together with Corollary 4.3.7, tells us that through appropriate scaling between the parameter sets of the subproofs and assuming SA model, we can derive simpler conditions to ensure PPS as the best strategy for any prover.

Chapter 5

Illustrative example

We will now put everything together and provide a simple example to illustrate how we can do collaborative formalization through Coq the proof assistant in a formalized proof version of the system we described in Chapter 3. Although we use Coq as our reference proof assistant, however, it can be replaced with any proof assistant with similar functionality. The scenarios, codes and figures presented in this example are based on the formalization of the sorting program described in Appendix A and are purely experimental as the system has not been implemented yet. All Coq files mentioned in this chapter can be found in the github repository: <https://github.com/jinxinglim/coq-chain>.

To begin, we will assume that the documents in the Coq standard library¹ are already stored in a decentralized file storage system, e.g., IPFS, and the system is bootstrapped with these documents via the contribution registry smart contract. For simplicity, we will use `0x0file` to represent the hash address of the Coq file *contribution*, *file.v*, in IPFS. The system will also come with its own form of cryptotoken, say *FormalCoin*, so that provers can put up rewards or get rewarded for publishing formalized proof code through this currency.

Suppose a prover, *C*, starts a new project to find proof/s of the specification of a sorting program as a conjecture through the Coq script, *ct00.v*, as shown in Listing 5.1.

```

1 Require Export Arith Sorted Permutation List.
2 Export List.ListNotations.
3 Open Scope list_scope.
4
5 Definition sorted := Sorted le.
6 Definition permutation := ·Permutation nat.
7
8 Conjecture sort_prog :
9   forall (l:list nat), {l':list nat | sorted l' ^ permutation l' l}.

```

LISTING 5.1: Coq script of *ct00.v*

Through some sort of user interface (an example is shown on Figure 5.1), *C* uploads the file *contribution ct00.v*, which will be eventually stored in IPFS. Minimally, the following data are also required from the user when uploading each contribution: (1) file type, (2) imported files required in the submitted file, (3) incentive mechanism to be implemented and the amount of reward, if any, and (4) the right-to-use. First, each contribution can be either one of these file types: (a) conjecture, (b) partial proof

¹<https://coq.inria.fr/library/>

- incomplete proof addressing some conjecture with unsolved subconjectures within the proof, or (c) completed objects - completed formalized definitions, tactics or proofs. Second, imported files required for the submitted contribution is required as to keep track of the linkages between files within the system for recording, analysis and reward distribution. Third, for contribution of conjecture type, user can either choose to implement the system's default incentive mechanism or his/her own incentive mechanism implementation. For the former, the default incentive mechanism is the one we described in Chapter 4 with $L = 0$ as all submitted proofs are formalized in this system. If it is the latter, the user needs to describe the incentive mechanism via a smart contract, which must follow some standard template so that it will be compatible for our system. The user also needs to state the total amount of reward to be distributed to the prover or set of provers who solved the conjecture. Lastly, if the contribution is of completed object type, the user needs to state the *right-to-use*, which can be either *free-to-use*, *pay-to-use* with the amount to be paid for using the contribution, or *restricted-to-use* as described in Chapter 3. In this example, *C* uploads the following information together with the file contribution `ct00.v`:

- File type: Conjecture
- Imports: "0x0Arith", "0x0Sorted", "0x0Permutation", "0x0List"
- Incentive mechanism: (default), Reward: 100 (FormalCoin)

Welcome to Collaborative Formalization

Upload your code here:

+

File type:	Conjecture	Partial proof	Completed object
Imports:	<i>Provide the imports your Coq file requires.</i>		
Incentive mechanism (conjecture only):	<i>If you are using the default mechanism, leave this empty. Otherwise, specify the address of your own mechanism that follows our standard template.</i>		Reward
Right-to-use (completed object only):	Free-to-use	Pay-to-use: <i>Amount</i>	Restricted-to-use

FIGURE 5.1: Example of how the user interface of our system can look like

At the backend, once the contribution is submitted, the `contribution_registry` smart contract records the entry with its necessary information as shown in Listing 5.2. Listing 5.3 provides an example of how an incentive contract describing the default incentive mechanism can be implemented. If the proof of all the conjectures in

contribution, in this case it is `sort_prog` from `0x0ct00`, are completed by a set of Provers, `iCont` will run the pre-defined allocation function to measure the weights of the contributions and allocate the `TokenReward` appropriately. Otherwise, all the `TokenReward` will be transferred to the unique prover.

Note that we will only be providing snippets/pseudo-codes of Solidity (programming language to write smart contracts for Ethereum) scripts instead of the full implementations. The goal is to provide a rough idea of how the smart contracts should look like and the actual full code implementations will vary depending on the blockchain the system is implemented on.

```

1  contract contribution_registry {
2      enum fileType{Conjecture, PartialProof, CompletedObject};
3
4      struct contribution {
5          address fileAddress;
6          fileType ctType;
7          address[] imports;
8          uint TokenReward;
9          int rightToUse; // 0: free, -1: restricted, otherwise: price to pay
10     }
11
12     contribution[] = contributions;
13     ...
14     function getAddress (address _fileAddress) public {
15         contribution.fileAddress = _fileAddress;
16     }
17
18     function getCtType (fileType _ctType) public {
19         contribution.ctType = _ctType;
20     }
21
22     function getImports (address[] _imports) public {
23         contribution.imports = _imports;
24     }
25
26     function getRewards (uint _TokenReward) public {
27         require(contribution.ctType == Conjecture);
28         contribution.TokenReward = _TokenReward;
29     }
30
31     function getRightToUse (int _rightToUse) public {
32         require(contribution.ctType == CompletedObject);
33         contribution.rightToUse = _rightToUse;
34     }
35
36     contributions.push(contribution);
37
38     if (contribution.ctType == Conjecture) {
39         iCont(Provers, "0x0ct00", TokenReward);
40     }
41
42     ...
43 }

```

LISTING 5.2: Code snippet of `contribution_registry`

```

1 contract iCont {
2   ...
3
4   // reward to set of provers, Provers, with Reward
5   function reward(list Provers, address contribution) public {
6     require(verify(contribution) == True);
7     if (length(Provers) == 1) {
8       transfer(Provers, TokenReward);
9     }
10    else {
11      // weightage of rewards will based on allocation fn
12      weights = allocation(sort_prog);
13      allocate(Provers, weights, TokenReward);
14    }
15  }
16
17  ...
18 }

```

LISTING 5.3: Pseudo-code example of iCont

As mentioned in Chapter 3, a TCR contract is implemented for bonded provers to decide and vote if a contribution is significant enough to be included in the canonical registry. Assuming there is no zk-SNARK implementation for checking the validity of every file contribution, the bonded provers would do validity check of each contribution either running Coq on their own computers or through some sort of Coq plug-in editor implemented at the user interface similar to jsCoq². After verification of the file contribution, bonded provers will then vote if it should be included in the canonical registry based on some some agreed upon criteria. The purpose of this TCR contract, as mentioned previously, is to consolidate novel, useful and valid submissions in the canonical registry. For example, after C 's `ct00.v` file contribution is recorded on the blockchain via the contribution registry, bonded provers will decide if the conjecture from the file should be considered as an important problem to be included in the canonical registry for provers to work on. Suppose C 's `sort_prog` is considered to be an important problem for provers to work on, and hence, included in the canonical registry, a string diagram (Figure 5.2) is presented in the user interface to track any progress to be made on the conjecture.

At this point, suppose a human prover P_1 provides the first contribution via the Coq script, `ct01.v`, as shown in Listing 5.4. The code consists of an initial partial proof of `sort_prog` which initiates the proof by using induction and leaves both the base case and inductive case empty. Similarly, he uploads the file via the user interface (Figure 5.1) and his contribution is then recorded via the `contribution_registry` smart contract (Listing 5.2). Once his contribution is included in the canonical registry, the string diagram in Figure 5.2 will be updated to the one shown in Figure 5.3.

²<https://coq.vercel.app/>

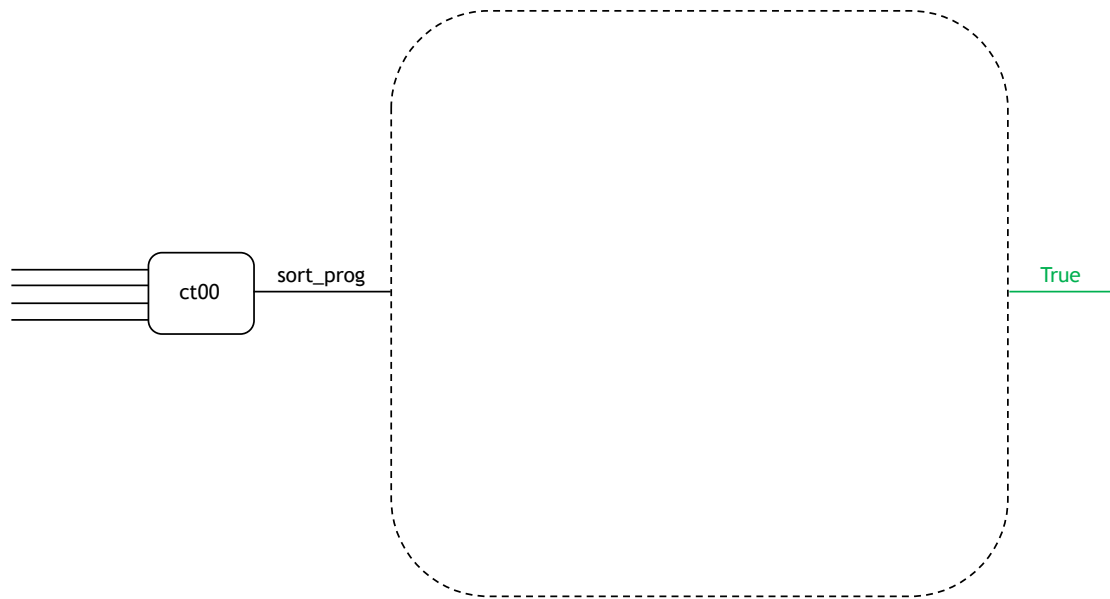


FIGURE 5.2: String diagram representing linkages between files (containing conjecture, partial proof or completed object). The 4 input edges of the node "ct00" are the 4 imported files of ct00.v. The output edge of node "ct00" represents the conjecture "sort_prog", which leads to a dotted node representing an empty proof yet to be completed.

```

1 Require Export ct00.
2
3 Conjecture sort_prog_base: {l': list nat | sorted l' ^ permutation l' []}.
4
5 Conjecture sort_prog_IH: forall (a: nat) (l x: list nat),
6   sorted x → permutation x l → {l': list nat | sorted l' ^ permutation l' (a :: l)}.
7
8 Lemma sort_prog:
9   forall (l: list nat), {l': list nat | sorted l' ^ permutation l' l}.
10 Proof.
11 induction l.
12 apply sort_prog_base. destruct IHl; destruct a0;
13 eapply sort_prog_IH; eassumption.
14 Qed.

```

LISTING 5.4: Coq script of ct01.v

One of the reasons why we want our system to be implemented on the blockchain instead of systems such as Github³ is that unsolved conjectures can also be attempted by *AI provers*. These users are the ones in the system who are running automated AI tools, such as the ones who mentioned in Chapter 2, on their own servers. They can either manually look for conjectures to run their automated tools or implement code to look for conjectures automatically to solve using their automated tools. Although most of these automated tools are open-source, training and perfecting them

³<https://github.com/>

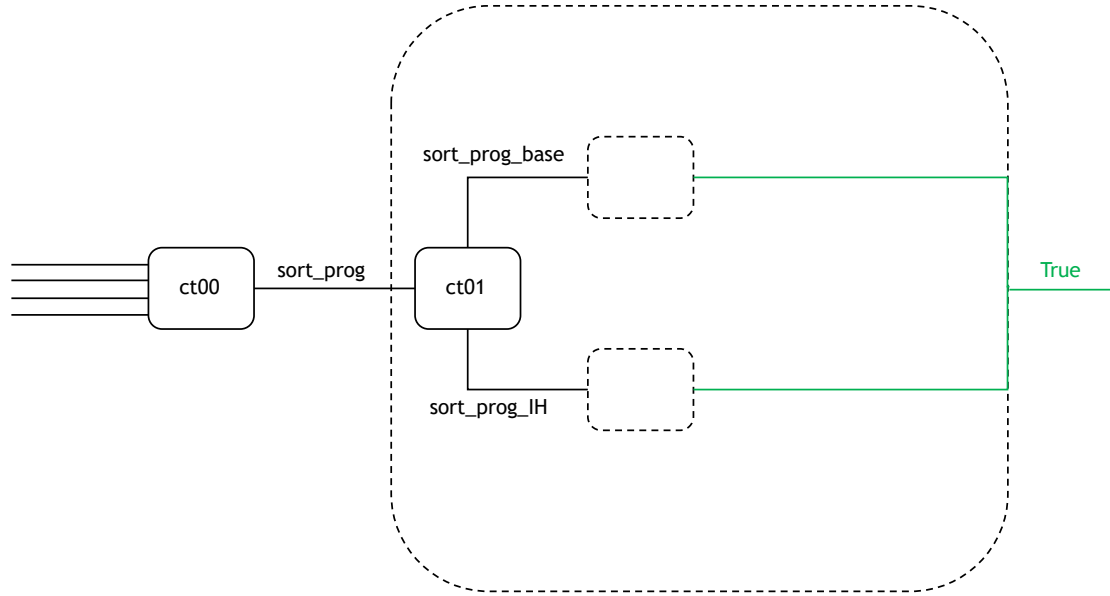


FIGURE 5.3: Update in the string diagram shown in Fig. 5.2 after P_1 uploads the entry of $ct00.v$.

may require a lot of computing power. With our blockchain-based system, users without high computing capabilities need not install these automated tools in their own servers and can rely on users who have to make progresses with the tools. Users who want to have their tools proprietary can also use our platform to look for conjectures to solve and also test their tools' efficiencies. For example, an AI prover, A , using automated AI tool, Coqhammer Czajka and Kaliszyk, 2018, will try to automate the proofs for the three available conjectures which are represented by the dotted nodes in Figure 5.3 ($sort_prog$, $sort_prog_base$ and $sort_prog_IH$). Although it only manages to automate the proof of $sort_prog_base$, it uploads its attempts via the Coq script, $ct02.v$ (Listing 5.5) and the contribution is recorded in the blockchain via the `contribution_registry` smart contract. This can be done manually by the human user implementing A or automatically by having a code implementation to interact with the smart contract directly. The user interface will then update the progress in the string diagram shown in Fig. 5.3 to the one shown in Fig. 5.4.

```

1 Require Import ct01.
2 From Hammer Require Import Hammer.
3
4 Lemma sort_prog: forall (l: list nat),
5   {l' : list nat | sorted l' ^ permutation l' l}.
6 Proof. Fail hammer. Abort.
7
8 Lemma sort_prog_base: {l' : list nat | sorted l' ^ permutation l' []}.
9 Proof. hammer. Defined.
10
11 Lemma sort_prog_IH: forall (a: nat) (l x: list nat), sorted x → permutation x l
12   → {l' : list nat | sorted l' ^ permutation l' (a :: l)}.

```

13 `Proof. Fail hammer. Abort.`

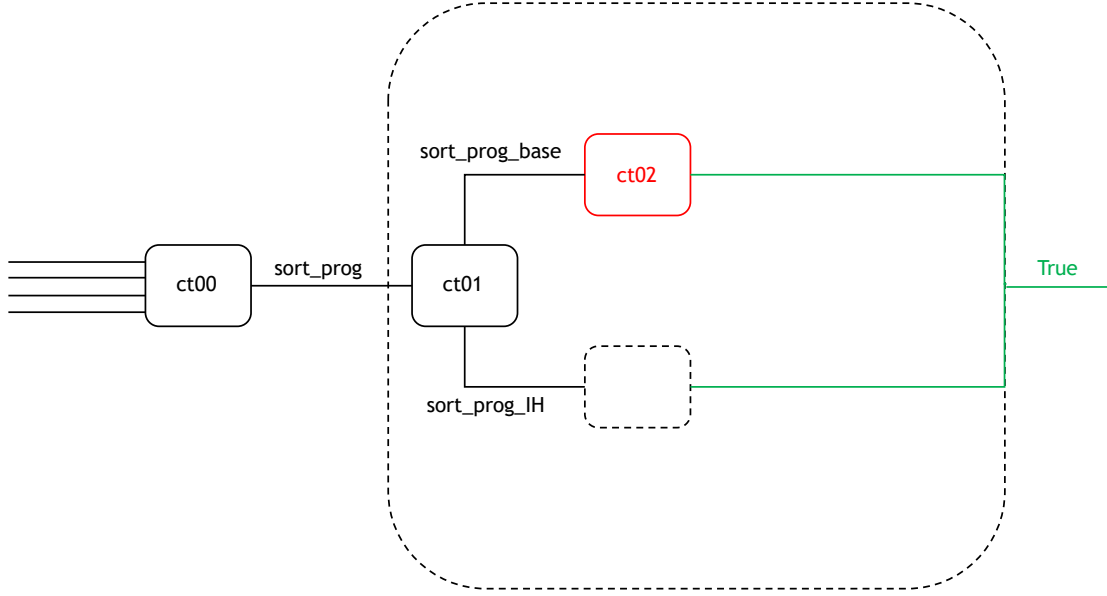
LISTING 5.5: Coq script of `ct02.v`

FIGURE 5.4: Update in the string diagram shown in Fig. 5.3 after A uploads the entry of `ct02.v`. We use red nodes to represent entries from AI provers.

Similarly, with the proof of `sort_prog_IH` being empty, another human prover, P_2 , makes some partial progress on it through `ct03.v` (Listing 5.6) and the string diagram shown in Figure 5.4 is updated to the one shown in Figure 5.5.

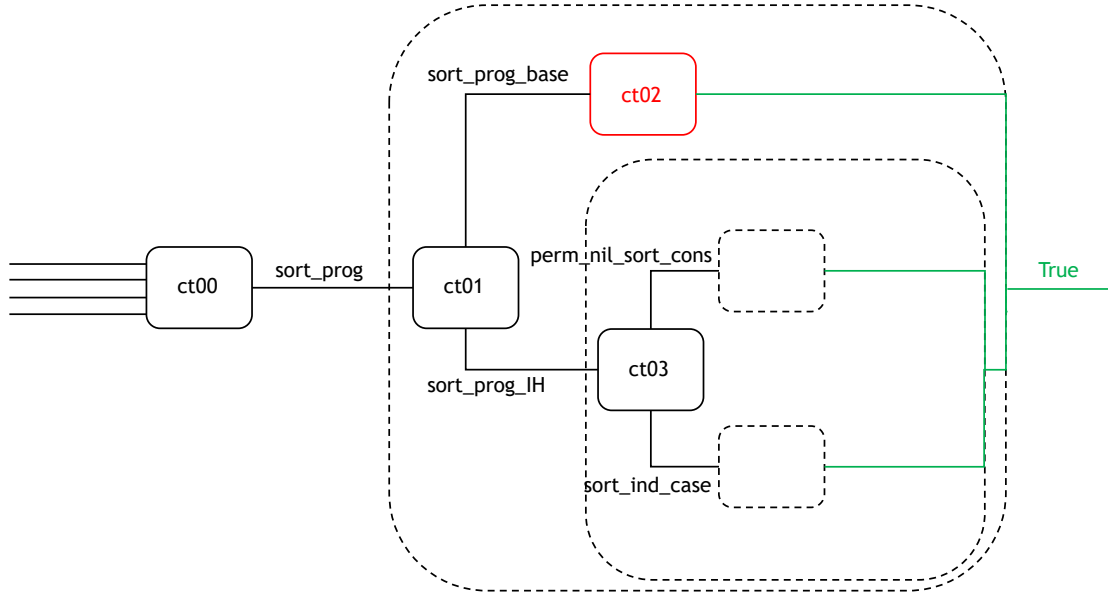
```

1 Require Export ct01.
2
3 Conjecture perm_nil_sort_cons: forall (a : nat) (l : list nat),
4   permutation [] l → {l'0 : list nat | sorted l'0 ∧ permutation l'0 (a :: l)}.
5
6 Conjecture sort_ind_case: forall (a a0 : nat) (l' l x : list nat),
7   sorted (a0 :: l') → permutation (a0 :: l') l
8   → sorted x → permutation x (a :: l')
9   → {l'0 : list nat | sorted l'0 ∧ permutation l'0 (a :: l)}.
10
11 Lemma sort_prog_IH: forall (a : nat) (l l' : list nat),
12   sorted l' → permutation l' l →
13   {l'0 : list nat | sorted l'0 ∧ permutation l'0 (a :: l)}.
14 Proof.
15 intros; revert l H H0; induction l'; intros.
16 - apply perm_nil_sort_cons; eassumption.
17 - destruct (IHl' l'); clear IHl'.
18   + inversion H; eassumption.
19   + apply Permutation_refl.
20   + destruct a1; eapply sort_ind_case; eassumption.

```

21 Qed.

LISTING 5.6: Coq script of ct03.v

FIGURE 5.5: Update in the string diagram shown in Fig. 5.4 after P_2 uploads the entry of ct03.v.

Eventually, all subproofs are completed, that is, no more dotted nodes between `sort_prog` and `True`, through the collaboration of contributions from both human and AI provers as shown in Fig. 5.6. As discussed in Appendix A, the completed proof from this string diagram of subproofs is essentially a proof of an insertion sort algorithm.

Once this is achieved, the `reward` function in `iCont00` will be triggered to allocate each contribution, seen in Fig. 5.6, with the appropriate rewards based on the `allocation` function. As mentioned, this `allocation` function satisfies the requirements of the incentive mechanism as discussed in Chapter 4 when $L = 0$. For example, we shall take a look at how we can derive the amount of reward to be allocated to the contribution of `ct01.v`.

First, as this system only accepts formalized proofs, parameter set Θ^u will only consists of R_0^u for any subproof u . This is because no reviewer is required to verify the correctness of the proof, and thus, stakes, bounties and rewards like R_1^u and R_2^u are not required. We shall remove the subscript 0 from R_0^u to avoid confusion for the rest of this chapter. Figure 5.7 shows the string diagram the incentive mechanism is implemented on in deciding the rewards for `ct01` (R^{ct01}), `sort_prog_base` (R^{base}) and `sort_prog_IH` (R^{IH}). Second, we need to decide on the simplicity scores for these three subproofs via either the TCR contract or a contract running a decentralized review system. Suppose we assume there are 3 provers with equal innate ability working on the problem (i.e., $a_i = a_j$ for $i \neq j \in \{1, 2, 3\}$) and the simplicities of these three subproofs have been decided as follows: $s_{ct01} = 10$, $s_{base} = 10$ and $s_{IH} = 3$. What

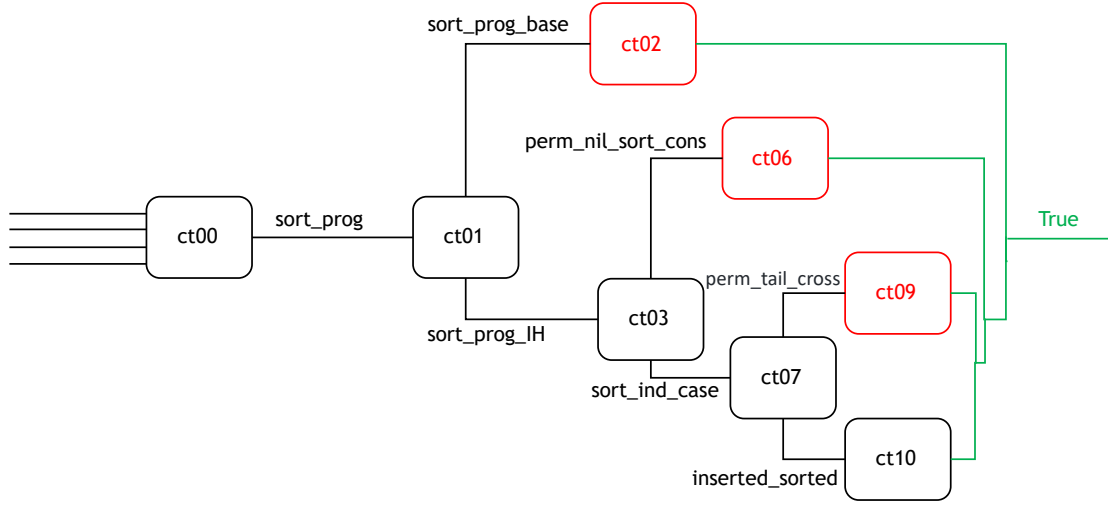


FIGURE 5.6: Collaboration of contributions from human and AI provers that yields a proof of insertion sort algorithm.

is left for the incentive mechanism to do is to find a feasible solution to the following linear constraints:

$$R^{ct01} + R^{base} + R^{IH} \leq 100 \quad (5.1)$$

$$R^{ct01} \geq \frac{1}{3} R^{base} \quad (5.2)$$

$$R^{ct01} \geq \frac{1}{10} R^{IH} \quad (5.3)$$

$$10R^{base} < 3R^{IH} \quad (5.4)$$

$$R^{ct01}, R^{base}, R^{IH} > 0 \quad (5.5)$$

Inequality 5.1 states that the total rewards for the three subproofs must be less than or equal to the total reward allocated for this problem, which is 100 FormalCoin. 5.2 and 5.3 are the conditions derived from Corollary 4.3.8 and 5.4 states the condition of Corollary 4.3.7 assuming that we want $base \prec IH$. As such, one possible solution is $R^{ct01} = R^{base} = 10, R^{IH} = 80$. C , the prover who posted the open conjecture `sort_prog`, can also modify the `iCont` smart contract and include an objective function such as $\max R^{IH}$, i.e., maximize the rewards for the proof of inductive case. By changing the inequality to equality in the inequality 5.1, an optimal integer solution would $R^{ct01} = 9, R^{base} = 1, R^{IH} = 90$. The incentive mechanism can be repeatedly implemented for each nested string diagram to derive the rewards for `ct03.v` and beyond.

Besides insertion sort, there are many different sorting algorithms, including merge sort, quick sort, etc. Unlike in mathematics, where it is sufficient to have one proof for each theorem, different proofs for a specification of a program may yield algorithms

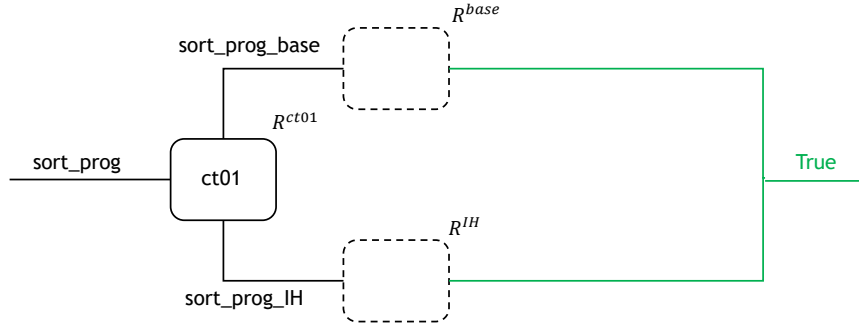


FIGURE 5.7: String diagram the incentive mechanism is implemented on in deciding the values of R^{ct01} , R^{base} and R^{IH} .

with different efficiencies. Thus, it is important to implement incentive mechanism/s via smart contract/s to encourage users to produce different proofs collaboratively as well. For example, C could implement the incentive mechanism of halving the the reward for each completed proof (mentioned in Chapter 3) similar to what was studied by Babaioff et al. Babaioff et al., 2012 via a smart contract. We shall provide a demonstration of how it could happen in our example.

After the contribution of `ct03.v` as seen in Figure 5.5, another prover P_3 , motivated to create an alternative proof to `sort_prog`, creates a new tactic, `div_conq_split` (Listing 5.7), which is a formalization of the divide-and-conquer algorithmic paradigm, in the Coq file `ct04.v`. The file is uploaded in the IPFS network and eventually is recorded as an entry in the canonical entry. As this file does not directly contribute as a partial proof to `sort_prog`, there will not be any update to any string diagram. However, it is considered as a contribution of completed object type and this new tactic can be applied not only to `sort_prog`, but to any proposition on lists. Thus, P_3 wants to impose a fee for importing and using the content of this file contribution, say 5 FormalCoin. P_3 then uses this new tactic to apply to `sort_prog`, instead of `induction` as used in `ct01.v`, in a Coq file contribution `ct05.v` (Listing 5.8). Completion of such approach will result in the proof of merge sort. Once this file contribution is recorded in the canonical registry, the string diagram shown in Figure 5.2 will be updated to another string diagram (Figure 5.8) alternative to the one in Figure 5.3. At this point, all provers in the system will have both string diagrams from Figure 5.5 and 5.8 presented to them and they can choose which proof they wish to progress on.

```

1 Theorem div_conq_split : forall (P : list A → Type),
2   P nil
3   → (forall (a : A), P (a :: nil))

```

```

4   → (forall (ls:list A), P fst(split ls) → P snd(split ls) → P ls
5   → forall (l : list A), P l.

```

LISTING 5.7: div_conq_split in Coq

```

1  Require Export ct00 ct02 ct04.
2
3  Conjecture sort_prog_one: forall a:nat,
4    {l' : list nat | sorted l' ∧ permutation l' [a]}.
5
6  Conjecture sort_prog_split: forall (ls l' l'0: list nat),
7    sorted l'0 → permutation l'0 (fst (split nat ls))
8    → sorted l' → permutation l' (snd (split nat ls))
9    → {l'1: list nat | sorted l'1 ∧ permutation l'1 ls}.
10
11 Lemma sort_prog: forall (l : list nat),
12   {l' : list nat | sorted l' ∧ permutation l' l}.
13 Proof.
14   div_conq_split.
15   - apply sort_prog_base.
16   - apply sort_prog_one.
17   - intros; destruct H; destruct a; destruct H0; destruct a;
18     eapply sort_prog_split. exact H. eassumption. exact H0. eassumption.
19   Qed.

```

LISTING 5.8: Coq script of ct05.v. Note that since sort_prog_base is already solved in ct02.v, the file simply imported and used the completed proof.

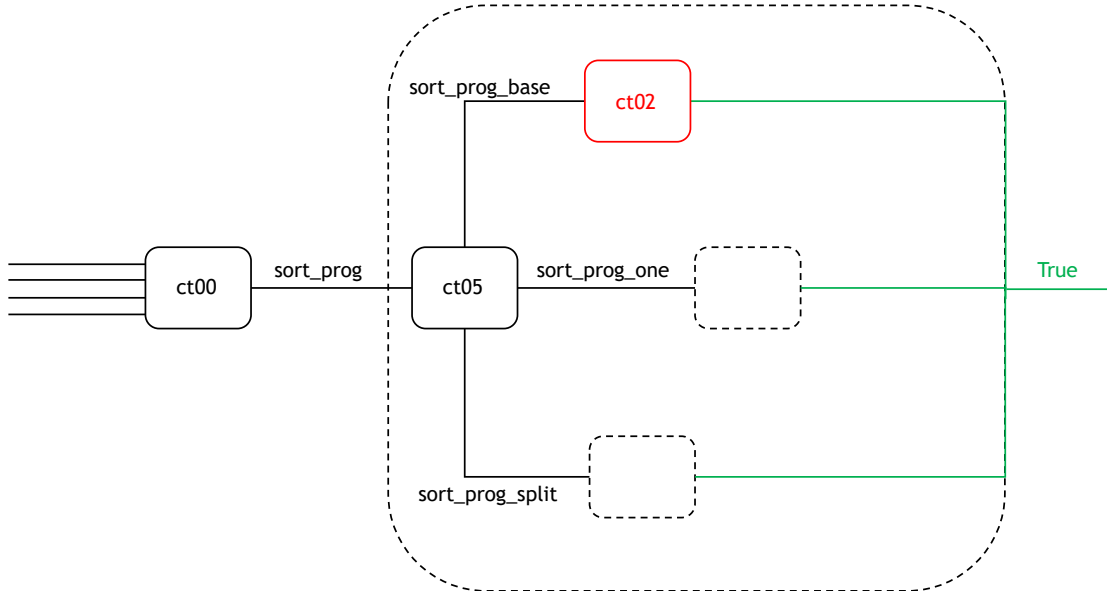


FIGURE 5.8: New string diagram representation of the alternative proof presented in ct05.v. Note that sort_prog_base is solved as P_3 imported and used the result from ct02.v.

Similar to the completion of insertion sort's proof (Figure 5.6), the proof of merge sort will eventually be completed through collaboration of provers as seen in the string diagram from Figure 5.9. As the proof of insertion sort is completed before merge sort's (by comparing the last contribution of the two string diagrams), 100 FormalCoin will be rewarded to the contributions of the former and half of this amount, i.e., 50 FormalCoin, will be rewarded to the contributions of the latter according to the incentive mechanism of halving the reward for each completed proof. Subsequent completed proofs will get the amount of rewards according to this halving mechanism.

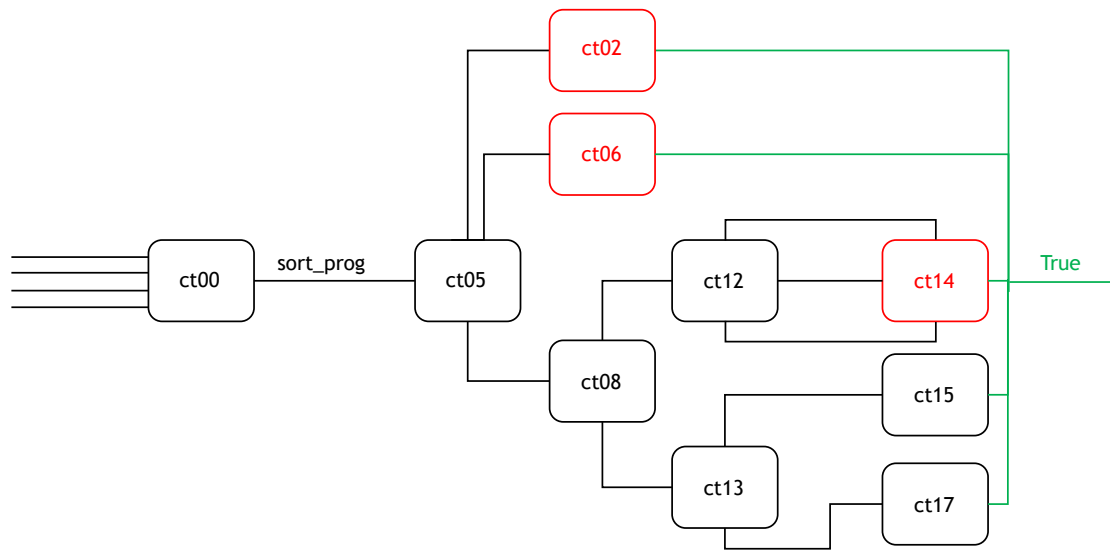


FIGURE 5.9: Collaboration of contributions from human and AI provers that yields a proof of merge sort algorithm. Note that labelling of the edges are removed for convenience.

The code repository, <https://github.com/jinxinglim/coq-chain>, provides an example of how the code discussed in Appendix A can be split up into different file contributions from provers.

Chapter 6

Conclusion and discussion

Research collaboration provides opportunities for researchers to learn how complementary insights can help to progress and solve existing problems. As a result, solutions, especially to difficult problems, can be found quicker as opposed to working in isolation. In current practices, collaborations are restricted to the social connections of the authors as there is little to no platform that allows decentralized collaborations. Having said that, there are several efforts in the area of mathematical research such as the Polymath project (Gowers, 2009), Qeditas (White, 2016) and Proofgold (Brown, 2020) that try to provide a platform for mathematicians and computer scientists to pose their results in a decentralized manner.

As compared to other research areas, it is particularly easier for a mathematical problem to be broken down into smaller problems as each proof can be usually broken down into several smaller subproofs (i.e., lemmas) to be completed before it is finalized. Connections between subproofs can also be easily illustrated with string diagrams (Chapter 2), which leads to the ease of subtasks' assignments to collaborators.

However, despite some successes from the Polymath project, where collaborators published papers solving difficult problems in number theory under the pseudonym of D. H. J. Polymath, new contribution has not been reported since 2014. The difficulty of appropriately incentivizing participation seems to have hampered from having a more widespread adoption despite its revolutionary potential (Cranshaw and Kittur, 2011; Friesike and Fecher, 2016).

To this end, the thesis proposes two blockchain-based approaches, one that accepts pen-and-paper proofs and another that only accepts formalized proofs, to allow agents, human and AI provers (for the formalized version), to collaborate through sharing partial proofs. Each approach comes with a system architecture that can be broken down into three interconnected layers: data layer (what is recorded?), client layer (what is meaningful?) and incentive layer (what is rewarded?). We see how existing blockchain-enabled technologies such as TCR, NFT, smart contracts, decentralized storage and review systems are used in our approaches. We design the first to our knowledge decentralized conference for collaborative mathematics through our pen-and-paper version. As for the formalized proofs version, what sets our approach apart from the other projects mentioned in Section 1.2.2 is that we design a system that enables and encourages human and AI provers to participate and formalize partial proofs. As AI technologies and techniques improve, we hope to see different automated AI tools use our blockchain-based system as a backbone and work on different parts of automation as an unified proof and program synthesizer in the future. Lastly, in order to incentivize provers to share their partial results and reviewers to review, we provide a game

theoretical proof of concept to show how one can design the incentive mechanisms to achieve that.

6.1 Discussion on assumptions and limitations

There are several assumptions we made in our work which we will be addressing in this section. Moreover, as this thesis discusses a relatively new concept of decentralizing collaboration for proofs and programs, we will also discuss some considerations and limitations that we need to keep in mind while implementing and improving the system.

Proof representation of mathematical proof

In the paper by Banerjee, Goel, and Kollagunta Krishnaswamy, 2014, they represented mathematical proof structures with DAGs and analyzed the incentive mechanisms for both sequential and general subproofs DAGs. On the other hand, we use string diagram representation due to the reasons stated in Chapter 2 and analyze mechanisms for diagrams of sequential and parallel subproofs as illustrated in Figure 2.4a and 2.4b. It is sufficient for us to do that as incentive mechanisms can be recursively applied to each individual string diagram of each node instead of finding a single mechanism that analyzes a more general diagram as it may complicate the analysis.

In our model, we also assume that once a prover P proposes a initial subproof which states all required lemmas, the string diagram associated to P 's approach is fixed and all provers will be committed to complete the whole proof. This assumption is reasonable in the case where the conjecture has been unsolved for many years and any initial proof proposed might be the only approach that one could think of, otherwise it would not be unsolved for that long. Thus, all provers who are interested in the conjecture would be committed to complete this partial proof.

In the event where there are more than one approaches for the conjecture, the rate of finding a claim needs to take into account of each prover i strategically-chosen distribution $\{x_i(u)\}$ where subproof u belongs to all subproofs from all different approaches. As a result, under condition (IV), the time of finding a claim would be $T_i^u \sim \text{Exp}(x_i(u)a_i(u))$ instead of $T_i^u \sim \text{Exp}(a_i(u))$. This is one consideration where the incentive mechanism can be enhanced.

Further, each proof approach is represented by an individual string diagram. Hence, another new proof approach will result in a new string diagram representation. The incentive mechanism will also be deployed on the new string diagram with different values for the parameters (i.e., stakes, bounties and rewards distribution) from the ones in original string diagram. However, the reward will only be distributed to the provers who have their string diagram completed first, assuming there is only one prize reward for completion of every conjecture. Thus, even if there is more than one proof approach for a particular conjecture, it will be a race against each other to see who can complete their string diagram, and hence winning the reward first.

For the stakes and bounties, the smart contract can be programmed to hold on to them and will only deduct from the provers' and reviewers' accounts respectively if the string diagram that they are working on is completed. This is done so that provers'

stakes and reviewers' bounties will be returned to the respective owners in case the string diagram they are working on fails to complete before another did.

Assumption of provable conjectures and lemmas in incentive mechanism

The model of the incentive mechanism assumes that the conjecture of interest and its subsequent lemmas required are provable. That is, there exists a proof that shows that the conjecture/lemma is true. However, what happens if it is false? What about the case where it is unprovable, i.e., it cannot be proven to be true or false? There are a few suggestions on addressing this issue.

Suppose a total reward of USD 1 million worth of tokens are to be given to provers who solve some conjecture. If the conjecture turns out to be provable, then the incentive mechanism works as described in Chapter 3. But in the case where it turns out to be false, the proposer could also give the reward (or some portion of it) to the prover/s that disprove it. If the proposer strongly believes that the conjecture is indeed provable, he could even assign even higher reward (e.g., double the original reward) to the prover/s that disprove it. As a result, this can incentivize and attract more provers to prove and disprove the conjecture of interest.

It is also important to note that there are different ways to disprove a conjecture or even an approach of an existing proof, which affect how the incentive should be given after doing so. For example, to disprove a conjecture is to simply provide a counter-example that makes the conjecture false. However, in the case where a prover, say prover P has proposed an initial proof of $A \implies B \implies C$, with A proven, to prove conjecture C as seen in Figure 2.5a, one could show that the approach provided by prover P is incorrect by showing that lemma B is actually false. In this case, there is still a challenge for the proposer to decide on how the reward should be given to the prover that shows lemma B is false. This is because conjecture C may still be provable but just that the approach of $A \implies B \implies C$ is wrong.

Lastly, for the case where the conjecture turns out to be unprovable, the reward can be held by the smart contract implementing the incentive mechanism for some period of time before eventually releasing back to the proposer. After which, although there will not be any reward attached to the conjecture, we can use the blockchain to keep an record of all the conjectures that are unprovable or simply too difficult to solve for the mathematicians to discuss and ponder upon. Thus, there are still some areas of improvement for the incentive mechanism in order to cover all scenarios. The benefit of implementing our system through blockchain is that this can be done easily through the use of smart contract (after all, smart contracts are simply program codes running on the blockchain).

Proof verification process

Our blockchain-based system works similar to how a typical conference would work, i.e., provers submit proofs and get a few rounds of reviews from reviewers. The main difference is that instead of having the program committee to eventually decide on the acceptance of the papers in conferences, our system has formalized proofs as the final arbiter. Although formalized mathematics is still not as prevalent as the traditional pen-and-paper proofs, there are several ways to get around it.

First, as we see from the analysis, stakes, bounties and rewards will affect the behaviour of the agents. One could put a high enough reward for the final round of the debate system ($l = 0$) to attract a pool of expert agents on formalized mathematics to work on it.

Second, the game designer could increase the maximum number of rounds of the reviewing process such that reaching round 0 will be highly unlikely (e.g., increase L from 2 to 5). However, game theoretical analysis for a generalized value of L needs to be done in the future. Another difference between the traditional conferences and our system is the selection of reviewers. In a typical conference, reviewers are either part of the conference committee or their affiliates. This may lead to biasness and unreliable reviews due to conflict of interests. However, in our system, as the reviewing process is decentralized and incentivized, reviews can come from any agent from the network and are subjected to rebuts from a decentralized pool of agents. As a result, quality of reviews can be assured due to the stake (to be won) and bounty (to be lost) required for each review. Further analysis of a decentralized market of provers and reviewers, instead of just between the Prover and Reviewer, would be beneficial.

Assumptions on variables and parameters

First, there is no assumption of unit and distribution is made for the variables, prover-subproof aptitude, $a_i(u)$, prover's innate ability a_i and simplicity of subproof, s_u . This allows future analysis and evaluation study to be conducted without any such assumptions. Anyway, these variables are commonly scored or rated through some form of ordinal measurement, which is independent of units. More rigorous analysis can also be done supposed that these variables follow certain distributions.

Second, the rate of finding a claim of a subproof is assumed to follow an exponential distribution as it simplifies the analysis, which is common when considering the dynamics of games (Banerjee, Goel, and Kollagunta Krishnaswamy, 2014; Ghosh and Kleinberg, 2013). However, from the analysis, we can see that even if the distribution is replaced with other distribution, similar approach can be done to find the equilibria (i.e., mainly the probability of prover i finding a claim before others and its related probabilities will be affected). Thus, our analysis is robust against change in distribution assumption to a certain extent.

Third, we mentioned that the time for validating proofs is negligible as compare to the time used to come out with proof claims. This assumption is not invalid as most of the time when solving mathematical problems, it may take years to come out with a subproof but it will only take days or at most weeks to validate its correctness. This is especially true when it comes to formalized proofs, where the proofs will be checked in a matter of seconds and are done by machines.

Lastly, in our analysis, we assume that every prover i has her own individual confidence of formalizing proofs, P_i , regardless of subproofs. P_i depends on the subproof u if the subproofs within a string diagram belong to different fields of mathematics. Thus, different provers have varying confidence in formalizing on a particular subproof, which is dependent on the field the subproof is in. However, in many cases, all the subproofs in the network belong to a fixed mathematical field, say number theory. Thus, independence of P_i and subproof u can be assumed as this confidence is usually

based on the ability of formalizing proofs the prover has in the field of the problem is in since all subproofs belong to the same field.

This can be said similarly for the assumption of separable-aptitudes (SA) model. In cases where all subproofs belong to the same field, their abilities in proving any subproof do not vary, as they are usually based on their experiences in that field. Simplicity s_u of any subproof u also scales comparably for all provers. As a result, SA model can be assumed as a_i and s_u are independent of each other.

Inappropriate reward allocation due to voting issues

Our incentive mechanism, in particular the reward distribution (similarly for stakes and bounties) among the provers (and reviewers), relies on truthful voting on the simplicities of subproofs. First, it is important to see that from the analysis from Chapter 4, voters just need to vote for relative simplicities between subproofs (i.e., $\frac{s_u}{s_v}$ for subproofs u, v) instead of subproofs' simplicities individually. This will provide less ambiguous scaling between rewards in the incentive mechanism as it would be harder for a voter to determine a subproof's simplicity individually than relative simplicity between two subproofs.

Second, rewards, stakes and bounties will only be distributed to and deducted from the respective accounts after the whole proof has been completed. This is crucial as voting of relative simplicities may vary over the course of deriving the whole proof. For example, the relative simplicity between two subproofs, say u and v , could be initially derived at some value, say 1. But over the course of progressing the proof, the voters realize that subproof v is not as simple as it was deemed to be initially. As the voting will only terminate after the whole proof has been completed, voters can still vote and cause the relative simplicity between these two subproofs to change to another value, say 0.5. This will result in changes in parameters, such as rewards, between subproofs. On contrary, if voting process terminates after a subproof is completed, such adjustment cannot be made, which may cause unfair realization for the values of parameters.

Lastly, voting can be subjected to adversarial attacks such as bribing and market manipulations to induce a preferred outcome. The effectiveness of a voting system depends on the trust model (Buterin, 2020) it is relying on. Depending on which statistic is being used to finalize the scoring of relative simplicities, it will affect the amount of trust required by the model. For example, in the case where every provers can be voters and statistic of central tendency (e.g., mean, median or mode) is used to finalize the scores, the trust model requires majority of voters to be honest. This could result in adversarial attacks if the attacker can influence a big portion of the voters. To combat this, there are also several solutions proposed by Buterin, 2021 to address the issue. Proof of personhood (i.e., system that ensures one vote per human by verifying accounts correspond to unique individual human) and/or proof of participation (i.e., proof that the person participated some related event, passed some training or performed in some useful work) can be implemented in the voting system so that every voter accounts for his/her vote. Tokens that found to be voting maliciously can be destroyed and voter's right to participate as prover or voter can be revoked as well.

6.2 Future directions

Moving forward, the next immediate goal is to implement our proposed blockchain-based approaches as dApp. Any developer can treat the thesis as a blueprint or whitepaper to implement the systems. Once the dApps are implemented, evaluations on the effectiveness and efficiencies of the systems and incentive mechanisms can be done. However, other than the limitations discussed above that the developer needs to consider, there are several considerations and improvements to think about before rolling out the systems.

First, it is to decide on the blockchain the systems should be implemented on. One main requirement is that the blockchain has to have the capability to run the incentive mechanisms through the smart contracts, for example, Ethereum (Gavin Wood et al., 2014) and Cardano (Hoskinson, 2017). Although it might be too much hassle to create a new blockchain to implement the systems, one could still move in this direction and incorporate it with a zk-SNARK scheme for formalized proof validation as mentioned in Chapter 3. This will reduce the time and energy for users to run each of the formalized proofs manually through their own servers.

Second, there can be two usages of the tokens used in the blockchain-based systems. On one hand, they can be used as a virtual currency purely to measure the contributions a user has made in the world of formalized mathematics and programs. This is analogous to the h-index (Hirsch, 2005) to measure the productivity of a researcher's publications. On the other hand, they can be exchanged for actual currencies, e.g., USD. This approach gamifies the process of formalization through monetary rewards, and hence, potentially attracts more users to take part. The decision to use tokens as "scores" or as monetary incentive may depend on the community participating in our protocol.

Lastly, the token valuation of a mathematical result should be proportionate to its impact and novelty. For example, suppose a mathematical result receives more citations over time, there should be a mechanism that allows the authors to receive more tokens (similarly to how the h-index of an author increases when his/her paper gets cited).

Appendix A

Formalization of divide-and-conquer algorithm in Coq

Programming demands that developers have a high level of reasoning, creativity and experience to design an executable program for its intended purpose. Moreover, coming out with a program is just part of the equation. One needs to check if the intention is met by testing his or her program with large sample data or by proving its correctness. In theoretical computer science, formal verification is the use of formal mathematical proofs to prove the program's correctness with respect to some stated specifications (Bjesse, 2005). This can be done through either interactive theorem provers (such as Isabelle (Wenzel, Lawrence C Paulson, and Nipkow, 2008), Coq (Bertot, 2008) and Lean (Moura et al., 2015)) or automated theorem provers (such as E (Schulz, 2002), Vampire (Riazanov and Voronkov, 2002) and Z3 (De Moura and Bjørner, 2008)), which include satisfiability modulo theories (SMT) solvers. We will look at a simple sorting program as an example to illustrate how it is currently done in Coq the proof assistant.

As seen in the Software Foundations book by Appel, 2021, the insertion sort function¹ is built through a *bottom-up* approach, likewise for most programs nowadays. That is, the `insert` function is created first, before using it in the `isort` function (Listing A.1).

```

1 Fixpoint insert (i : nat) (l : list nat) :=
2   match l with
3   | [] => [i]
4   | h :: t => if i <= h then i :: h :: t else h :: insert i t
5   end.
6
7 Fixpoint isort (l : list nat) : list nat :=
8   match l with
9   | [] => []
10  | h :: t => insert h (isort t)
11  end.
12
13 Example sort_pi : isort [3;1;4;1;5;9;2;6;5;3;5] = [1;1;2;3;3;4;5;5;6;9].

```

LISTING A.1: Insertion sort from Software Foundations

¹<https://softwarefoundations.cis.upenn.edu/vfa-current/Sort.html>

Line 13 of Listing A.1 demonstrates a test function to check if the sort function indeed works as it supposed to. However, such testing is insufficient to prove its correctness as it is not exhaustive and bugs can only be found at runtime. The benefit of using Coq and other formal verification tools mentioned above is that they allow us to write the specification of a program and prove that it is satisfied mathematically. As a result, the correctness of the program can be verified even before running it. For example, to check for the correctness of the sort function in Listing A.1, we need to state the specification of the function as a proposition in Coq (Listing A.2). That is, we need to specify its intention, which is to input a sequence and output a sorted sequence in some specific ordering. Moreover, the output sequence must be a permutation of the input sequence.

```

1 Inductive sorted: list nat → Prop :=
2   | sorted_nil: sorted []
3   | sorted_l: forall x, sorted [x]
4   | sorted_cons: forall x y l,
5     x <= y → sorted (y :: l) → sorted (x :: y :: l).
6
7 Inductive Permutation: list A → list A → Prop :=
8   | perm_nil: Permutation [] []
9   | perm_skip x l l': Permutation l l' → Permutation (x::l) (x::l')
10  | perm_swap x y l: Permutation (y::x::l) (x::y::l)
11  | perm_trans l l' l'': Permutation l l' → Permutation l' l'' → Permutation l l''.
12
13 Definition is_a_sorting_algorithm(f: list nat → list nat) :=
14   forall al, Permutation al (f al) ∧ sorted (f al).
15
16 Theorem isort_correct: is_a_sorting_algorithm isort.

```

LISTING A.2: Specification of sort function

Line 1 to 5 gives the definition of `sorted`, whereas line 7 to 11 provides the definition of `Permutation`. With these two definitions, line 13 and 14 provides the definition that specifies the requirements of a sorting algorithm. To prove the correctness of the `isort` function, one need to provide a formalized proof for the theorem stated on line 16.

Similarly, the implementation of merge sort² from the Coq Standard Library³ also took a bottom-up approach, creating multiple smaller functions before using them in the `msort` function as shown in Listing A.3. To prove its correctness, a proof needs to be provided for the theorem stated in Line 33.

```

1 Fixpoint merge l1 l2 :=
2   let fix merge_aux l2 :=
3     match l1, l2 with
4     | [], _ ⇒ l2
5     | _, [] ⇒ l1
6     | a1::l1', a2::l2' ⇒
7       if a1 <=? a2 then a1 :: merge l1' l2 else a2 :: merge_aux l2'
8   end

```

²<https://coq.inria.fr/library/Coq.Sorting.Mergesort.html>

³<https://coq.inria.fr/library/>

```

9   in merge_aux l2.
10
11  Fixpoint merge_list_to_stack stack l :=
12    match stack with
13    | [] => [Some l]
14    | None :: stack' => Some l :: stack'
15    | Some l' :: stack' => None :: merge_list_to_stack stack' (merge l' l)
16  end.
17
18  Fixpoint merge_stack stack :=
19    match stack with
20    | [] => []
21    | None :: stack' => merge_stack stack'
22    | Some l :: stack' => merge l (merge_stack stack')
23  end.
24
25  Fixpoint iter_merge stack l :=
26    match l with
27    | [] => merge_stack stack
28    | a :: l' => iter_merge (merge_list_to_stack stack [a]) l'
29  end.
30
31  Definition msort := iter_merge [].
32
33  Theorem msort_correct : is_a_sorting_algorithm msort.

```

LISTING A.3: Merge sort from Coq Standard Library

There is a downside through this approach, that is, some of the intermediate functions such as `merge_list_to_stack`, `merge_stack` and `iter_merge_stack`, are created specifically just for the sorting function and have little to no use to other functions. Furthermore, from the viewpoint of program synthesis, it would be close to impossible for a machine to build smaller functions from bottom up to eventually complete a program with some stated specification.

Our work takes an opposite approach, that is, a top-down approach in creating a program. We start off by writing the specification as a proposition type in Coq and prove it. Due to the Curry-Howard proofs-as-programs correspondence (Howard, 1980), Coq's extraction tools (Filliâtre and Letouzey, 2004) allow us to extract working programs from the formalized proofs. To prove the proposition type, we create new generalized functions, called *tactics*, to break down the proposition type into simpler subgoals for users or machines to solve. Unlike the intermediate functions created through bottom-up approach which are specific to some specification, these tactics can be applied to any specification written as a proposition type in Coq as long as the input types match. Having such a top-down approach also mimics how human reasoning works, i.e., having an end goal or problem to solve and thinking of different strategies to break down our problem into smaller problems that are easier to solve. Furthermore, there has been a rising trend in developing automated reasoning tools for proof assistants such as Sledgehammer (Meng, Quigley, and Lawrence C Paulson, 2006) (for Isabelle), CoqHammer (Czajka and Kaliszyk, 2018) and Tactician (Blaauwbroek, Urban, and Geuvers, 2020) (both for Coq). These automated tools take a deductive

top-down approach search for tactics and functions to break down the given proposition type (Gulwani, Polozov, and Singh, 2017). Thus, creating generalized tactics also improves the proof automation capabilities of these tools.

For the rest of this chapter, we will explain how we used well-founded induction to derive and formalize different variations of the divide-and-conquer algorithm paradigm (Smith, 1985) as propositions in Coq. We will then apply these formalized paradigms on the sorting specification and generate different sorting algorithms. To our knowledge, we are the first to formalize these different divide-and-conquer algorithms. The code repository for this chapter can be found on <https://github.com/jinxinglim/coq-formalized-divide-and-conquer>.

A.1 Well-founded induction

Well-founded induction, sometimes called Noetherian induction (Nicolas, 1972), states that the following:

Theorem A.1.1 (Well-founded induction). *Suppose (X, R) is a well-founded relation (i.e., X has a minimal element with respect to relation R), where X is a set and R is a relation between any two elements in X , P is some proposition that holds for elements of X , if we want to prove P holds for all elements in X , one must show:*

- If x is an element of X and P holds for any y in X such that yRx , then P holds for x .

The following is the proposition type of the well-founded induction in Coq⁴:

```
1 Theorem well_founded_induction_type : forall (P : A → Type) (R : A → A → Prop),
2   (forall (x : A), (forall (y : A), R y x → P y) → P x)
3   → forall (a : A), P a.
```

LISTING A.4: Well-founded induction type in Coq

Depending on the X and R , we can have different forms of induction/inductive principle. For example, by letting X be the set of natural numbers, $\mathbb{N}$ and R be the preorder of $\mathbb{N}$, $<$, we can derive the well-known mathematical induction. Another example would be to let X be the set of lists containing elements of some type A and R be the relation of length between two lists $l1, l2$, i.e., $lengthOrder\ l1\ l2$ is defined as $length(l1) < length(l2)$, then we derive a well-founded induction for lists with respect to $lengthOrder$ (Chlipala, 2008). Note that the set (type in type theory) of lists with type A elements has a minimal element with respect to $lengthOrder$ (i.e., the empty list), and thus, the set of lists with type A elements and $lengthOrder$ fulfil the X and R in Theorem A.1.1.

Theorem A.1.2 (Well-founded induction for lists with respect to $lengthOrder$). *For some type A and proposition P , P holds for all lists of type A if the following holds:*

- If l is a list of type A and P holds for any list l' of type A where $lengthOrder\ l'\ l$ (i.e., $length(l') < length(l)$), then P holds for l .

⁴<https://coq.inria.fr/library/Coq.Init.Wf.html>

```

1 Definition lengthOrder (A : Type) (ls1 ls2 : list A) := length ls1 < length ls2.
2
3 Theorem lengthOrder_wf : forall (A : Type) (P : A → Type),
4   (forall (l : list A), (forall (l' : list A), lengthOrder l' l → P l') → P l)
5   → forall (ls : list A), P ls.

```

LISTING A.5: Well-founded induction for lists w.r.t *lengthOrder* in Coq

Theorem A.1.2 is important for us to derive different algorithm paradigms as induction principles for lists of some type A . We shall see how we can do that in the next section.

A.2 Different algorithm paradigms for lists

Lists is defined inductively in Coq (Listing A.6) and we will use the following notations for lists of a given type A will be used for the rest of the paper:

- $[:]$ represents empty list.
- $[: a_1; a_2; \dots; a_n]$ represents a list with terms $a_1, a_2, \dots, a_n$ of some type A .
- $(a :: l)$ represents a list with some term a as its head and some list l as its tail.

```

1 Inductive list (A : Type) :=
2   | nil : list
3   | cons : A → list → list.

```

LISTING A.6: Definition of lists in Coq

When defining a type inductively in Coq, induction principles of that type are automatically generated after the creation. For example, after defining lists as seen in Listing A.6, *list_rect* (Listing A.7) is generated automatically as list induction (Theorem A.2.1) for users to apply to prove other propositions, which we will see how in the next section.

```

1 Theorem list_rect : forall (A : Type) (P : list A → Type),
2   P nil
3   → (forall (a : A) (l : list A), P l → P (a :: l))
4   → forall ls : list A, P ls

```

LISTING A.7: List induction in Coq

Theorem A.2.1 (List induction). *Let P be some property and A be some type. P holds for all lists of type A if the following holds:*

1. P holds for $[:]$.
2. For any term a of type A , if P holds for a list l , then P holds for $(a :: l)$.

We shall see how Theorem A.2.1 is actually derivable from the well-founded induction for lists with respect to *lengthOrder* (Theorem A.1.2).

Proof. Suppose property 1 and 2 hold, we want to prove P holds for all lists of type A . For an arbitrary list ls , it can either be $[:]$ or $(a :: l)$ for some element a and list l . If ls is the former, by property 1, P holds for ls . If ls is $(a :: l)$, we know that $\text{length}(l) < 1 + \text{length}(l) = \text{length}(a :: l)$, which implies that $\text{lengthOrder } l (a :: l)$. Furthermore, property 2 implies that if P holds for l , then P holds for $(a :: l)$. Thus, P holds for any ls by Theorem A.1.2. $\square$

Other than the standard list induction, where it recursively check the property P for tail list tl after removing the head element hd , Theorem A.1.2 can be used to prove different induction principles. These induction variations, when applied to a proposition type stating the specification of a program, produce different proofs, and hence, algorithms. We will derive different induction principles, in particular variations of the well-known divide-and-conquer algorithm paradigm, before applying these inductions to the sort program specification in Coq to produce different sorting algorithms in the next section. We will prove the different variations of divide-and-conquer algorithm paradigm through pen-and-paper proofs but their formalized proofs can be found on the github repository <https://github.com/jinxinglim/coq-formalized-divide-and-conquer>.

Theorem A.2.2 (Pairs-and-conquer). *Let P be some property and A be some type. P holds for all lists of type A if the following holds:*

1. P holds for $[:]$.
2. P holds for single-element list, $[: a]$.
3. P holds for two-elements list, $[: a_1; a_2]$.
4. For any list l , if P holds for $[: a_1; a_2]$ and l , then P holds for $(a_1 :: a_2 :: l)$.

Proof. Suppose property 1 to 4 hold, we want to prove P holds for all lists of type A . For an arbitrary list ls , it can either be $[:]$, $[: a]$, $[: a_1; a_2]$ or $a_1 :: a_2 :: l$ for some elements a, a_1, a_2 and list l . In any of the first three cases, we have property 1 to 3 to prove that P holds for ls . Suppose ls is $(a_1 :: a_2 :: l)$. We know that $\text{lengthOrder } [: a_1; a_2] ls$ and $\text{lengthOrder } l ls$. Moreover, by property 4, if P holds for $[: a_1; a_2]$ and l , P holds for ls as well. Thus, P holds for any ls by Theorem A.1.2. $\square$

Theorem A.2.2 works similarly to the standard list induction (Theorem A.2.1). The difference is that instead of recursively removing a single head element a to check if the property P holds for the tail list l , it removes two head elements each time. Listing A.8 shows the proposition type of Theorem A.2.2 in Coq.

```

1 Theorem div_conq_pair : forall (A : Type) (P : list A → Type),
2   P []
3   → (forall a : A, P [a])
4   → (forall a1 a2 : A, P [:: a1; a2])
5   → (forall (a1 a2 : A) (l : list A), P [:: a1; a2] → P l → P (a1 :: a2 :: l))
6   → forall ls : list A, P ls

```

LISTING A.8: Pairs-and-conquer (Theorem A.2.2) in Coq

Next, we will see how we can prove and formalize the algorithm paradigm, divide-and-conquer, from Theorem A.1.2.

```

1 Theorem div_conq_split : forall (A : Type) (P : list A → Type),
2   P [::]
3   → (forall a : A, P [a])
4   → (forall l : list A, P (fst (split A l)) → P (snd (split A l)) → P l)
5   → forall ls : list A, P ls

```

LISTING A.9: Divide-and-conquer (Theorem A.2.3) in Coq

Theorem A.2.3 (Divide-and-conquer). *Let P be some property and A be some type. Suppose we have a 'split' function takes in a list as an input and produces two sub-lists, whose lengths are less than the input list, as outputs. P holds for all lists of type A if the following holds:*

1. P holds for $[::]$.
2. P holds for single-element list, $[:: a]$.
3. For any list l , if P holds for $\text{fst}(\text{split}(l))$ (i.e., first sub-list) and $\text{snd}(\text{split}(l))$ (i.e., second sub-list), then P holds for the list l .

Proof. Suppose property 1 to 3 hold, we want to prove P holds for all lists of type A . For an arbitrary list ls , it can either be $[::]$, $[:: a]$ for some element a or of the form other than the first two. In any of the first two cases, we have property 1 to 2 to prove that P holds for ls . Suppose ls is of form other than the first two. From the assumption, we know that both sub-lists from $\text{split}(ls)$'s output have length less than ls , which implies $\text{lengthOrder } \text{fst}(\text{split}(ls)) \text{ } ls$ and $\text{lengthOrder } \text{snd}(\text{split}(ls)) \text{ } ls$. And by property 3, we have if P holds for $\text{fst}(\text{split}(ls))$ and $\text{snd}(\text{split}(ls))$, P holds for ls as well. Thus, P holds for any ls by Theorem A.1.2. $\square$

The split function is kept arbitrary so that the prover/programmer can decide how a given list should be split. The only required specification is that the function must output two sub-lists whose each of their lengths is less than the length of the input list. Depending on the split function used, when applying Theorem A.2.3 (Listing A.9) to program specification as proposition type, it will generate different proofs, and hence, programs, which will affect the efficiency of the program implemented. We implemented the split function (Listing A.10) that splits an list into odd- and even-indexed sub-lists in our code.

```

1 Fixpoint split (ls : list A) : list A * list A :=
2   match ls with
3   | nil ⇒ (nil, nil)
4   | h :: nil ⇒ (h :: nil, nil)
5   | h1 :: h2 :: ls' ⇒ let (ls1, ls2) := split ls' in
6     (h1 :: ls1, h2 :: ls2)
7   end.

```

LISTING A.10: split function that splits an list into odd- and even-indexed sub-lists in Coq

Lastly, we will derive a variation of the divide-and-conquer algorithm, where the *split* function, beside taking an input list, takes in a pivot that will eventually help in splitting the list into two sub-lists.

Theorem A.2.4 (Divide-and-conquer with pivot). *Let P be some property and A be some type. Suppose we have a 'split_pivot' function takes in an input list and a pivot to produce two sub-lists, according to each of the elements' relation with the pivot and whose lengths are less than or equal to the input list, as outputs. P holds for all lists of type A if the following holds:*

1. P holds for $[:]$.
2. P holds for single-element list, $[:: a]$.
3. For any list l , if P holds for $\text{fst}(\text{split_pivot}(a, l))$ (i.e., first sub-list) and $\text{snd}(\text{split_pivot}(a, l))$ (i.e., second sub-list), then P holds for the list $(a :: l)$.

Proof. Suppose property 1 to 3 hold, we want to prove P holds for all lists of type A . For an arbitrary list ls , it can either be $[:]$, $[:: a]$ or $(a :: l)$ for some element a . In any of the first two cases, we have property 1 to 2 to prove that P holds for ls . Suppose $ls = (a :: l)$. From the assumption, we know that both sub-lists from *split_pivot*'s output have length less than or equal to l , which implies both's lengths are less than that of $ls = (a :: l)$. So we have $\text{lengthOrder } \text{fst}(\text{split_pivot}(a, l)) \text{ } ls$ and $\text{lengthOrder } \text{snd}(\text{split_pivot}(a, l)) \text{ } ls$. And by property 3, we have if P holds for $\text{fst}(\text{split_pivot}(a, l))$ and $\text{snd}(\text{split_pivot}(a, l))$, P holds for $(a :: l) = ls$ as well. Thus, P holds for any ls by Theorem A.1.2. $\square$

Listing A.11 shows the *split_pivot* function used in our code and the formalization of Theorem A.2.4. The relation `le` in line 1 is meant to be kept as arbitrary so that prover/programmer can choose the relationship between the pivot and the list they are looking out for. Line 2 extends the relation `le` to `le_dec` to say that this relation is decidable (i.e., for any two elements a_1, a_2 of type A , either $le \ a_1 \ a_2$ or $le \ a_2 \ a_1$).

```

1 Variable le: A → A → Prop.
2 Variable le_dec: forall (x y: A), {le x y} + {le y x}.
3
4 Fixpoint split_pivot (pivot: A) l: list A * list A :=
5   match l with
6   | nil => (nil, nil)
7   | a :: l' => let (l1, l2) := (split_pivot pivot l') in
8     if le_dec a pivot
9     then (a :: l1, l2) else (l1, a :: l2)
10  end.
11
12 Theorem div_conq_pivot: forall (A: Type) (P: list A → Type),
13   P []
14   → (forall (a: A) (l: list A),
15     P (fst (split_pivot a l)) → P (snd (split_pivot a l)) → P (a :: l))
16   → forall ls: list A, P ls

```

LISTING A.11: Divide-and-conquer with pivot (Theorem A.2.4) in Coq

A.3 Sorting algorithms based on different inductive principles

After going through the different variations of divide-and-conquer algorithm as inductive principles, we shall see how when applied to the specification of a sorting program (for lists of natural numbers) written as proposition type in Coq (Theorem A.3.1 and Listing A.12), different proofs, and hence, sorting algorithms are produced. Figure A.1 shows the links from the well-founded induction (Theorem A.1.1) to the different sorting algorithms.

Theorem A.3.1 (Sort program specification). *For any lists of natural numbers, l , there exists a list l' such that it is sorted (by ascending order) and is a permutation of l .*

```
1 Theorem sort_prog : forall (l : list nat),
2   {l' : list nat | sorted l' ^ permutation l' l}.
```

LISTING A.12: Specification of sorting program as proposition type in Coq

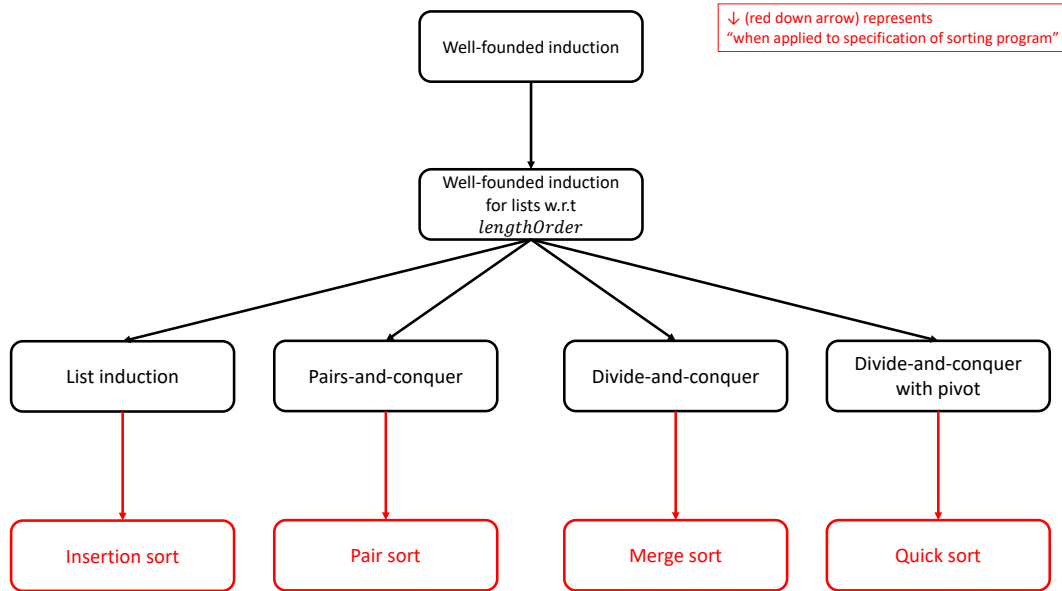


FIGURE A.1: Summary of how well-found induction eventually leads to different sorting algorithms

As mentioned in Chapter 2, Coq the proof assistant allow users to guide and assist the proof engine to derive the proof by displaying current sub-goals to prove through Coq Integrated Development Environment (CoqIDE)⁵. After applying `induction l` (i.e., Theorem A.2.1, list induction) to `sort_prog` in Coq (Listing A.13), it will output the current (smaller) sub-goals to be solved (Listing A.14). The two sub-goals to be proven are:

1. (Base case) There exists a list l' such that it is sorted and is a permutation of the empty list `[]`

⁵<https://coq.inria.fr/refman/practical-tools/coqide.html>

2. (Inductive case) For any natural number, a , and list l , if there exists a list l' such that it is sorted and is a permutation of l , then there exists a list l'' such that it is sorted and is a permutation of $(a :: l)$.

```

1 Theorem sort_prog: forall (l: list nat),
2   {l' : list nat | sorted l' ^ permutation l' l}.
3 Proof.
4 induction l.

```

LISTING A.13: Applying list induction (Theorem A.2.1) to *sort_prog*

```

1 2 subgoals
2                                     (1/2)
3 {l' : list nat | sorted l' ^ permutation l' []}
4                                     (2/2)
5 forall (a: nat) (l: list nat),
6 {l' : list nat | sorted l' ^ permutation l' l}
7 → {l' : list nat | sorted l' ^ permutation l' (a :: l)}

```

LISTING A.14: Coq's output after applying *induction l*

Our code repository (i.e., the file `isort.v`) contains the completed proof of the initial subproof from Listing A.13. As Coq is built upon dependent type theory, a constructive logic, the proof provides us the necessary steps to construct the existential output l' . With Coq's extraction tool, we are able to extract the completed proof to OCaml programming language as a working program. Through this top-down approach in generating proof in Coq, it not only generates the necessary program but also verifies its correctness. Furthermore, while generating a proof, every proof of intermediate lemmas along the process can be extracted as programs as well. For example, the second sub-goal (i.e., the inductive case) states the specification of the `insert` function seen in Listing A.1. The completed proof of this sub-goal can be extracted as a working *insert* function in OCaml. Thus, such top-down approach also provides us a procedure to find the necessary intermediate functions for that program.

Instead of applying `induction l` to the `sort_prog`, we can apply other inductive principles that we have created in the previous section. For example, after applying the inductive principle `div_conq_split` to the `sort_prog` (Listing A.15), the three sub-goals required to be proven (Listing A.16) are:

1. (Base case) There exists a list l' such that it is sorted and is a permutation of the empty list `[]`
2. (One-element case) For any a of type A , there exists a list l' such that it is sorted and is a permutation of the one-element list `[a]`
3. (Inductive case) For any list ls , if there exist lists l' and l'' such that both are sorted, l' is a permutation of $fst(split\ nat\ ls)$ and l'' is a permutation of $snd(split\ nat\ ls)$, then there exists a list l''' such that it is sorted and is a permutation of ls .

```

1 Theorem sort_prog: forall (l: list nat),
2   {l' : list nat | sorted l' ^ permutation l' l}.
3 Proof.
4 apply div_conq_split.

```

LISTING A.15: Applying list induction (Theorem A.2.1) to *sort_prog*

```

1 3 subgoals
2 _____(1/3)
3 {l' : list nat | sorted l' ^ permutation l' []}
4 _____(2/3)
5 forall a:nat, {l' : list nat | sorted l' ^ permutation l' [a]}
6 _____(3/3)
7 forall ls: list nat,
8 {l' : list nat | sorted l' ^ permutation l' (fst (split nat ls))} →
9 {l' : list nat | sorted l' ^ permutation l' (snd (split nat ls))} →
10 {l' : list nat | sorted l' ^ permutation l' ls}

```

LISTING A.16: Coq's output after applying *div_conq_split*

Extraction of the complete proof of the initial subproof in Listing A.15 (in the file `msort.v`) gives the merge sort (Donald et al., 1999) we commonly know. As seen in Figure A.1, different inductive principles will lead to different sorting algorithms, where the proofs and their extracted programs can be found in our code repository. As we know, different sorting algorithms will lead to different efficiencies in sorting a list. In the code repository, we also include a OCaml file to test the speed of sorting a randomly generated list from different sorting algorithms that we extracted.

Bibliography

- Appel, Andrew W. (2021). *Verified Functional Algorithms*. Ed. by Benjamin C. Pierce. Vol. 3. Software Foundations. Version 1.5.1, <http://softwarefoundations.cis.upenn.edu>. Electronic textbook.
- Arias, Emilio Jesús Gallego, Benoît Pin, and Pierre Jouvelot (2017). “jsCoq: Towards hybrid theorem proving interfaces”. In: *arXiv preprint arXiv:1701.07125*.
- Asgaonkar, Aditya and Bhaskar Krishnamachari (2018). “Token curated registries-a game theoretic approach”. In: *arXiv preprint arXiv:1809.01756*.
- Aung, Yu Nandar and Thitinan Tantidham (2017). “Review of Ethereum: Smart home case study”. In: *2017 2nd International Conference on Information Technology (INCIT)*. IEEE, pp. 1–4.
- Avigad, Jeremy and John Harrison (2014). “Formally verified mathematics”. In: *Communications of the ACM* 57.4, pp. 66–75.
- Avyukt, Anusha, Gowri Ramachandran, and Bhaskar Krishnamachari (2021). “A Decentralized Review System for Data Marketplaces”. In: *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–9. DOI: [10.1109/ICBC51069.2021.9461149](https://doi.org/10.1109/ICBC51069.2021.9461149).
- Babaioff, Moshe et al. (2012). “On bitcoin and red balloons”. In: *Proceedings of the 13th ACM conference on electronic commerce*, pp. 56–73.
- Banerjee, Siddhartha, Ashish Goel, and Anilesh Kollagunta Krishnaswamy (2014). “Reincentivizing discovery: mechanisms for partial-progress sharing in research”. In: *Proceedings of the fifteenth ACM conference on Economics and computation*, pp. 149–166.
- Benet, Juan (2014). “Ipfs-content addressed, versioned, p2p file system”. In: *arXiv preprint arXiv:1407.3561*.
- Benisi, Nazanin Zahed, Mehdi Aminian, and Bahman Javadi (2020). “Blockchain-based decentralized storage networks: A survey”. In: *Journal of Network and Computer Applications* 162, p. 102656.
- Bertot, Yves (2008). “A short presentation of Coq”. In: *International Conference on Theorem Proving in Higher Order Logics*. Springer, pp. 12–16.
- Bjesse, Per (2005). “What is formal verification?” In: *ACM SIGDA Newsletter* 35.24, 1–es.
- Blaauwbroek, Lasse, Josef Urban, and Herman Geuvers (2020). “The Tactician”. In: *International Conference on Intelligent Computer Mathematics*. Springer, pp. 271–277.
- Blum, Manuel, Paul Feldman, and Silvio Micali (2019). “Non-interactive zero-knowledge and its applications”. In: *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*, pp. 329–349.
- Boyer, Robert (1994). “The QED manifesto”. In: *Automated Deduction—CADE 12*, pp. 238–251.
- Breidenbach, Lorenz et al. (2017). *An In-Depth Look at the Parity Multisig Bug*. <https://hackingdistributed.com/2017/07/22/deep-dive-parity-bug/>. Accessed: 2022-01-10.

- Brown, Chad E. (2014). “The Egal Manual”. In: URL: <http://grid01.ciirc.cvut.cz/~chad/egalmanual.pdf>.
- (2020). *A theory supporting higher-order abstract syntax*. Tech. rep. Tech. rep., Czech Technical University in Prague (Aug 2020).
- Buterin, Vitalik (Aug. 2020). *Trust Models*. URL: <https://vitalik.ca/general/2020/08/20/trust.html>.
- (Aug. 2021). *Moving beyond coin voting governance*. URL: <https://vitalik.ca/general/2021/08/16/voting3.html>.
- Buterin, Vitalik and Virgil Griffith (2017). “Casper the friendly finality gadget”. In: *arXiv preprint arXiv:1710.09437*.
- Buterin, Vitalik, Diego Hernandez, et al. (2020). “Combining GHOST and Casper”. In: *arXiv preprint arXiv:2003.03052*.
- Carré, Sylvain et al. (2021). “Smart Proofs via Smart Contracts: Succinct and Informative Mathematical Derivations via Decentralized Markets”. In: *arXiv preprint arXiv:2102.03044*.
- Castro, Miguel, Barbara Liskov, et al. (1999). “Practical byzantine fault tolerance”. In: *OsDI*. Vol. 99. 1999, pp. 173–186.
- Charanya, R and M Aramudhan (2016). “Survey on access control issues in cloud computing”. In: *2016 International Conference on Emerging Trends in Engineering, Technology and Science (ICETETS)*. IEEE, pp. 1–4.
- Chauhan, Anamika et al. (2018). “Blockchain and scalability”. In: *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, pp. 122–128.
- Chlipala, Adam (2008). “Certified programming with dependent types”. In: <http://adam.chlipala.net/cpdt>.
- Church, Alonzo (1932). “A set of postulates for the foundation of logic”. In: *Annals of mathematics*, pp. 346–366.
- Community, The mathlib (2020). “The Lean Mathematical Library”. In: *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP 2020. New Orleans, LA, USA: Association for Computing Machinery, pp. 367–381. ISBN: 9781450370974. DOI: [10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824). URL: <https://doi.org/10.1145/3372885.3373824>.
- Cranshaw, Justin and Aniket Kittur (2011). “The polymath project: lessons from a successful online collaboration in mathematics”. In: *Proceedings of the SIGCHI conference on human factors in computing systems*, pp. 1865–1874.
- Czajka, Łukasz and Cezary Kaliszyk (2018). “Hammer for Coq: Automation for dependent type theory”. In: *Journal of automated reasoning* 61.1-4, pp. 423–453.
- De Moura, Leonardo and Nikolaj Bjørner (2008). “Z3: An efficient SMT solver”. In: *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, pp. 337–340.
- Devlin, Keith J (2000). *The math gene: How mathematical thinking evolved and why numbers are like gossip*. Vol. 329. Basic Books New York.
- Donald, E Knuth et al. (1999). “The art of computer programming”. In: *Sorting and searching* 3, pp. 426–458.
- Duncan, Ross et al. (2020). “Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus”. In: *Quantum* 4, p. 279.

- Eberhardt, Jacob and Stefan Tai (2018). "Zokrates-scalable privacy-preserving off-chain computations". In: *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. IEEE, pp. 1084–1091.
- Fagin, Ronald et al. (2003). *Reasoning about knowledge*. MIT press.
- Filliâtre, Jean-Christophe (2011). "Deductive software verification". In: *International Journal on Software Tools for Technology Transfer* 13.397.
- Filliâtre, Jean-Christophe and Pierre Letouzey (2004). "Functors for proofs and programs". In: *European Symposium on Programming*. Springer, pp. 370–384.
- Fong, Brendan and David I Spivak (2018). "Seven sketches in compositionality: An invitation to applied category theory". In: *arXiv preprint arXiv:1803.05316*.
- Friesike, Sascha and Benedikt Fecher (2016). "Collaboration, participation and transparency: the promise of digitizing academic research". In: *Research Handbook on Digital Transformations*. Edward Elgar Publishing.
- Gauthier, Thibault, Cezary Kaliszyk, and Josef Urban (2018). "Learning to Reason with HOL4 tactics". In: *arXiv preprint arXiv:1804.00595*.
- Gebron, Hunter (2017). *adChain: Introducing the adChain Registry!* <https://medium.com/metax-publication/introducing-the-adchain-registry-cc5b8b831a7e>. Accessed: 2022-02-09.
- Gewin, Virginia (2019). "How the scientific meeting has changed since Nature's founding 150 years ago". In: *Nature* 576.7787, S70–S70.
- Ghosh, Arpita and Jon Kleinberg (2013). "Incentivizing participation in online forums for education". In: *Proceedings of the fourteenth ACM conference on Electronic commerce*, pp. 525–542.
- Goldin, Mike (2017). *Token-Curated Registries 1.0*. https://docs.google.com/document/d/1BWWC__Kms09b7yCI_R7ysoGFIT9D_sfjH3axQsmB6E/edit. Accessed: 2021-01-20.
- Goldwasser, Shafi, Silvio Micali, and Charles Rackoff (1989). "The knowledge complexity of interactive proof systems". In: *SIAM Journal on computing* 18.1, pp. 186–208.
- Golosova, Julija and Andrejs Romanovs (2018). "The advantages and disadvantages of the blockchain technology". In: *2018 IEEE 6th workshop on advances in information, electronic and electrical engineering (AIEEE)*. IEEE, pp. 1–6.
- Gonthier, Georges et al. (2008). "Formal proof—the four-color theorem". In: *Notices of the AMS* 55.11, pp. 1382–1393.
- Gonthier, Georges and Assia Mahboubi (2010). "An introduction to small scale reflection in Coq". In: *Journal of formalized reasoning* 3.2, pp. 95–152.
- Gowers, Tim (2009). *Is massively collaborative mathematics possible?* URL: <https://gowers.wordpress.com/2009/01/27/is-massively-collaborative-mathematics-possible/>.
- Green, Cordell (1969). "Application of theorem proving to problem solving". In: *Proceedings of the 1st international joint conference on Artificial intelligence*, pp. 219–239.
- Gresham, David (July 2021). *Non fungible tokens (NFTs) for academic publications?* URL: <https://gencore.bio.nyu.edu/nfts-for-academic-publications/>.

- Gulwani, Sumit (2010). "Dimensions in program synthesis". In: *Proceedings of the 12th international ACM SIGPLAN symposium on Principles and practice of declarative programming*, pp. 13–24.
- Gulwani, Sumit, William R Harris, and Rishabh Singh (2012). "Spreadsheet data manipulation using examples". In: *Communications of the ACM* 55.8, pp. 97–105.
- Gulwani, Sumit, Alex Polozov, and Rishabh Singh (2017). *Program Synthesis*. Vol. 4. NOW, pp. 1–119. URL: <https://www.microsoft.com/en-us/research/publication/program-synthesis/>.
- Hamburg, Sarah (Feb. 2022). *A Guide to DeSci, the Latest Web3 Movement*. URL: <https://future.com/what-is-decentralized-science-aka-desci/>.
- Han, Jesse Michael et al. (2021). "Proof Artifact Co-training for Theorem Proving with Language Models". In: *arXiv preprint arXiv:2102.06203*.
- Harrison, John, Josef Urban, and Freek Wiedijk (2014). "History of Interactive Theorem Proving." In: *Computational Logic*. Vol. 9, pp. 135–214.
- Hartnett, Kevin (2021). *Proof Assistant Makes Jump to Big-League Math*. <https://www.quantamagazine.org/lean-computer-program-confirms-peter-scholze-proof-20210728/>. Accessed: 2022-04-19.
- Hirsch, Jorge E (2005). "An index to quantify an individual's scientific research output". In: *Proceedings of the National academy of Sciences* 102.46, pp. 16569–16572.
- Hoskinson, Charles (2017). "Cardano white paper". In: URL: <https://whitepaper.io/document/581/cardano-whitepaper>.
- Hotz, Günter (1965). "Eine Algebraisierung des Syntheseproblems von Schaltkreisen I." In: *Elektronische Informationsverarbeitung und Kybernetik* 1.3, pp. 185–205.
- Howard, William A (1980). "The formulae-as-types notion of construction". In: *To HB Curry: essays on combinatory logic, lambda calculus and formalism* 44, pp. 479–490.
- Ito, Kensuke and Hideyuki Tanaka (2019). "Token-Curated Registry with Citation Graph". In: *arXiv preprint arXiv:1906.03300*.
- Al-Jaroodi, Jameela and Nader Mohamed (2019). "Blockchain in industries: A survey". In: *IEEE Access* 7, pp. 36500–36515.
- Jiang, Yaqing, Petros Papapanagiotou, and Jacques Fleuriot (2018). "Machine learning for inductive theorem proving". In: *International Conference on Artificial Intelligence and Symbolic Computation*. Springer, pp. 87–103.
- Jones, Nicola (June 2021). *How scientists are embracing NFTs*. URL: <https://www.nature.com/articles/d41586-021-01642-3>.
- Kerber, Manfred, Christoph Lange, and Colin Rowat (2016). "An introduction to mechanized reasoning". In: *Journal of Mathematical Economics* 66, pp. 26–39.
- King, Jerry P (2006). *The art of mathematics*. Courier Corporation.
- Kosmarski, Artyom and Nikolay Gordiyuchuk (2020). "Token-curated registry (TCR) in a scholarly journal: blockchain meets invisible colleges". In: *arXiv preprint arXiv:2005.11534*.
- Kunen, Kenneth (2007). *The foundations of mathematics*.
- Kurmi, Jitendra and Ankur Sodhi (2015). "A survey of zero-knowledge proof for authentication". In: *International Journal of Advanced Research in Computer Science and Software Engineering* 5.1.
- Kwon, Jae and Ethan Buchman (2018). "A Network of Distributed Ledgers". In: Labs., Storj (2018). "Storj: A Decentralized Cloud Storage Network Framework". In:

- Lambert, Nick (2021). "Beyond NFTs: A Possible Future for Digital Art". In: *ITNOW* 63.3, pp. 8–10.
- Leroy, Xavier et al. (2016). "CompCert-a formally verified optimizing compiler". In: *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*.
- Li, Elaine et al. (2020). "Formalizing Correct-by-Construction Casper in Coq". In: *2020 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, pp. 1–3.
- Lim, Jin Xing et al. (2021). "A Blockchain-Based Approach for Collaborative Formalization of Mathematics and Programs". In: *2021 IEEE International Conference on Blockchain (Blockchain)*. IEEE, pp. 321–326.
- Mackey, Tim K et al. (2019). "A framework proposal for blockchain-based scientific publishing using shared governance". In: *Frontiers in Blockchain* 2, p. 19.
- Manna, Zohar and Richard Waldinger (1980). "A deductive approach to program synthesis". In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 2.1, pp. 90–121.
- Manna, Zohar and Richard J Waldinger (1971). "Toward automatic program synthesis". In: *Communications of the ACM* 14.3, pp. 151–165.
- Martin-Löf, Per and Giovanni Sambin (1984). *Intuitionistic type theory*. Vol. 9. Bibliopolis Naples.
- Mehar, Muhammad Izhar et al. (2019). "Understanding a revolutionary and flawed grand experiment in blockchain: the DAO attack". In: *Journal of Cases on Information Technology (JCIT)* 21.1, pp. 19–32.
- Meng, Jia, Claire Quigley, and Lawrence C Paulson (2006). "Automation for interactive proof: First prototype". In: *Information and computation* 204.10, pp. 1575–1596.
- Moura, Leonardo de et al. (2015). "The Lean theorem prover (system description)". In: *International Conference on Automated Deduction*. Springer, pp. 378–388.
- Nakamoto, Satoshi (2008). "Bitcoin: A peer-to-peer electronic cash system". In: *Decentralized Business Review*, p. 21260.
- Nawaz, M Saqib et al. (2019). "A survey on theorem provers in formal methods". In: *arXiv preprint arXiv:1912.03028*.
- Nicolas, Bourbaki (1972). *Elements of Mathematics Commutative Algebra*.
- Optimism and Vitalik Buterin (July 2021). *Retroactive Public Goods Funding*. URL: <https://medium.com/ethereum-optimism/retroactive-public-goods-funding-33c9b7d00f0c>.
- Osborne, Martin J and Ariel Rubinstein (1994). *A course in game theory*. MIT press.
- Patterson, Evan, David I Spivak, and Dmitry Vagner (2021). "Wiring diagrams as normal forms for computing in symmetric monoidal categories". In: *arXiv preprint arXiv:2101.12046*.
- Paulson, Lawrence and Jasmin Blanchette (Feb. 2015). "Three Years of Experience with Sledgehammer, a Practical Link between Automatic and Interactive Theorem Provers". In: DOI: [10.29007/tnfd](https://doi.org/10.29007/tnfd).
- Paulson, Lawrence Charles (2020). "Machine Learning and the Formalisation Of Mathematics: Research Challenges". Invited talk at 5th Conference on Artificial Intelligence and Theorem Proving (AITP 2020).
- Penrose, Roger (1971a). "Angular momentum: an approach to combinatorial space-time". In: *Quantum theory and beyond*, pp. 151–180.

- Penrose, Roger (1971b). "Applications of negative dimensional tensors". In: *Combinatorial mathematics and its applications* 1, pp. 221–244.
- Peter, R (2014). "Spin-offs: Bootstrap your alt-coin with a Bitcoin-blockchain-based initial coin distribution". In: Retrieved Jan 28, p. 2020.
- Pnueli, Amir and Roni Rosner (1989). "On the synthesis of a reactive module". In: *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pp. 179–190.
- Polymath, D. H. J. (2012). "A new proof of the density Hales-Jewett theorem". In: *Annals of Mathematics*, pp. 1283–1327.
- (2014a). *The "bounded gaps between primes" Polymath project - a retrospective*. arXiv: [1409.8361 \[math.HO\]](https://arxiv.org/abs/1409.8361).
- (2014b). "Variants of the Selberg sieve, and bounded intervals containing many primes". In: *Research in the Mathematical sciences* 1.1, pp. 1–83.
- Poon, Joseph and Vitalik Buterin (2017). "Plasma: Scalable autonomous smart contracts". In: *White paper*, pp. 1–47.
- Poon, Joseph and Thaddeus Dryja (2016). *The bitcoin lightning network: Scalable off-chain instant payments*.
- Quisquater, Jean-Jacques et al. (1989). "How to explain zero-knowledge protocols to your children". In: *Conference on the Theory and Application of Cryptology*. Springer, pp. 628–631.
- Raatikainen, Panu (2013). *Gödel's incompleteness theorems*. URL: <https://stanford.library.sydney.edu.au/entries/goedel-incompleteness/>.
- Rashid, Adnan et al. (2017). "Formal reasoning about systems biology using theorem proving". In: *PloS one* 12.7, e0180179.
- Riazanov, Alexandre and Andrei Voronkov (2002). "The design and implementation of VAMPIRE". In: *AI communications* 15.2, 3, pp. 91–110.
- Sanchez-Stern, Alex et al. (2020). "Generating correctness proofs with neural networks". In: *Proceedings of the 4th ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pp. 1–10.
- Sariboz, Emrah et al. (2021). "Off-chain Execution and Verification of Computationally Intensive Smart Contracts". In: *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*. IEEE, pp. 1–3.
- Sasson, Eli Ben et al. (2014). "Zerocash: Decentralized anonymous payments from bitcoin". In: *2014 IEEE Symposium on Security and Privacy*. IEEE, pp. 459–474.
- Scholze, Peter (2020). *Liquid tensor experiment*. <https://xenaproject.wordpress.com/2020/12/05/liquid-tensor-experiment/>. Accessed: 2022-04-19.
- (2021). "Liquid tensor experiment". In: *Experimental Mathematics*, pp. 1–6.
- Schulz, Stephan (2002). "E—a brainiac theorem prover". In: *Ai Communications* 15.2, 3, pp. 111–126.
- Selkis, Ryan and Dan McArdle (2018). *Messari Token-Curated Registry*. <https://messari.github.io/tcr/whitepaper.pdf>. Accessed: 2022-02-09.
- Sergey, Ilya, James R Wilcox, and Zachary Tatlock (2017). "Programming and proving with distributed protocols". In: *Proceedings of the ACM on Programming Languages* 2.POPL, pp. 1–30.

- Shaker, Monir, Fereidoon Shams Aliee, and Reza Fotohi (2021). "Online rating system development using blockchain-based distributed ledger technology". In: *Wireless Networks* 27.3, pp. 1715–1737.
- Shakhbulatov, Denisolt et al. (2020). "How blockchain enhances supply chain management: A survey". In: *IEEE Open Journal of the Computer Society* 1, pp. 230–249.
- Shaw, David E, William R Swartout, and C Cordell Green (1975). "Inferring LISP Programs From Examples." In: *IJCAI*. Vol. 75, pp. 260–267.
- Smith, Douglas R (1985). "The design of divide and conquer algorithms". In: *Science of Computer Programming* 5, pp. 37–58.
- Solar-Lezama, Armando and Rastislav Bodik (2008). *Program synthesis by sketching*. Cite-seer.
- Stephenson, Matthew (2021). "Glyphs: Knowledge Tokens on the Blockchain (Non-technical 2018 Litepaper)". In: *Researchers.One*. URL: <https://researchers.one/articles/21.03.00002v1>.
- Su, Borching (2018). "MathCoin: A Blockchain Proposal that Helps Verify Mathematical Theorems In Public." In: *IACR Cryptol. ePrint Arch.* 2018, p. 271.
- Summers, Phillip D (1977). "A methodology for LISP program construction from examples". In: *Journal of the ACM (JACM)* 24.1, pp. 161–175.
- Sun, Xiaoqiang et al. (2021). "A survey on zero-knowledge proof in blockchain". In: *IEEE Network* 35.4, pp. 198–205.
- Swan, Melanie (2015). *Blockchain: Blueprint for a new economy*. "O'Reilly Media, Inc."
- Taylor, Richard and Andrew Wiles (1995). "Ring-theoretic properties of certain Hecke algebras". In: *Annals of Mathematics*, pp. 553–572.
- Tolmach, Palina et al. (2021). "A survey of smart contract formal specification and verification". In: *ACM Computing Surveys (CSUR)* 54.7, pp. 1–38.
- Trovò, Bianca and Nazzareno Massari (2021). "Ants-Review: a Protocol for Incentivized Open Peer-Reviews on Ethereum". In: *CoRR abs/2101.09378*. arXiv: [2101.09378](https://arxiv.org/abs/2101.09378). URL: <https://arxiv.org/abs/2101.09378>.
- Vorick, David and Luke Champine (2014). "Sia: Simple decentralized storage". In: *Retrieved May 8*, p. 2018.
- Wang, Qin et al. (2021). "Non-Fungible Token (NFT): Overview, Evaluation, Opportunities and Challenges". In: *CoRR abs/2105.07447*. arXiv: [2105.07447](https://arxiv.org/abs/2105.07447). URL: <https://arxiv.org/abs/2105.07447>.
- Weiss, Kyle (Oct. 2021). *Gitcoin Grants - Quadratic Funding for the World*. URL: <https://gitcoin.co/blog/gitcoin-grants-quadratic-funding-for-the-world/>.
- Wenzel, Makarius, Lawrence C Paulson, and Tobias Nipkow (2008). "The isabelle framework". In: *International Conference on Theorem Proving in Higher Order Logics*. Springer, pp. 33–38.
- Weyns, Danny et al. (2012). "A survey of formal methods in self-adaptive systems". In: *Proceedings of the Fifth International C* Conference on Computer Science and Software Engineering*, pp. 67–79.
- White, B (2016). "Qeditas: A formal library as a bitcoin spin-off (2016)". In: <http://qeditas.org/docs/qeditas.pdf>.
- Wiles, Andrew (1995). "Modular elliptic curves and Fermat's last theorem". In: *Annals of mathematics* 141.3, pp. 443–551.

- Wood, G (2015). "PoA private chains: GitHub". In: WWW Document], GitHub, available at: <https://github.com/ethereum/guide/blob/master/poa.md> (accessed 30 December 2019).
- Wood, Gavin et al. (2014). "Ethereum: A secure decentralised generalised transaction ledger". In: *Ethereum project yellow paper* 151.2014, pp. 1–32.
- Wu, Yi-Chi, Ching-Sung Chen, and Yu-Jiun Chan (2020). "The outbreak of COVID-19: An overview". In: *Journal of the Chinese medical association* 83.3, p. 217.
- Yang, Kaiyu and Jia Deng (2019). "Learning to prove theorems via interacting with proof assistants". In: *arXiv preprint arXiv:1905.09381*.
- Zeng, Wenying et al. (2009). "Research on cloud storage architecture and key technologies". In: *Proceedings of the 2nd International Conference on Interaction Sciences: Information Technology, Culture and Human*, pp. 1044–1048.